# Core Java & OOP — 47 Interview Q&A (Senior-Level, Interview Perspective)

## Q1. What is Java?

**Summary:** Java is a high-level, object-oriented, platform-independent language that compiles to bytecode and runs on the JVM — making it one of the most widely adopted languages for enterprise backend systems.

**Practical Explanation:** In real projects, Java gives us strong typing, mature ecosystem, and write-once-run-anywhere capability. It's not just a language — it's a full ecosystem with frameworks like Spring Boot, build tools like Maven/Gradle, and a massive community.

**When/Why Used:** We use Java extensively in microservices, REST APIs, banking systems, e-commerce platforms — anywhere you need reliability, scalability, and long-term maintainability.

**Real-World Scenario:** In my projects, we've built payment processing microservices in Java using Spring Boot. The strong type system catches bugs at compile time, and the JVM's performance tuning capabilities handle millions of transactions daily.

**Common Mistakes:** Developers sometimes confuse Java with JavaScript — they're completely different. Also, people think Java is slow — modern JVMs with JIT compilation make it extremely performant.

**Closing:** Java remains the backbone of enterprise development, and its ecosystem maturity is unmatched for building scalable backend systems.

---

## Q2. Why is Java platform-independent?

**Summary:** Java achieves platform independence through bytecode — the compiler produces `.class` files that any JVM on any OS can execute, following the "Write Once, Run Anywhere" principle.

**Practical Explanation:** When we compile Java code, `javac` doesn't produce machine-specific binary — it produces bytecode. The JVM installed on each platform interprets or JIT-compiles that bytecode into native machine code at runtime. So the same `.jar` file runs on Linux, Windows, or Mac without recompilation.

**When/Why Used:** This is critical in enterprise environments where development happens on Mac/Windows but production runs on Linux servers. We build locally, run the same artifact in Docker containers on Linux in production — zero platform issues.

**Real-World Scenario:** In CI/CD pipelines, we build the JAR once on a Jenkins agent (could be any OS), and deploy the same artifact across dev, staging, and production environments running different Linux distributions. No recompilation needed.

**Common Mistakes:** People say "Java is platform-independent" — technically, Java code is platform-independent, but the JVM itself is platform-dependent. Each OS needs its own JVM installation.

**Closing:** Platform independence is one of Java's strongest value propositions and a key reason enterprises standardize on it for cross-environment deployments.

---

## Q3. What is JVM?

**Summary:** The JVM (Java Virtual Machine) is the runtime engine that executes Java bytecode, manages memory, performs garbage collection, and applies runtime optimizations like JIT compilation.

**Practical Explanation:** The JVM is not just an interpreter — it's a sophisticated runtime. It loads classes via the ClassLoader, verifies bytecode, allocates memory in Heap and Stack, runs the Garbage Collector, and uses Just-In-Time (JIT) compilation to convert hot bytecode paths into optimized native code.

**When/Why Used:** Every Java application runs on the JVM. As senior developers, we need to understand JVM internals for performance tuning — setting heap sizes (`-Xmx`, `-Xms`), choosing GC algorithms, analyzing thread dumps and heap dumps.

**Real-World Scenario:** In production, we've tuned JVM parameters like `-Xmx4g -XX:+UseG1GC -XX:MaxGCPauseMillis=200` to optimize a high-throughput order processing service. Understanding JVM memory model helped us diagnose an OutOfMemoryError caused by a connection pool leak.

**Common Mistakes:** Confusing JVM, JRE, and JDK. Also, not monitoring JVM metrics in production — GC pause times, heap usage, thread counts — is a common oversight that leads to production incidents.

**Closing:** Deep JVM knowledge separates senior engineers from juniors — it's essential for writing performant, production-ready Java applications.

---

## Q4. JDK vs JRE?

**Summary:** JDK is the full development kit (compiler + tools + JRE), while JRE is just the runtime environment needed to run Java applications.

**Practical Explanation:** JDK includes `javac` (compiler), `jdb` (debugger), `jconsole`, `jvisualvm`, and other development/monitoring tools plus the JRE. JRE includes the JVM and standard class libraries — everything needed to run compiled Java programs, but nothing to develop them.

**When/Why Used:** On developer machines, we install JDK because we need to compile and debug. On production servers or Docker images, we traditionally needed only JRE — though with modern containerized deployments, we often use slim JDK images or custom runtimes via `jlink`.

**Real-World Scenario:** In our Dockerfiles, we use multi-stage builds — JDK in the build stage to compile, and a minimal JRE-based image for the runtime stage. This reduces our container image size from ~400MB to ~150MB.

**Common Mistakes:** Since Java 11, Oracle stopped shipping standalone JRE. Many developers don't realize this and keep looking for JRE downloads. With modern Java, you use JDK everywhere or create custom runtimes using `jlink`.

**Closing:** Understanding this distinction matters for optimizing build pipelines, Docker images, and production deployments.

---

# Q5. What is OOP?

**Summary:** OOP is a programming paradigm where we model software around objects that encapsulate data and behavior, making code modular, reusable, and easier to maintain.

**Practical Explanation:** Instead of writing procedural code with functions and data scattered everywhere, OOP lets us bundle related state and behavior into objects. A `User` object has fields like `name`, `email` and methods like `updateProfile()`, `deactivate()`. This mirrors how we think about real-world entities.

**When/Why Used:** Every Java enterprise application uses OOP. Domain-Driven Design, design patterns, Spring's dependency injection — all built on OOP principles. It helps large teams work on the same codebase by establishing clear boundaries between components.

**Real-World Scenario:** In an e-commerce system, we model `Order`, `Product`, `Customer`, `Payment` as objects. Each has its own state and behavior. When requirements change — say adding a discount feature — we modify the `Order` class without breaking `Payment` logic. That's OOP in action.

**Common Mistakes:** Over-engineering with too many abstractions. Not every piece of code needs to be an object — utility functions, simple data transformations can be functional. Also, anemic domain models where objects are just data holders with no behavior defeat the purpose of OOP.

**Closing:** OOP is the foundation of Java development — mastering it means writing code that scales with team size and project complexity.

---

# Q6. Four pillars of OOP?

**Summary:** The four pillars are Encapsulation, Inheritance, Polymorphism, and Abstraction — they work together to create modular, maintainable, and extensible software.

**Practical Explanation:**

- **Encapsulation** — Hide internal state, expose controlled access via methods.
- **Inheritance** — Reuse common behavior by extending parent classes.
- **Polymorphism** — Same interface, different implementations at runtime.
- **Abstraction** — Define "what" without "how" — hide complexity behind interfaces.

**When/Why Used:** These aren't academic concepts — we use them daily. Spring's `@Service` classes encapsulate business logic. We inherit from base classes for common functionality. We program to interfaces (polymorphism) so we can swap implementations. We abstract away infrastructure concerns behind clean APIs.

**Real-World Scenario:** Consider a notification system: `NotificationService` interface (Abstraction) with `EmailNotification` and `SMSNotification` implementations (Polymorphism). Each implementation encapsulates its own connection details (Encapsulation). Both extend `BaseNotification` for common retry logic (Inheritance).

**Common Mistakes:** Favoring inheritance over composition is the most common mistake. "Prefer composition over inheritance" — use inheritance only for true "is-a" relationships, not for code reuse.

**Closing:** These four pillars are the language of every design discussion in Java — you'll use them in every code review and architecture decision.

---

# Q7. What is encapsulation?

**Summary:** Encapsulation is binding data and methods together in a class while controlling access through access modifiers — it protects object integrity and reduces coupling.

**Practical Explanation:** We make fields `private` and expose `public` getters/setters (or better, only the methods that make business sense). This way, the internal representation can change without affecting external code. It's not just about getters/setters — it's about controlling how state changes happen.

**When/Why Used:** Every well-designed Java class uses encapsulation. In Spring services, we encapsulate business rules. In domain objects, we ensure state transitions are valid. It prevents other parts of the code from putting an object into an invalid state.

**Real-World Scenario:** A `BankAccount` class has a private `balance` field. Instead of a `setBalance()` method, we expose `deposit(amount)` and `withdraw(amount)` methods that validate the amount, check for sufficient funds, and maintain audit logs. No one can directly set balance to a negative value.

**Common Mistakes:** Blindly generating getters and setters for every field defeats encapsulation. If you expose `setBalance()`, you've broken encapsulation. Also, returning mutable internal collections from getters — always return defensive copies or unmodifiable views.

**Closing:** True encapsulation is about exposing behavior, not data — it's the first line of defense for maintaining object integrity.

---

# Q8. What is inheritance?

**Summary:** Inheritance allows a child class to acquire fields and methods from a parent class, enabling code reuse and establishing "is-a" relationships.

**Practical Explanation:** Using `extends`, a subclass inherits all non-private members of the parent. It can override methods to provide specialized behavior. Java supports single inheritance for classes (one parent only) but multiple inheritance through interfaces.

**When/Why Used:** We use inheritance when there's a genuine "is-a" relationship. Framework classes often use it — extending `HttpServlet`, `WebSecurityConfigurerAdapter`, or `AbstractController`. It's useful for template method patterns where the parent defines the algorithm skeleton.

**Real-World Scenario:** In a payment gateway, `PaymentProcessor` is the base class with common logic like logging, validation, and error handling. `StripeProcessor` and `PayPalProcessor` extend it, overriding only the `processPayment()` method with provider-specific integration code.

**Common Mistakes:** Overusing inheritance for code reuse when composition is better. Deep inheritance hierarchies (more than 2-3 levels) become hard to understand and maintain. Also, forgetting that constructors are NOT inherited — you must explicitly call `super()`.

**Closing:** Use inheritance judiciously — it creates tight coupling between parent and child. When in doubt, prefer composition over inheritance.

---

# Q9. What is polymorphism?

**Summary:** Polymorphism means one interface, multiple implementations — it lets us write flexible code that works with different types through a common contract.

**Practical Explanation:** There are two types:

- **Compile-time (method overloading):** Same method name, different parameter lists. Resolved at compile time.
- **Runtime (method overriding):** Subclass provides its own implementation of a parent method. Resolved at runtime via dynamic dispatch.

**When/Why Used:** Runtime polymorphism is the cornerstone of flexible design. Spring's dependency injection is built on it — we code to interfaces, and Spring injects the concrete implementation. Strategy pattern, Factory pattern — all rely on polymorphism.

**Real-World Scenario:** We have a `PaymentGateway` interface with `charge()` method. `StripeGateway` and `RazorpayGateway` implement it differently. The service layer just calls `paymentGateway.charge(amount)` — it doesn't know or care which implementation is running. We can switch payment providers by changing a configuration, not code.

```
PaymentGateway gateway = paymentGatewayFactory.getGateway(region);
gateway.charge(order.getAmount()); // Polymorphic call
```

**Common Mistakes:** Confusing overloading with overriding. Also, using `instanceof` checks everywhere instead of leveraging polymorphism — if you're writing `if (obj instanceof X)` chains, you probably need polymorphism instead.

**Closing:** Polymorphism is what makes Java code truly extensible — master it, and you'll write code that's open for extension but closed for modification.

---

# Q10. What is abstraction?

**Summary:** Abstraction is hiding complex implementation details and exposing only the essential behavior through interfaces or abstract classes — it lets users interact with "what" something does, not "how."

**Practical Explanation:** We define contracts (interfaces/abstract classes) that specify what operations are available without revealing internal logic. The caller doesn't need to know the database query, the HTTP call, or the algorithm — just the method signature and what it returns.

**When/Why Used:** Every layered architecture uses abstraction. Controller doesn't know how the service works. Service doesn't know how the repository queries the DB. `JdbcTemplate`, `RestTemplate`, `JpaRepository` — all abstractions hiding complex infrastructure code.

**Real-World Scenario:** `UserRepository` interface has `findByEmail(String email)`. The caller doesn't care if it's backed by MySQL, PostgreSQL, MongoDB, or an in-memory cache. During testing, we swap in a mock implementation. In production, Spring Data JPA generates the actual implementation.

**Common Mistakes:** Creating abstractions too early (YAGNI) or creating leaky abstractions where implementation details bleed through. Also, abstracting for the sake of abstraction when there's only one implementation and no foreseeable second one.

**Closing:** Good abstraction is the hallmark of clean architecture — it makes code testable, swappable, and maintainable at scale.

---

# Q11. Interface vs abstract class?

**Summary:** Interface defines a pure contract with support for multiple inheritance; abstract class provides partial implementation with shared state — choose based on whether you need a contract or a base template.

**Practical Explanation:** | Feature | Interface | Abstract Class | |---------|----------|----------| | Multiple inheritance | Yes | No | | Fields | Only constants (`static final`) | Can have instance fields/state | | Constructors | No | Yes | | Methods | Abstract + default + static (Java 8+) | Abstract + concrete | | Access modifiers | Public by default | Any access modifier |

**When/Why Used:** Use interfaces when you want to define a capability contract — `Serializable`, `Comparable`, `Runnable`. Use abstract classes when you have shared state or a template method pattern — common fields and partial implementation that subclasses build on.

**Real-World Scenario:** In Spring, `CrudRepository` is an interface — it defines CRUD operations. But `AbstractRoutingDataSource` is an abstract class — it has state (data source map) and a template method where subclasses only override `determineCurrentLookupKey()`.

**Common Mistakes:** Since Java 8 added default methods to interfaces, the line blurred. But interfaces still can't hold mutable state. Don't use abstract classes just for code sharing — if there's no shared state, use interface with default methods. Also, don't implement multiple interfaces with conflicting default methods without resolving the diamond problem.

**Closing:** In modern Java, I default to interfaces and only reach for abstract classes when I need shared mutable state or constructor logic.

---

# Q12. What is immutability?

**Summary:** An immutable object is one whose state cannot be modified after creation — once constructed, it stays the same for its entire lifetime.

**Practical Explanation:** Immutability means no setters, no state changes, no surprises. You create the object with all its data, and it never changes. If you need a "modified" version, you create a new object. `String`, `Integer`, `LocalDate` — all immutable in Java.

**When/Why Used:** Immutable objects are inherently thread-safe (no synchronization needed), safe as HashMap keys, and easier to reason about. In functional programming style, microservices, and concurrent applications, immutability is a first-class design choice.

**Real-World Scenario:** DTOs (Data Transfer Objects) and Value Objects in DDD should be immutable. A `Money` value object with `amount` and `currency` — once created, it never changes. `money.add(otherMoney)` returns a new `Money` instance.

**Common Mistakes:** Thinking immutability means just removing setters — you also need to handle mutable fields like `Date` or `List` via defensive copies. Also, forgetting that `final` on a reference only prevents reassignment, not mutation of the object itself.

**Closing:** Immutability eliminates an entire class of bugs — I make objects immutable by default and only make them mutable when there's a clear reason.

---

# Q13. Why is String immutable?

**Summary:** String is immutable in Java for security, thread-safety, String Pool caching, and hashcode consistency — it's a deliberate design decision that impacts the entire JVM ecosystem.

**Practical Explanation:**

- **Security:** Strings hold class names, file paths, DB URLs, passwords. If mutable, code could alter these after security checks.
- **Thread-safety:** Immutable Strings are inherently safe to share across threads without synchronization.
- **String Pool:** JVM caches Strings in a pool. If "hello" is shared by 100 references and someone mutates it, all 100 break.
- **Hashcode caching:** String caches its hashcode. Since the value never changes, the hashcode is computed once and reused — critical for HashMap performance.

**When/Why Used:** Every Java application uses Strings extensively. Understanding immutability explains why `String.concat()` returns a new String, why we use `StringBuilder` for loops, and why Strings are safe as HashMap keys.

**Real-World Scenario:** In a multi-threaded web server, the same String URL path is shared across multiple request-handling threads. Because it's immutable, there's no race condition — no thread can corrupt it for another.

**Common Mistakes:** Concatenating Strings in loops (`str += "data"`) creates thousands of intermediate String objects — use `StringBuilder`. Also, thinking `new String("hello")` and `"hello"` are the same — the `new` keyword bypasses the String Pool.

**Closing:** String immutability is a performance and security cornerstone of the JVM — understanding it is essential for writing efficient Java code.

---

# Q14. How to create an immutable class?

**Summary:** Make the class `final`, fields `private final`, no setters, initialize everything via constructor, and return defensive copies for mutable fields.

**Practical Explanation:** The recipe:

1. Declare the class as `final` — prevents subclass from breaking immutability.
2. Make all fields `private` and `final`.
3. No setter methods.
4. Initialize all fields via constructor.
5. For mutable fields (like `List`, `Date`), make defensive copies in constructor and getter.

```
public final class Employee {
    private final String name;
    private final List<String> skills;

    public Employee(String name, List<String> skills) {
        this.name = name;
        this.skills = new ArrayList<>(skills); // defensive copy
    }

    public String getName() { return name; }

    public List<String> getSkills() {
        return Collections.unmodifiableList(skills); // defensive copy
    }
}
```

**When/Why Used:** Value objects in DDD, configuration objects, cache keys, DTOs shared across threads. Java 16+ Records (`record Employee(String name, List<String> skills)`) simplify this but still need defensive copies for mutable fields.

**Common Mistakes:** Forgetting defensive copies is the #1 mistake. If you accept a `List` in constructor and store the reference directly, the caller can still modify it externally. Same for returning mutable fields from getters.

**Closing:** Immutable classes are a senior-level design skill — getting defensive copies right is what separates solid code from subtle bugs in production.

---

# Q15. String vs StringBuilder?

**Summary:** `String` is immutable — every modification creates a new object. `StringBuilder` is mutable and efficient for repeated string manipulations.

**Practical Explanation:** When you do `str = str + "data"`, Java creates a new String object each time. In a loop of 10,000 iterations, that's 10,000 temporary objects for GC. `StringBuilder` modifies its internal `char[]` buffer in-place — one object, no garbage.

**When/Why Used:** Use `String` for fixed values, constants, and simple assignments. Use `StringBuilder` when building strings dynamically — in loops, constructing SQL queries, building log messages, generating HTML/JSON.

**Real-World Scenario:**

```
// BAD — O(n^2) and creates n String objects
String result = "";
for (String item : items) {
    result += item + ",";
}

// GOOD — O(n) and single object
StringBuilder sb = new StringBuilder();
for (String item : items) {
    sb.append(item).append(",");
}
```

**Common Mistakes:** Using `StringBuilder` for simple concatenations like `"Hello " + name` is overkill — the compiler optimizes this anyway. Also, not pre-sizing the StringBuilder when the final size is known — `new StringBuilder(1024)` avoids internal resizing.

**Closing:** This is a classic performance question — knowing when to use each is a practical skill that shows up in code reviews daily.

---

# Q16. StringBuilder vs StringBuffer?

**Summary:** `StringBuilder` is non-synchronized and faster; `StringBuffer` is synchronized and thread-safe — in modern Java, we almost always use `StringBuilder`.

**Practical Explanation:** `StringBuffer` was the original (Java 1.0) — every method is `synchronized`, adding overhead even in single-threaded scenarios. `StringBuilder` (Java 1.5) is the drop-in replacement without synchronization. Same API, better performance.

**When/Why Used:** Use `StringBuilder` in 99% of cases — most string building is local to a method and doesn't cross thread boundaries. If you truly need thread-safe string building (rare), consider other concurrency patterns rather than `StringBuffer`.

**Real-World Scenario:** In a request-scoped service method building a response string, `StringBuilder` is the right choice — the object lives entirely within one thread. I haven't used `StringBuffer` in production in years.

**Common Mistakes:** Using `StringBuffer` "just to be safe" — unnecessary synchronization hurts performance. Also, thinking `StringBuffer` makes string operations atomic — individual method calls are synchronized, but a sequence of `append()` calls is not atomic without external synchronization.

**Closing:** `StringBuilder` is the default choice — `StringBuffer` is essentially a legacy class that exists for backward compatibility.

---

# Q17. What is equals()?

**Summary:** `equals()` is the method we override to define logical/content equality between objects — as opposed to reference equality (`==`).

**Practical Explanation:** The default `equals()` from `Object` class checks reference equality (same as `==`). We override it to compare actual content. For example, two `User` objects with the same `userId` should be "equal" even if they're different instances in memory.

**When/Why Used:** Critical for Collections — `HashMap.get()`, `List.contains()`, `Set.add()` all rely on `equals()`. Without proper `equals()`, your objects won't behave correctly in any collection.

**Real-World Scenario:**

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    User user = (User) o;
    return Objects.equals(userId, user.userId);
}
```

**Common Mistakes:**

- Overriding `equals()` without overriding `hashCode()` — violates the contract and breaks HashMaps.
- Using `instanceof` instead of `getClass()` — can break symmetry with subclasses.
- Forgetting null checks.
- Not making it reflexive, symmetric, transitive, and consistent.

**Closing:** Always override `equals()` and `hashCode()` together — I use Lombok's `@EqualsAndHashCode` or IDE generation to avoid manual errors.

---

# Q18. hashCode() contract?

**Summary:** The hashCode contract states that if two objects are equal via `equals()`, they must return the same `hashCode()` — this is critical for hash-based collections to function correctly.

**Practical Explanation:** The contract:

1. If `a.equals(b)` is `true`, then `a.hashCode() == b.hashCode()` must be `true`.
2. If `a.equals(b)` is `false`, hashCodes can still collide (but shouldn't for performance).
3. `hashCode()` must be consistent — same object returns same hashCode across calls (unless fields used in computation change).

**When/Why Used:** `HashMap`, `HashSet`, `Hashtable` use `hashCode()` to determine the bucket. If equal objects have different hashCodes, a `HashSet` could contain "duplicates" and `HashMap.get()` might not find a value you just put in.

**Real-World Scenario:** I once debugged a production issue where objects were being stored in a `HashSet` but `.contains()` returned `false`. The root cause was `equals()` was overridden but `hashCode()` was not — the set checked the wrong bucket every time.

**Common Mistakes:** Using mutable fields in `hashCode()` calculation — if the field changes after insertion into a `HashMap`, the object becomes unreachable (it's in the wrong bucket). Always use immutable fields like `id`.

**Closing:** Violating the hashCode contract is one of the most insidious bugs in Java — it causes silent data loss in collections. Always override both `equals()` and `hashCode()`.

---

# Q19. == vs equals()?

**Summary:** `==` compares references (are they the same object in memory?), while `equals()` compares logical content (are they meaningfully the same?).

**Practical Explanation:**

- `==` for primitives compares values. For objects, it checks if both references point to the exact same memory location.
- `equals()` compares object content — what we define as "equality" by overriding the method.

**When/Why Used:** Use `==` for primitives (`int`, `boolean`) and for null checks. Use `equals()` for all object comparisons — Strings, DTOs, entities.

**Real-World Scenario:**

```
String a = new String("hello");
String b = new String("hello");
System.out.println(a == b);      // false — different objects
System.out.println(a.equals(b)); // true — same content

String c = "hello";
String d = "hello";
System.out.println(c == d);      // true — String Pool reuse
```

**Common Mistakes:** Comparing Strings with `==` — works sometimes due to String Pool interning but fails unpredictably. Comparing wrapper types like `Integer` with `==` — works for cached values (-128 to 127) but fails for larger values. Always use `equals()`.

**Closing:** This is a classic interview question because getting it wrong causes real production bugs — especially with String and Integer comparisons.

---

# Q20. What is the final keyword?

**Summary:** `final` prevents modification — final variables can't be reassigned, final methods can't be overridden, and final classes can't be extended.

**Practical Explanation:**

- **Final variable:** Value assigned once. For references, the reference can't change, but the object itself can still be mutated.
- **Final method:** Subclasses cannot override it — used to lock down critical behavior.
- **Final class:** Cannot be subclassed — `String`, `Integer`, `Math` are all final.

**When/Why Used:** Constants (`public static final`), ensuring immutability, preventing inheritance of sensitive classes, and effectively-final variables in lambdas. In Spring, `@Service` beans are typically stored in final fields via constructor injection.

**Real-World Scenario:**

```
@Service
public class OrderService {
    private final OrderRepository orderRepository; // final — assigned once via constructor
    private final PaymentGateway paymentGateway;

    public OrderService(OrderRepository orderRepository, PaymentGateway paymentGateway) {
        this.orderRepository = orderRepository;
        this.paymentGateway = paymentGateway;
    }
}
```

**Common Mistakes:** Thinking `final List<String> list` means the list can't be modified — only the reference is final, you can still `list.add()`. For true immutability, use `Collections.unmodifiableList()` or `List.of()`.

**Closing:** `final` is a key tool for writing predictable, thread-safe code — I use it on almost every field and method parameter.

---

# Q21. What is the static keyword?

**Summary:** `static` means the member belongs to the class itself, not to any specific instance — it's shared across all objects of that class.

**Practical Explanation:** Static fields are shared — one copy for all instances. Static methods can be called without creating an object. Static blocks run once when the class is loaded. Static inner classes don't hold a reference to the outer class.

**When/Why Used:** Utility methods (`Math.max()`, `Collections.sort()`), constants (`static final`), factory methods (`LocalDate.now()`), singleton patterns, and counters shared across instances.

**Real-World Scenario:**

```
public class AppConstants {
    public static final String API_VERSION = "v2";
    public static final int MAX_RETRY_COUNT = 3;
}

// Utility class
public class DateUtils {
    public static LocalDate parseDate(String date) { ... }
}
```

**Common Mistakes:** Overusing static — static mutable state is essentially a global variable and makes testing/mocking difficult. Static methods can't be overridden (only hidden), breaking polymorphism. Also, accessing instance variables from static context — compilation error.

**Closing:** Use static for truly class-level concerns — constants, utilities, and factory methods. Avoid static mutable state as it hurts testability and concurrency.

---

# Q22. What is a constructor?

**Summary:** A constructor is a special method invoked at object creation time to initialize the object's state — it has the same name as the class and no return type.

**Practical Explanation:** Constructors ensure objects are born in a valid state. If no constructor is defined, Java provides a default no-arg constructor. Once you define any constructor, the default disappears. Constructors can call other constructors via `this()` or parent constructors via `super()`.

**When/Why Used:** Every object creation uses a constructor. In Spring, constructor injection is the recommended DI approach — dependencies are passed via constructor, stored in `final` fields, ensuring the bean is fully initialized.

**Real-World Scenario:**

```
@Service
public class UserService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    // Constructor injection — Spring recommended approach
    public UserService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }
}
```

**Common Mistakes:** Heavy logic in constructors — constructors should initialize, not perform business logic. Throwing checked exceptions from constructors makes the class hard to use. Also, forgetting to call `super()` when the parent class doesn't have a no-arg constructor.

**Closing:** Constructors are the gatekeepers of object validity — well-designed constructors guarantee that objects are always in a usable state from the moment they're created.

---

# Q23. Constructor overloading?

**Summary:** Constructor overloading means defining multiple constructors with different parameter lists, giving callers flexibility in how they create objects.

**Practical Explanation:** Each constructor has a different signature (number, type, or order of parameters). They can chain to each other using `this()` — the most specific constructor does the actual initialization, others delegate to it.

**When/Why Used:** Providing convenience constructors with default values, backward compatibility when adding new fields, and the telescoping constructor pattern (though Builder pattern is preferred for many parameters).

**Real-World Scenario:**

```
public class Notification {
    private final String recipient;
    private final String message;
    private final Priority priority;

    public Notification(String recipient, String message) {
        this(recipient, message, Priority.NORMAL); // delegates
    }

    public Notification(String recipient, String message, Priority priority) {
        this.recipient = recipient;
        this.message = message;
        this.priority = priority;
    }
}
```

**Common Mistakes:** Telescoping constructors become unreadable with many parameters — use Builder pattern instead. Also, `this()` must be the first statement in the constructor. And remember, `this()` and `super()` cannot be used together in the same constructor.

**Closing:** Constructor overloading provides flexibility, but for classes with more than 3-4 parameters, switch to the Builder pattern for readability.

---

# Q24. What is garbage collection?

**Summary:** Garbage Collection (GC) is the JVM's automatic memory management — it identifies and reclaims memory occupied by objects that are no longer reachable, freeing developers from manual memory management.

**Practical Explanation:** The GC tracks object references. When no live thread can reach an object (no reference chain from GC roots), it's eligible for collection. The GC runs in its own threads, and during collection, it may cause "stop-the-world" pauses where application threads freeze.

**When/Why Used:** GC runs automatically — but as senior developers, we tune it. We choose GC algorithms, set heap sizes, monitor GC logs, and optimize code to reduce GC pressure (e.g., object pooling, avoiding unnecessary allocations).

**Real-World Scenario:** In a high-throughput microservice handling 10K requests/second, we noticed latency spikes every 30 seconds. GC logs revealed Full GC pauses of 500ms. We switched from Parallel GC to G1GC, tuned `-XX:MaxGCPauseMillis=50`, and the spikes disappeared.

**Common Mistakes:** Calling `System.gc()` — it's only a suggestion, not guaranteed, and can cause worse pauses. Creating excessive short-lived objects in hot loops increases GC pressure. Also, not enabling GC logs in production — `-Xlog:gc*` is essential for diagnosis.

**Closing:** You don't manage memory manually in Java, but you absolutely need to understand GC to build high-performance, low-latency applications.

---

# Q25. Types of GC?

**Summary:** Java provides multiple GC algorithms — Serial, Parallel, CMS, G1, and ZGC — each optimized for different workload characteristics like throughput, latency, or heap size.

**Practical Explanation:**

- **Serial GC:** Single-threaded, for small applications. `-XX:+UseSerialGC`
- **Parallel GC:** Multi-threaded, maximizes throughput. Default in Java 8. `-XX:+UseParallelGC`
- **CMS (Concurrent Mark-Sweep):** Low-pause, deprecated in Java 14. Concurrent collection.
- **G1 GC:** Region-based, balanced throughput/latency. Default since Java 9. `-XX:+UseG1GC`
- **ZGC:** Ultra-low latency (<10ms pauses even on TB heaps). `-XX:+UseZGC`
- **Shenandoah:** Similar to ZGC, concurrent compaction. Red Hat developed.

**When/Why Used:** G1 is the default and works well for most applications. ZGC for latency-critical systems (trading platforms, real-time APIs). Parallel GC for batch processing where throughput matters more than pause times.

**Real-World Scenario:** For our real-time pricing engine, we use ZGC with a 16GB heap — pause times stay under 2ms even under heavy load. For our nightly batch ETL jobs, we use Parallel GC to maximize throughput.

**Common Mistakes:** Not choosing the right GC for your workload. Running a latency-sensitive API with Parallel GC causes long stop-the-world pauses. Also, not monitoring GC metrics — use tools like JVisualVM, GCViewer, or Grafana with Micrometer.

**Closing:** GC tuning is a critical production skill — the right GC choice can mean the difference between smooth operations and customer-facing latency issues.

---

# Q26. What is a memory leak?

**Summary:** A memory leak in Java occurs when objects are no longer needed but still referenced, preventing the GC from reclaiming them — eventually leading to OutOfMemoryError.

**Practical Explanation:** Even with GC, Java can have memory leaks. If you hold references to objects you no longer need — in static collections, caches without eviction, unclosed resources, listener registrations — those objects accumulate and eat heap space.

**When/Why Used:** Memory leak detection is a critical production debugging skill. Tools like JProfiler, VisualVM, Eclipse MAT, and heap dump analysis (`jmap -dump:live,format=b,file=heap.hprof <pid>`) are essential.

**Real-World Scenario:** We had a service running out of memory every 3 days. Heap dump analysis revealed a `static HashMap` used as a cache that grew unbounded — keys were session IDs, never removed after session expiry. Fix: replaced with `ConcurrentHashMap` with TTL-based eviction (or Caffeine/Guava cache).

**Common Mistakes:**

- `static` collections that grow without bounds.
- Not closing resources (`Connection`, `InputStream`, `ResultSet`) — use try-with-resources.
- Inner classes holding implicit references to outer class instances.
- Registering listeners/callbacks and never deregistering them.
- `ThreadLocal` variables not cleaned up — especially dangerous in thread pools.

**Closing:** Memory leaks in Java are subtle — they don't crash immediately but degrade over days. Knowing how to analyze heap dumps is a must-have skill.

---

# Q27. Stack vs Heap?

**Summary:** Stack stores method-local variables and call frames (fast, LIFO, per-thread); Heap stores objects and instance variables (shared across threads, managed by GC).

**Practical Explanation:**

- **Stack:** Each thread has its own stack. Stores primitive local variables, method parameters, and object references. Automatically cleaned when the method returns. Fixed size — `StackOverflowError` if too deep (recursion).
- **Heap:** Shared memory area. All objects (`new` keyword) live here. Managed by Garbage Collector. `OutOfMemoryError` if full.

**When/Why Used:** Understanding this is critical for debugging — `StackOverflowError` means infinite recursion, `OutOfMemoryError` means heap exhaustion or memory leak. Also essential for understanding thread safety — stack is thread-safe by nature (thread-local), heap is shared and needs synchronization.

**Real-World Scenario:** A recursive method without proper base case caused `StackOverflowError` in production. The fix was switching to iterative approach. Separately, a large file upload storing the entire content as a byte array caused `OutOfMemoryError: Java heap space` — fixed by using streaming/chunked processing.

**Common Mistakes:** Thinking all variables live on the stack — only local primitives and references live on the stack; the actual objects are on the heap. Also, the JVM's escape analysis can sometimes allocate objects on the stack if they don't escape the method — but that's JVM optimization, not something to code for.

**Closing:** Stack vs Heap is foundational for understanding JVM memory, threading, and debugging production memory issues.

---

# Q28. Checked vs unchecked exception?

**Summary:** Checked exceptions must be handled at compile time (declared or caught); unchecked exceptions (RuntimeExceptions) occur at runtime and don't require explicit handling.

**Practical Explanation:**

- **Checked:** `IOException`, `SQLException`, `ClassNotFoundException` — the compiler forces you to handle them. They represent recoverable conditions.
- **Unchecked:** `NullPointerException`, `IllegalArgumentException`, `ArrayIndexOutOfBoundsException` — programming errors, not expected to be caught everywhere.
- **Error:** `OutOfMemoryError`, `StackOverflowError` — JVM-level, not meant to be caught.

**When/Why Used:** In modern Java (especially with Spring), the trend is toward unchecked exceptions. Spring wraps checked exceptions (like `SQLException`) into unchecked ones (`DataAccessException`). Custom business exceptions should extend `RuntimeException`.

**Real-World Scenario:**

```
// Custom unchecked business exception
public class OrderNotFoundException extends RuntimeException {
    public OrderNotFoundException(Long orderId) {
        super("Order not found: " + orderId);
    }
}
// Handled globally via @ControllerAdvice
```

**Common Mistakes:** Catching `Exception` or `Throwable` — too broad, hides bugs. Empty catch blocks — silently swallowing errors. Also, wrapping every checked exception with `throws` up the call chain without handling — defeats the purpose.

**Closing:** Use unchecked exceptions for business errors, handle checked exceptions close to where they occur, and always have a global exception handler in your API layer.

---

# Q29. Try-with-resources?

**Summary:** Try-with-resources (Java 7+) automatically closes resources implementing `AutoCloseable` at the end of the try block — eliminating resource leak bugs.

**Practical Explanation:** Before Java 7, we needed verbose try-finally blocks to close resources. Now, any `AutoCloseable` resource declared in the try parentheses is automatically closed, even if an exception occurs. The close happens in reverse order of declaration.

**When/Why Used:** Every time you work with I/O: file streams, database connections, HTTP connections, `BufferedReader`, `PreparedStatement`. Especially critical in production to prevent connection pool exhaustion.

**Real-World Scenario:**

```
// Modern approach
try (Connection conn = dataSource.getConnection();
     PreparedStatement ps = conn.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        // process results
    }
} // All three resources auto-closed in reverse order

// vs Old approach — error-prone, verbose
Connection conn = null;
try {
    conn = dataSource.getConnection();
    // ...
} finally {
    if (conn != null) conn.close(); // Could itself throw!
}
```

**Common Mistakes:** Resources declared outside the try block won't be auto-closed. Also, suppressed exceptions — if both the try body and the close method throw, the close exception is suppressed (accessible via `getSuppressed()`). Forgetting to make custom resources implement `AutoCloseable`.

**Closing:** Try-with-resources is a must-use pattern — any resource that needs closing should go in a try-with-resources block. No exceptions.

---

# Q30. What is Optional?

**Summary:** `Optional` is a container object (Java 8+) that may or may not hold a value — it forces callers to handle the absence of a value explicitly, reducing `NullPointerException` bugs.

**Practical Explanation:** Instead of returning `null` (which the caller might forget to check), return `Optional<User>`. The caller is forced to handle the empty case — `orElse()`, `orElseThrow()`, `ifPresent()`, `map()`.

**When/Why Used:** Method return types where "no result" is a valid outcome. Spring Data's `findById()` returns `Optional<Entity>`. Not meant for method parameters, fields, or collections (use empty collection instead).

**Real-World Scenario:**

```
public Optional<User> findByEmail(String email) {
    return userRepository.findByEmail(email);
}

// Caller
User user = userService.findByEmail(email)
    .orElseThrow(() -> new UserNotFoundException(email));

// Or with functional chaining
String city = userService.findByEmail(email)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElse("Unknown");
```

**Common Mistakes:** `optional.get()` without checking `isPresent()` — defeats the purpose and can still throw `NoSuchElementException`. Using Optional for fields or method parameters — it's not `Serializable` and adds overhead. Returning `Optional.of(null)` — throws NPE; use `Optional.ofNullable()`.

**Closing:** Optional is a powerful tool for API design — use it for return types to make the "null case" impossible to ignore.

---

# Q31. What is Stream API?

**Summary:** The Stream API (Java 8+) provides a declarative, functional way to process collections — filter, map, reduce, collect — with support for parallel execution.

**Practical Explanation:** Streams don't store data — they're pipelines. You create a stream from a collection, apply intermediate operations (lazy), and trigger execution with a terminal operation. They support method chaining, lambda expressions, and can be parallelized with `.parallelStream()`.

**When/Why Used:** Any time you process collections — filtering lists, transforming data, aggregating results, grouping. Replaces verbose for-loops with concise, readable code.

**Real-World Scenario:**

```
// Find active premium users' emails
List<String> emails = users.stream()
    .filter(User::isActive)
    .filter(u -> u.getPlan() == Plan.PREMIUM)
    .map(User::getEmail)
    .sorted()
    .collect(Collectors.toList());

// Group orders by status
Map<OrderStatus, List<Order>> ordersByStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::getStatus));
```

**When/Why Used:** Data transformation, filtering, aggregation, reporting. Cleaner than nested loops and conditionals.

**Common Mistakes:** Reusing a stream after terminal operation — streams are one-use. Using `parallelStream()` without understanding — it uses the common ForkJoinPool and can hurt performance for I/O-bound or small collections. Side effects in stream operations (modifying external state in `forEach`) violate the functional contract.

**Closing:** Stream API is essential for modern Java — it makes collection processing concise, readable, and potentially parallelizable.

---

# Q32. map vs flatMap?

**Summary:** `map()` transforms each element one-to-one; `flatMap()` transforms each element into a stream and flattens the results into a single stream — it's a map + flatten operation.

**Practical Explanation:**

- `map(x -> f(x))` — transforms each element. If `f(x)` returns a collection/Optional, you get a stream of collections.
- `flatMap(x -> f(x))` — transforms each element into a stream, then merges all resulting streams into one.

**When/Why Used:** `flatMap` when dealing with nested structures — list of lists, Optional of Optional, one-to-many relationships.

**Real-World Scenario:**

```
// map — one-to-one: User -> name
List<String> names = users.stream()
    .map(User::getName)
    .collect(Collectors.toList());

// flatMap — one-to-many: User -> List<Order> -> flat stream of orders
List<Order> allOrders = users.stream()
    .flatMap(user -> user.getOrders().stream())
    .collect(Collectors.toList());

// Optional flatMap
Optional<String> city = user
    .flatMap(User::getAddress)  // returns Optional<Address>
    .map(Address::getCity);     // returns Optional<String>
```

**Common Mistakes:** Using `map` when you need `flatMap` — results in `Stream<Stream<T>>` or `Optional<Optional<T>>` instead of the flattened version. Not understanding that `flatMap` expects a function that returns a Stream (or Optional in Optional's case).

**Closing:** Once you internalize the difference, you'll naturally reach for `flatMap` whenever you see nested structures — it's a pattern that shows up constantly in real code.

---

# Q33. Intermediate vs terminal operations?

**Summary:** Intermediate operations (filter, map, sorted) are lazy and return a new Stream; terminal operations (collect, forEach, count) trigger actual processing and produce a result.

**Practical Explanation:**

- **Intermediate:** `filter()`, `map()`, `flatMap()`, `sorted()`, `distinct()`, `limit()`, `peek()` — they build up the pipeline but nothing executes yet.
- **Terminal:** `collect()`, `forEach()`, `count()`, `reduce()`, `findFirst()`, `anyMatch()`, `toList()` — they trigger the pipeline and produce the final result.

**When/Why Used:** Laziness is a performance feature — intermediate operations can be fused and short-circuited. For example, `stream.filter(...).findFirst()` stops processing as soon as the first match is found, not after filtering the entire collection.

**Real-World Scenario:**

```
// Nothing executes until collect() is called
Stream<User> pipeline = users.stream()
    .filter(u -> u.isActive())      // intermediate — lazy
    .map(u -> u.getEmail())         // intermediate — lazy
    .sorted();                       // intermediate — lazy

// Now everything executes
List<String> result = pipeline.collect(Collectors.toList()); // terminal
```

**Common Mistakes:** Using `peek()` for side effects in production code — it's meant for debugging. Forgetting the terminal operation — the stream never executes. Calling multiple terminal operations on the same stream — `IllegalStateException`.

**Closing:** Understanding lazy evaluation is key to writing efficient stream pipelines — the JVM optimizes the execution path based on terminal operation needs.

---

# Q34. What is a lambda?

**Summary:** A lambda is a concise, anonymous function that implements a functional interface — it brings functional programming to Java, making code more expressive and compact.

**Practical Explanation:** Instead of writing a full anonymous inner class to implement a single-method interface, you write `(parameters) -> expression`. The compiler infers the functional interface from context. Lambdas can capture effectively-final variables from the enclosing scope.

**When/Why Used:** Everywhere in modern Java — Stream API operations, event handlers, `Comparator`, `Runnable`, `Callable`, Spring's callback-based APIs, and custom functional interfaces.

**Real-World Scenario:**

```
// Before lambdas
Collections.sort(users, new Comparator<User>() {
    @Override
    public int compare(User a, User b) {
        return a.getName().compareTo(b.getName());
    }
});

// With lambdas
users.sort((a, b) -> a.getName().compareTo(b.getName()));

// Even cleaner with method reference
users.sort(Comparator.comparing(User::getName));
```

**Common Mistakes:** Lambdas with too much logic — if it's more than 2-3 lines, extract it to a named method and use a method reference. Modifying external variables — lambdas can only capture effectively-final variables. Also, exception handling in lambdas is verbose — checked exceptions don't play well with functional interfaces.

**Closing:** Lambdas are the backbone of modern Java — they're not just syntactic sugar, they enable a functional programming style that makes code cleaner and more composable.

---

# Q35. Functional interface?

**Summary:** A functional interface has exactly one abstract method and serves as the target type for lambdas and method references — annotated with `@FunctionalInterface` for safety.

**Practical Explanation:** Java's built-in functional interfaces in `java.util.function`:

- `Function<T, R>` — takes T, returns R (`map`)
- `Predicate<T>` — takes T, returns boolean (`filter`)
- `Consumer<T>` — takes T, returns void (`forEach`)
- `Supplier<T>` — takes nothing, returns T (factory)
- `BiFunction<T, U, R>` — takes T and U, returns R

**When/Why Used:** Every lambda needs a functional interface as its target type. Custom business logic that should be pluggable — validation rules, transformation strategies, callback handlers.

**Real-World Scenario:**

```
@FunctionalInterface
public interface PricingStrategy {
    BigDecimal calculatePrice(Product product, Customer customer);
}

// Usage
PricingStrategy premiumPricing = (product, customer) ->
    product.getBasePrice().multiply(BigDecimal.valueOf(0.9));

PricingStrategy regularPricing = (product, customer) ->
    product.getBasePrice();
```

**Common Mistakes:** Adding more than one abstract method — breaks the contract. Confusing default methods with abstract methods — an interface with one abstract method and multiple default methods is still a functional interface. Not using `@FunctionalInterface` annotation — it's optional but prevents accidental addition of a second abstract method.

**Closing:** Functional interfaces are the bridge between OOP and functional programming in Java — understanding the built-in ones makes Stream API and lambda usage second nature.

---

# Q36. Default methods?

**Summary:** Default methods (Java 8+) allow adding method implementations in interfaces without breaking existing implementations — they solved the interface evolution problem.

**Practical Explanation:** Before Java 8, adding a method to an interface broke every implementing class. Default methods provide a body using the `default` keyword. Implementing classes can override them or use the default. This enabled adding `stream()`, `forEach()` to `Collection` interface without breaking millions of existing classes.

**When/Why Used:** API evolution (adding new methods to published interfaces), providing common behavior that most implementations share, and enabling multiple inheritance of behavior (not state).

**Real-World Scenario:**

```
public interface Auditable {
    default String getAuditLog() {
        return "Modified at: " + getModifiedDate() + " by: " + getModifiedBy();
    }
    LocalDateTime getModifiedDate();
    String getModifiedBy();
}
// All entities implementing Auditable get getAuditLog() for free
```

**Common Mistakes:** Diamond problem — if a class implements two interfaces with the same default method, it must override and resolve the conflict explicitly. Default methods can't access instance state (no fields in interfaces). Don't abuse them to make interfaces look like abstract classes.

**Closing:** Default methods were a game-changer for Java's evolution — they let the JDK team evolve core interfaces like `Collection` and `Map` without breaking backward compatibility.

---

# Q37. What is multithreading?

**Summary:** Multithreading is concurrent execution of two or more threads within a single process, enabling better CPU utilization and responsive applications.

**Practical Explanation:** A thread is a lightweight unit of execution within a process. Multithreading lets us perform multiple tasks simultaneously — handling multiple HTTP requests, background processing, parallel computations. In Java, we create threads via `Thread` class, `Runnable`, `Callable`, or (preferred) using `ExecutorService`.

**When/Why Used:** Web servers (Tomcat uses thread pools to handle concurrent requests), async processing (sending emails in background), parallel data processing, scheduled tasks, and responsive UIs.

**Real-World Scenario:**

```
// Modern approach with ExecutorService
ExecutorService executor = Executors.newFixedThreadPool(10);
Future<Report> reportFuture = executor.submit(() -> generateReport());
Future<Stats> statsFuture = executor.submit(() -> calculateStats());

// Both run concurrently
Report report = reportFuture.get();
Stats stats = statsFuture.get();

// Even better with CompletableFuture
CompletableFuture.supplyAsync(() -> fetchUserData())
    .thenApply(data -> processData(data))
    .thenAccept(result -> saveResult(result));
```

**Common Mistakes:** Creating raw threads instead of using thread pools — leads to unbounded thread creation and OOM. Not handling thread safety — shared mutable state without synchronization causes race conditions. Also, not handling `InterruptedException` properly — either re-interrupt the thread or propagate it.

**Closing:** Multithreading is powerful but complex — modern Java provides higher-level abstractions like `CompletableFuture` and virtual threads (Java 21) that make concurrent programming safer and simpler.

---

# Q38. Process vs Thread?

**Summary:** A process is an independent program with its own memory space; a thread is a lightweight unit of execution within a process that shares the process's memory.

**Practical Explanation:** | Aspect | Process | Thread | |--------|---------|--------| | Memory | Separate address space | Shares heap within process | | Communication | IPC (sockets, pipes) — expensive | Direct shared memory — fast | | Creation cost | Heavy (new memory space) | Lightweight | | Crash impact | Isolated — one crash doesn't affect others | One thread crash can kill entire process | | Context switch | Expensive | Cheaper |

**When/Why Used:** In microservices, each service is a separate process (JVM) — isolation and independent deployment. Within a service, we use threads for concurrent request handling, background tasks, and parallel processing.

**Real-World Scenario:** A Spring Boot application is one process (one JVM). Tomcat's thread pool (default 200 threads) handles concurrent HTTP requests within that process. Each request runs on a separate thread, sharing the application's heap memory — that's why `@Service` beans must be thread-safe.

**Common Mistakes:** Thinking more threads always means better performance — too many threads cause excessive context switching. Not understanding that thread-shared memory requires synchronization. Also, confusing process-level isolation (microservices) with thread-level concurrency (within one service).

**Closing:** Understanding process vs thread is fundamental for designing microservice architectures and for writing correct concurrent code within each service.

---

# Q39. What is synchronization?

**Summary:** Synchronization controls concurrent access to shared resources, ensuring only one thread can execute a critical section at a time — preventing race conditions and data corruption.

**Practical Explanation:** The `synchronized` keyword acquires a monitor lock on an object. Only one thread can hold the lock at a time — others block until it's released. Can be applied to methods or blocks. Java also provides `java.util.concurrent` locks (`ReentrantLock`, `ReadWriteLock`) for more fine-grained control.

**When/Why Used:** When multiple threads access and modify shared mutable state — counters, shared caches, connection pools. In Spring, singleton beans are shared across threads, so any mutable state in them needs synchronization.

**Real-World Scenario:**

```
// synchronized method
public synchronized void incrementCounter() {
    this.count++;
}

// Better — synchronized block (finer granularity)
public void updateCache(String key, String value) {
    synchronized(this.cache) {
        cache.put(key, value);
    }
}

// Best — use concurrent collections
private final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();
```

**Common Mistakes:** Over-synchronizing — locking entire methods when only a few lines need protection. Synchronizing on the wrong object. Deadlocks from inconsistent lock ordering. Also, not considering `ConcurrentHashMap`, `AtomicInteger`, and other `java.util.concurrent` alternatives that are often better than manual synchronization.

**Closing:** Synchronization is essential but should be used surgically — prefer concurrent utilities from `java.util.concurrent` over raw `synchronized` blocks.

---

# Q40. volatile keyword?

**Summary:** `volatile` ensures visibility of a variable's changes across threads — when one thread writes to a volatile variable, all other threads immediately see the updated value.

**Practical Explanation:** Without `volatile`, threads may cache variable values in CPU registers/caches. Thread A updates `flag = true`, but Thread B might still see `flag = false` from its local cache. `volatile` forces reads/writes directly to main memory, establishing a happens-before relationship.

**When/Why Used:** Simple flags for thread signaling (stop flags), double-checked locking in singleton pattern, status indicators. Not a replacement for synchronization — `volatile` guarantees visibility but NOT atomicity.

**Real-World Scenario:**

```
public class GracefulShutdown {
    private volatile boolean running = true;

    public void run() {
        while (running) {  // Without volatile, thread might never see the change
            processNextTask();
        }
    }

    public void shutdown() {
        running = false;  // Visible to the run() thread immediately
    }
}
```

**Common Mistakes:** Thinking `volatile` makes operations atomic — `volatile int count; count++` is NOT thread-safe because `count++` is read-modify-write (three operations). Use `AtomicInteger` for that. Also, overusing volatile when `synchronized` or `Atomic*` classes are more appropriate.

**Closing:** `volatile` is a lightweight synchronization mechanism for visibility — use it for flags and status variables, but reach for `AtomicInteger` or `synchronized` when you need atomicity.

---

# Q41. Deadlock?

**Summary:** Deadlock is a situation where two or more threads are permanently blocked, each waiting for a lock held by the other — creating a circular dependency that never resolves.

**Practical Explanation:** Classic scenario: Thread A holds Lock 1, needs Lock 2. Thread B holds Lock 2, needs Lock 1. Neither can proceed. Deadlocks require four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. Eliminating any one prevents deadlock.

**When/Why Used:** Deadlock prevention is a critical concern in multi-threaded applications. We need to design lock acquisition strategies carefully.

**Real-World Scenario:**

```
// DEADLOCK-PRONE
public void transferMoney(Account from, Account to, BigDecimal amount) {
    synchronized(from) {           // Thread 1: locks accountA
        synchronized(to) {         // Thread 1: waits for accountB
            // transfer
        }
    }
}
// Thread 1: transfer(A, B) — locks A, waits for B
// Thread 2: transfer(B, A) — locks B, waits for A → DEADLOCK

// FIX — consistent lock ordering
public void transferMoney(Account from, Account to, BigDecimal amount) {
    Account first = from.getId() < to.getId() ? from : to;
    Account second = from.getId() < to.getId() ? to : from;
    synchronized(first) {
        synchronized(second) {
            // transfer — always locks lower ID first
        }
    }
}
```

**Common Mistakes:** Not establishing a consistent lock ordering strategy. Using nested `synchronized` blocks carelessly. Not using `tryLock()` with timeout from `ReentrantLock` which can detect potential deadlocks. Also, not monitoring for deadlocks — `jstack` and `ThreadMXBean.findDeadlockedThreads()` are diagnostic tools.

**Closing:** Deadlocks are among the hardest bugs to reproduce and debug — prevention through design (consistent ordering, timeouts, avoiding nested locks) is far better than detection.

---

# Q42. What is serialization?

**Summary:** Serialization is converting a Java object into a byte stream for storage or transmission, and deserialization is the reverse — reconstructing the object from bytes.

**Practical Explanation:** Implement `Serializable` (marker interface), and Java can convert your object graph to bytes. Used for persisting objects to files, sending over network, caching (Redis, Memcached), and session replication in distributed systems.

**When/Why Used:** Distributed caching (Redis stores serialized Java objects), JMS message queues, HTTP session persistence across server restarts, deep copying objects. Though in modern systems, JSON/Protobuf serialization is preferred over Java's native serialization.

**Real-World Scenario:**

```
public class User implements Serializable {
    private static final long serialVersionUID = 1L; // Version control
    private String name;
    private transient String password; // Won't be serialized
}

// Modern alternative — JSON serialization with Jackson
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(user);
User deserialized = mapper.readValue(json, User.class);
```

**Common Mistakes:** Not defining `serialVersionUID` — if the class changes, deserialization of old data fails with `InvalidClassException`. Serializing sensitive data (passwords, tokens) without marking them `transient`. Also, Java's native serialization has known security vulnerabilities — deserialization attacks can execute arbitrary code. Prefer JSON/Protobuf.

**Closing:** While Java's native serialization still exists, modern applications favor JSON (Jackson) or Protobuf — they're faster, cross-language, and more secure.

---

# Q43. transient keyword?

**Summary:** `transient` marks a field to be excluded from serialization — the field won't be written to the byte stream and will have its default value upon deserialization.

**Practical Explanation:** When an object is serialized, `transient` fields are skipped. Upon deserialization, these fields get default values (`null` for objects, `0` for numbers, `false` for boolean). Useful for sensitive data, derived/computed fields, and non-serializable fields.

**When/Why Used:** Passwords, security tokens, cached/computed values, logger references, database connections — anything that shouldn't be persisted or transmitted, or can't be serialized.

**Real-World Scenario:**

```
public class UserSession implements Serializable {
    private String username;
    private String authToken;
    private transient String rawPassword;       // Security — never serialize
    private transient Logger log = LoggerFactory.getLogger(this.getClass()); // Not serializable
    private transient BigDecimal cachedBalance;  // Can be recomputed
}
```

**Common Mistakes:** Forgetting that `transient` fields are `null`/default after deserialization — need to reinitialize them, typically in a `readObject()` method or a post-deserialization hook. Also, `static` fields are never serialized regardless of `transient` — `transient` on a static field is redundant.

**Closing:** `transient` is a simple but important modifier — it's your tool for controlling exactly what gets serialized and protecting sensitive data.

---

# Q44. Cloneable?

**Summary:** `Cloneable` is a marker interface that indicates an object can be cloned via `Object.clone()` — but it's a broken API, and modern Java avoids it in favor of copy constructors or factory methods.

**Practical Explanation:** To make a class cloneable: implement `Cloneable` and override `clone()` method. Without implementing `Cloneable`, calling `clone()` throws `CloneNotSupportedException`. The default `clone()` does a shallow copy — primitive fields are copied, but object references still point to the same objects.

**When/Why Used:** Rarely used in modern Java. Copy constructors, static factory methods, or serialization-based copying are preferred. However, you may encounter it in legacy codebases.

**Real-World Scenario:**

```
// Legacy approach (avoid)
public class Order implements Cloneable {
    @Override
    public Order clone() {
        try {
            Order copy = (Order) super.clone();
            copy.items = new ArrayList<>(this.items); // Deep copy mutable fields
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(); // Can't happen
        }
    }
}

// Modern approach (preferred)
public class Order {
    public Order(Order other) { // Copy constructor
        this.id = other.id;
        this.items = new ArrayList<>(other.items);
    }
}
```

**Common Mistakes:** Relying on default shallow copy for objects with mutable fields — changes to cloned object's mutable fields affect the original. Not overriding `clone()` to do deep copy. Joshua Bloch (Effective Java) calls `Cloneable` "a broken API" — prefer copy constructors.

**Closing:** Know `Cloneable` for interviews and legacy code, but in your own code, always use copy constructors or builder patterns for object copying.

---

# Q45. Shallow vs deep copy?

**Summary:** Shallow copy copies field values (references point to same objects); deep copy recursively copies all referenced objects — creating a completely independent clone.

**Practical Explanation:**

- **Shallow copy:** Creates a new object, copies primitive fields, but reference fields still point to the same objects as the original. Modifying a mutable field in the copy affects the original.
- **Deep copy:** Creates a new object AND recursively clones all referenced objects. The copy is completely independent.

**When/Why Used:** Deep copy is needed when you want a fully independent copy — undo/redo operations, caching original state before modifications, passing objects to untrusted code.

**Real-World Scenario:**

```
class Department {
    String name;
    List<Employee> employees;

    // Shallow copy — employees list is shared
    Department shallowCopy() {
        Department copy = new Department();
        copy.name = this.name;
        copy.employees = this.employees; // Same list reference!
        return copy;
    }

    // Deep copy — employees list is independent
    Department deepCopy() {
        Department copy = new Department();
        copy.name = this.name;
        copy.employees = this.employees.stream()
            .map(Employee::new) // Copy constructor for each employee
            .collect(Collectors.toList());
        return copy;
    }
}
```

**Common Mistakes:** Assuming `clone()` does deep copy — it's shallow by default. Not deep-copying nested mutable objects. Using serialization for deep copy works but is slow and requires `Serializable`. Forgetting that `String` is immutable so it doesn't need deep copying, but `Date` and collections do.

**Closing:** Always be explicit about whether you need shallow or deep copy — subtle bugs from shared mutable references are hard to track down in production.

---

# Q46. What is reflection?

**Summary:** Reflection allows inspecting and modifying class structure, methods, fields, and constructors at runtime — it's the backbone of frameworks like Spring, Hibernate, and JUnit.

**Practical Explanation:** Using `java.lang.reflect` API, you can discover class metadata, invoke methods dynamically, access private fields, create instances without `new`, and inspect annotations — all at runtime. The `Class` object is the entry point.

**When/Why Used:** Frameworks use reflection extensively — Spring's dependency injection, Hibernate's ORM mapping, Jackson's JSON serialization, JUnit's test discovery. As application developers, we rarely use it directly, but understanding it helps debug framework behavior.

**Real-World Scenario:**

```
// How Spring roughly works behind the scenes
Class<?> clazz = Class.forName("com.example.UserService");
Object instance = clazz.getDeclaredConstructor().newInstance();

// Inject dependencies
Field field = clazz.getDeclaredField("userRepository");
field.setAccessible(true);  // Access private field
field.set(instance, repositoryInstance);

// How JUnit discovers test methods
Arrays.stream(clazz.getDeclaredMethods())
    .filter(m -> m.isAnnotationPresent(Test.class))
    .forEach(m -> m.invoke(instance));
```

**Common Mistakes:** Using reflection in application code when regular OOP would suffice — it's slow, breaks encapsulation, and loses compile-time safety. `setAccessible(true)` bypasses access modifiers, which is a security concern. Also, reflection doesn't play well with refactoring — method/field name changes won't be caught at compile time.

**Closing:** Reflection is a power tool — essential for framework developers, but application developers should understand it for debugging rather than using it directly.

---

# Q47. What is a package?

**Summary:** A package is a namespace that organizes related classes and interfaces into logical groups — it prevents naming conflicts and controls access through package-level visibility.

**Practical Explanation:** Packages map to directory structures (`com.example.service` → `com/example/service/`). They provide namespace isolation (two classes with the same name in different packages), access control (package-private is the default access), and logical organization of large codebases.

**When/Why Used:** Every Java project uses packages. Standard conventions:

- `com.company.project.controller` — REST controllers
- `com.company.project.service` — Business logic
- `com.company.project.repository` — Data access
- `com.company.project.model` — Domain entities
- `com.company.project.config` — Configuration classes
- `com.company.project.exception` — Custom exceptions

**Real-World Scenario:** In a Spring Boot microservice:

```
com.example.orderservice
├── controller/    → OrderController.java
├── service/       → OrderService.java
├── repository/    → OrderRepository.java
├── model/         → Order.java, OrderItem.java
├── dto/           → OrderRequest.java, OrderResponse.java
├── exception/     → OrderNotFoundException.java
└── config/        → AppConfig.java, SecurityConfig.java
```

**Common Mistakes:** Default (package-private) access is often overlooked — classes and methods without explicit access modifiers are only visible within the same package. This is actually useful for hiding implementation details. Also, avoid circular package dependencies — if `service` depends on `repository` and `repository` depends on `service`, your architecture has a problem.

**Closing:** Good package structure reflects clean architecture — it's the first thing I look at when joining a new project to understand the codebase organization.

---

*End of Core Java & OOP — 47 Interview Q&A*

# HashMap Deep Dive — 40 Interview Q&A (Senior-Level, Interview Perspective)

---

## Q1. What is HashMap?

**Summary:** HashMap is a hash-table-based implementation of the Map interface that stores key-value pairs and provides O(1) average-time complexity for put, get, and remove operations.

**Practical Explanation:** Internally, HashMap maintains an array of `Node<K,V>` (called buckets). When you `put(key, value)`, it computes the key's `hashCode()`, determines the bucket index, and stores the entry there. On `get(key)`, it repeats the hash computation to jump directly to the right bucket — no scanning needed.

**When/Why Used:** It's the most commonly used Map implementation in Java. Caching, lookup tables, counting occurrences, grouping data, storing configuration — any time you need fast key-based access. In Spring applications, it's used everywhere internally — bean definitions, request parameter mapping, property sources.

**Real-World Scenario:** Caching user profiles by userId for quick lookup during request processing:

```
Map<Long, UserProfile> userCache = new HashMap<>(1024);
userCache.put(userId, profile);
UserProfile cached = userCache.get(userId); // O(1)
```

**Common Mistakes:** Not overriding `hashCode()` and `equals()` on custom key objects — leads to "phantom" entries you can never retrieve. Using HashMap in multi-threaded code without synchronization — causes data corruption silently.

**Closing:** HashMap is the workhorse of Java collections — understanding its internals is what separates a senior developer from a junior one.

---

## Q2. How HashMap works internally?

**Summary:** HashMap uses an array of buckets (Node array), where the bucket index is determined by hashing the key. Each bucket can hold a linked list (or a red-black tree in Java 8+) of entries to handle collisions.

**Practical Explanation:** The internal flow of `put(key, value)`:

1. Compute `key.hashCode()` and apply a spread function: `hash = hashCode ^ (hashCode >>> 16)` — this mixes high and low bits.
2. Determine bucket index: `index = hash & (capacity - 1)` — bitwise AND since capacity is a power of 2.
3. If bucket is empty → create new `Node` and place it there.
4. If bucket is occupied → traverse the linked list/tree:
   - If a node with the same key exists (`equals()` returns true) → replace the value.
   - Otherwise → append a new node to the end of the list.
5. If the list length exceeds 8 (and capacity >= 64) → convert to red-black tree.
6. If total size exceeds threshold (`capacity * loadFactor`) → resize (double the array).

For `get(key)`:

1. Compute hash → find bucket index.
2. Traverse nodes in that bucket, comparing with `equals()`.
3. Return value if found, `null` if not.

**When/Why Used:** This knowledge is essential for writing performant code — knowing the internal mechanism helps you choose proper initial capacity, write good `hashCode()` methods, and diagnose performance issues.

**Real-World Scenario:** We had a production issue where a HashMap-backed cache was extremely slow. Profiling revealed all keys had the same `hashCode()` (a badly implemented custom key class), so all entries went into one bucket — degrading from O(1) to O(n). Fixing the `hashCode()` distribution resolved it instantly.

**Common Mistakes:** Not understanding the spread function — even a perfect `hashCode()` gets further processed. Thinking HashMap preserves insertion order (it doesn't — use `LinkedHashMap`). Not accounting for the Node object overhead in memory calculations.

**Closing:** Interviewers love this question because it touches hashing, data structures, equals/hashCode contract, and performance — it's a complete test of your fundamentals.

---

# Q3. What is hash collision?

**Summary:** A hash collision occurs when two different keys produce the same bucket index after hashing — they end up in the same bucket and must be stored together.

**Practical Explanation:** Since the bucket array has a finite size (say 16), and keys are potentially infinite, multiple keys will inevitably map to the same index. For example, if `"Aa".hashCode()` and `"BB".hashCode()` both resolve to bucket index 5, that's a collision. The bucket then holds both entries in a linked list (or tree).

**When/Why Used:** Understanding collisions is critical for designing good `hashCode()` methods. A poor hash function causes many collisions, degrading HashMap performance from O(1) to O(n). The JVM's spread function `hash ^ (hash >>> 16)` helps distribute keys more evenly.

**Real-World Scenario:** In a system indexing millions of products by SKU, a naive `hashCode()` that only used the first two characters caused massive clustering — all SKUs starting with "PR" collided. Switching to a proper hash using all characters (`Objects.hash(sku)`) eliminated the bottleneck.

**Common Mistakes:** Thinking collisions are bugs — they're normal and expected. The goal is to minimize them, not eliminate them. Also, confusing `hashCode()` collision (same hashCode value) with bucket collision (same bucket index) — the spread function and capacity mask can cause bucket collisions even with different hashCodes.

**Closing:** Some collisions are inevitable — what matters is distributing keys uniformly across buckets to keep bucket chains short and lookups fast.

---

# Q4. How collisions handled in Java 7?

**Summary:** In Java 7, HashMap handles collisions using a singly linked list within each bucket — new entries are added at the head of the list (head insertion).

**Practical Explanation:** When two keys hash to the same bucket, Java 7 creates a linked list of `Entry<K,V>` nodes at that bucket index. On `put()`, the new entry is inserted at the head of the list (prepend). On `get()`, the list is traversed linearly, checking `equals()` on each node until the key is found.

**When/Why Used:** This was the standard collision resolution from Java 1.2 through Java 7. While simple to implement, it has a fatal flaw — if many keys collide, the linked list grows long, and lookup degrades to O(n).

**Real-World Scenario:** A well-known Denial-of-Service attack vector (HashDoS) exploited this — attackers crafted HTTP request parameters with keys that all hashed to the same bucket, turning HashMap operations into O(n) and overwhelming the server. This was a major motivation for the Java 8 treeification improvement.

**Common Mistakes:** Head insertion in Java 7 causes a critical concurrency bug — during concurrent resize, the linked list can form a cycle, causing `get()` to loop infinitely. This is the infamous Java 7 HashMap infinite loop bug. It was fixed in Java 8 by switching to tail insertion.

**Closing:** Java 7's linked list approach is simple but vulnerable — understanding its limitations explains exactly why Java 8 introduced treeification.

---

# Q5. How collisions handled in Java 8?

**Summary:** In Java 8, HashMap starts with a linked list for collisions but converts to a balanced red-black tree when a bucket's chain exceeds 8 nodes — improving worst-case from O(n) to O(log n).

**Practical Explanation:** Java 8 introduced two key changes:

1. **Tail insertion** instead of head insertion — eliminates the infinite loop bug during concurrent resize.
2. **Treeification** — when a bucket's linked list grows beyond `TREEIFY_THRESHOLD` (8) and the table capacity is >= 64, the list is converted into a self-balancing red-black tree. This brings worst-case lookup from O(n) to O(log n).

The `Node<K,V>` is replaced with `TreeNode<K,V>` which extends `LinkedHashMap.Entry` and maintains left/right/parent pointers.

**When/Why Used:** This is the current behavior in all modern Java (8+). It protects against pathological hash distributions and HashDoS attacks while maintaining O(1) average case.

**Real-World Scenario:** In a high-traffic API, we had keys with deliberately poor hash distribution (external input used as keys). In Java 7, this would've been catastrophic. With Java 8's treeification, even worst-case buckets stayed at O(log n) — the system remained responsive.

**Common Mistakes:** Assuming treeification happens at exactly 8 entries — it also requires table capacity >= 64 (otherwise it resizes the table first). Also, for treeification to work efficiently, keys should implement `Comparable` — otherwise TreeNodes fall back to comparing by `hashCode()` and `identityHashCode`, which works but is less optimal.

**Closing:** Java 8's collision handling is a brilliant balance between simplicity (linked list for small buckets) and performance (tree for large buckets) — it's one of the most impactful internal improvements in Java's history.

---

# Q6. When does treeification occur?

**Summary:** Treeification occurs when a single bucket has more than 8 nodes (TREEIFY_THRESHOLD) AND the total table capacity is at least 64 (MIN_TREEIFY_CAPACITY).

**Practical Explanation:** Two conditions must be met:

1. Bucket chain length > `TREEIFY_THRESHOLD` (8)
2. Table capacity >= `MIN_TREEIFY_CAPACITY` (64)

If the chain exceeds 8 but capacity is below 64, HashMap resizes the table instead — the assumption being that a small table with long chains just needs more buckets. Only when the table is already large enough does it convert to a tree.

**When/Why Used:** This is automatic and internal — developers don't trigger it manually. But understanding these thresholds helps when designing capacity planning and hash functions.

**Real-World Scenario:** If you initialize a HashMap with default capacity 16 and insert keys that all collide, the first 8 inserts create a linked list. On the 9th insert, instead of treeifying, the HashMap resizes to 32, then 64. Only after capacity reaches 64 does treeification actually kick in.

**Common Mistakes:** Thinking treeification always happens at 8 — the capacity check is often overlooked. Also, each treeified `TreeNode` consumes roughly twice the memory of a regular `Node` — so treeification trades memory for speed.

**Closing:** These thresholds are carefully chosen by the JDK team — 8 is a statistical sweet spot where the probability of a linked list that long under random hashing is extremely low (about 0.00000006).

---

# Q7. When does untreeification occur?

**Summary:** Untreeification converts a red-black tree back to a linked list when the node count in a bucket drops below `UNTREEIFY_THRESHOLD` (6), typically during resize or removal.

**Practical Explanation:** During resize, entries are redistributed across the new (doubled) buckets. If a formerly treeified bucket splits and the resulting bucket has 6 or fewer nodes, it's converted back to a simpler linked list. The threshold is 6 (not 8) to provide hysteresis — avoiding repeated tree/list oscillation when node count hovers around the boundary.

**When/Why Used:** This is an internal optimization. The gap between treeify (8) and untreeify (6) prevents "thrashing" — if both thresholds were 8, adding and removing a single entry would constantly convert between tree and list.

**Real-World Scenario:** A cache backed by HashMap grows during peak traffic (treeification happens on hot buckets) and shrinks during off-peak as entries expire (untreeification recovers memory). The hysteresis ensures stable behavior during transitions.

**Common Mistakes:** Thinking untreeification happens during `remove()` on every deletion — it primarily occurs during resize operations. The threshold gap of 2 (8 vs 6) is deliberate engineering, not arbitrary.

**Closing:** The treeify/untreeify balance is a great example of JDK engineering — optimizing for real-world workload patterns rather than theoretical edge cases.

---

# Q8. Why capacity is power of 2?

**Summary:** HashMap's capacity is always a power of 2 so that bucket index calculation can use `hash & (capacity - 1)` instead of the expensive modulo operation — this is a bitwise optimization.

**Practical Explanation:** To find a bucket index from a hash, you need `hash % capacity`. Modulo is slow. But when `capacity` is a power of 2, `capacity - 1` produces a bitmask of all 1s (e.g., 16 - 1 = 15 = `0b1111`). Then `hash & (capacity - 1)` is equivalent to `hash % capacity` but much faster — a single CPU instruction.

```
capacity = 16     → binary: 10000
capacity - 1 = 15 → binary: 01111
hash & 01111      → keeps last 4 bits → value 0-15 (valid bucket indices)
```

**When/Why Used:** Every `put()` and `get()` call computes the bucket index, so this optimization matters enormously at scale — millions of operations per second.

**Real-World Scenario:** If you pass a non-power-of-2 initial capacity (e.g., `new HashMap<>(100)`), HashMap automatically rounds it up to the next power of 2 (128). Understanding this prevents wasting memory — `new HashMap<>(17)` actually allocates 32 buckets.

**Common Mistakes:** Specifying arbitrary initial capacities and assuming that's what you get. Also, not understanding that this power-of-2 constraint is why the spread function `hash ^ (hash >>> 16)` exists — it ensures high bits influence the index even when only low bits are used by the mask.

**Closing:** This is a beautiful low-level optimization — a single design choice (power-of-2 capacity) enables bitwise indexing, motivates the spread function, and simplifies resize (entries either stay or move exactly `oldCapacity` positions).

---

# Q9. Load factor meaning?

**Summary:** The load factor (default 0.75) determines when the HashMap resizes — it's the ratio of entries to capacity at which the table doubles in size to maintain performance.

**Practical Explanation:** `threshold = capacity * loadFactor`. When `size >= threshold`, the HashMap resizes. With default capacity 16 and load factor 0.75, resize happens at 12 entries. Lower load factor = more empty buckets = fewer collisions but more memory. Higher load factor = denser table = more collisions but less memory.

**When/Why Used:** The default 0.75 is a well-tested balance between time (collision probability) and space (memory usage). Adjust only when you have strong reasons — memory-constrained environments might use higher (0.9), latency-critical systems might use lower (0.5).

**Real-World Scenario:** For a cache that stores 10,000 entries and rarely changes, we set initial capacity to `10000 / 0.75 + 1 ≈ 13334` to avoid any resize during population:

```
Map<String, Config> configCache = new HashMap<>(13334, 0.75f);
```

This prevents the expensive resize operations during bulk loading.

**Common Mistakes:** Setting load factor too low (like 0.1) wastes huge amounts of memory. Setting it too high (like 1.0) means the table is nearly full before resizing — heavily degraded performance. Also, forgetting that load factor is set at construction time and cannot be changed later.

**Closing:** The 0.75 default is the right choice 99% of the time — only change it with data backing your decision.

---

# Q10. When resize happens?

**Summary:** HashMap resizes (doubles its capacity) when the number of entries exceeds `capacity * loadFactor` — this is called the threshold.

**Practical Explanation:** With default settings (capacity=16, loadFactor=0.75), the threshold is 12. After the 12th entry, `put()` triggers a resize:

1. Create a new `Node[]` array of double the size (16 → 32).
2. Redistribute every existing entry to the new array (rehashing).
3. Replace the old array with the new one.

This is O(n) — every entry must be repositioned. The new index is either the same or `oldIndex + oldCapacity` (because of the power-of-2 optimization).

**When/Why Used:** Resize is automatic and unavoidable as data grows. But knowing it happens helps us prevent it when possible — pre-sizing the HashMap to avoid multiple resizes during bulk operations.

**Real-World Scenario:** Loading 1 million entries into a default HashMap causes ~20 resizes (16→32→64→...→2M). Each resize rehashes all entries. Pre-sizing avoids this:

```
// Prevents ALL resizes
int expectedSize = 1_000_000;
Map<String, Data> map = new HashMap<>((int)(expectedSize / 0.75) + 1);
```

**Common Mistakes:** Not pre-sizing when the approximate size is known. Also, assuming resize is free — in latency-sensitive applications, a sudden resize of a large HashMap can cause a noticeable latency spike in the request that triggers it.

**Closing:** Resize is the single biggest performance concern with HashMap — pre-sizing is a simple optimization that prevents expensive rehashing during bulk operations.

---

# Q11. What is rehashing?

**Summary:** Rehashing is the process of recalculating bucket indices for all existing entries when the HashMap resizes — every entry is repositioned in the new, larger bucket array.

**Practical Explanation:** When the array doubles, the bitmask changes (e.g., `0b01111` → `0b11111` for 16→32). Each entry's bucket index is recalculated: `newIndex = hash & (newCapacity - 1)`. In Java 8, this is optimized — entries in each old bucket either stay at the same index or move to `oldIndex + oldCapacity`. This is determined by checking a single bit: `hash & oldCapacity`.

**When/Why Used:** Rehashing is automatic during resize. It's needed because the bucket index depends on capacity — when capacity changes, indices change.

**Real-World Scenario:** During a load test, we noticed periodic latency spikes every time the HashMap doubled. Thread dumps showed the application thread stuck in `HashMap.resize()`. Solution: pre-calculate the expected capacity and initialize the HashMap accordingly.

**Common Mistakes:** In Java 7, rehashing under concurrent access could create circular linked lists (infinite loops). Java 8's tail-insertion fixed this. Also, rehashing is why `HashMap` is not efficient for concurrent writes — the entire table is being restructured.

**Closing:** Rehashing is expensive but necessary — understanding its cost justifies the effort of pre-sizing HashMaps in performance-critical code.

---

# Q12. Worst-case complexity Java 7?

**Summary:** In Java 7, HashMap's worst-case time complexity is O(n) — when all keys hash to the same bucket, forming a single long linked list.

**Practical Explanation:** Java 7 uses linked lists for collision resolution. If all `n` keys land in one bucket, every `get()`, `put()`, and `remove()` must traverse the entire list. This degrades HashMap to essentially a linked list — destroying its O(1) promise.

**When/Why Used:** This is a theoretical worst case, but it became a real security concern with HashDoS attacks where attackers crafted inputs to force worst-case collisions.

**Real-World Scenario:** In 2011, the HashDoS attack exploited this — web frameworks parsed query parameters into HashMaps. Attackers sent requests with thousands of parameters that all hashed to the same bucket, causing O(n^2) total processing time (n insertions × O(n) per insertion), overwhelming servers.

**Common Mistakes:** Assuming O(1) is guaranteed — it's only the average case. In Java 7, there's no protection against pathological hash distributions.

**Closing:** Java 7's O(n) worst case is the primary reason Java 8 introduced treeification — reducing worst case to O(log n) was a critical security and performance fix.

---

# Q13. Worst-case complexity Java 8?

**Summary:** In Java 8, HashMap's worst-case time complexity improved to O(log n) thanks to red-black tree conversion when bucket chains exceed 8 nodes.

**Practical Explanation:** When a bucket's linked list grows beyond 8 entries, it converts to a balanced red-black tree. Tree operations (search, insert, delete) are O(log n) in the worst case. So even if all keys collide into one bucket, you get O(log n) instead of O(n).

**When/Why Used:** This improvement is automatic — you benefit from it just by using Java 8+. It provides a safety net against poor hash distributions and malicious inputs.

**Real-World Scenario:** After migrating a service from Java 7 to Java 8, we stress-tested with worst-case hash inputs. Java 7 took 45 seconds for 50,000 colliding inserts. Java 8 completed the same operation in under 1 second — the O(n) to O(log n) improvement is dramatic at scale.

**Common Mistakes:** Thinking O(log n) is the normal case — it's the worst case. Normal operations with good hash distribution are still O(1). Also, the tree requires keys to be comparable for optimal performance — if keys don't implement `Comparable`, the tree uses `identityHashCode` as a tiebreaker, which works but isn't ideal.

**Closing:** O(log n) worst case makes Java 8+ HashMap production-safe against both accidental and malicious hash collisions — it's one of the most important improvements in the collections framework.

---

# Q14. Role of hashCode()?

**Summary:** `hashCode()` determines which bucket an entry goes into — it's the first step in locating where a key-value pair is stored or retrieved.

**Practical Explanation:** When you call `map.put(key, value)` or `map.get(key)`, HashMap:

1. Calls `key.hashCode()`.
2. Applies the spread function: `hash = hashCode ^ (hashCode >>> 16)`.
3. Computes bucket index: `index = hash & (capacity - 1)`.

A good `hashCode()` distributes keys uniformly across buckets. A bad one clusters keys, causing collisions and degrading performance.

**When/Why Used:** Every HashMap operation starts with `hashCode()`. Its quality directly determines HashMap performance. In production, the difference between a good and bad `hashCode()` can be O(1) vs O(n) per operation.

**Real-World Scenario:**

```
// BAD hashCode — all objects go to same bucket
@Override
public int hashCode() { return 42; } // O(n) everything

// GOOD hashCode — uses all meaningful fields
@Override
public int hashCode() {
    return Objects.hash(id, name, department);
}
```

**Common Mistakes:** Using mutable fields in `hashCode()` — if the field changes after insertion, the entry becomes unreachable (wrong bucket). Inconsistency between `equals()` and `hashCode()` — if two objects are `equals()`, they MUST have the same `hashCode()`. Not overriding `hashCode()` when overriding `equals()`.

**Closing:** `hashCode()` is the performance engine of HashMap — invest time in getting it right, and let Lombok's `@EqualsAndHashCode` or `Objects.hash()` do the heavy lifting.

---

# Q15. Role of equals()?

**Summary:** `equals()` determines key identity within a bucket — after `hashCode()` locates the bucket, `equals()` finds the exact key among potentially colliding entries.

**Practical Explanation:** When multiple keys hash to the same bucket, HashMap traverses the chain and calls `equals()` on each key to find the exact match. The flow is:

1. `hashCode()` → bucket index (which bucket?)
2. `equals()` → key match (which entry in the bucket?)

Without proper `equals()`, HashMap can't distinguish between colliding keys.

**When/Why Used:** Every `get()`, `put()` (checking for existing key), `containsKey()`, and `remove()` uses `equals()` to identify the exact key within a bucket. Both `hashCode()` and `equals()` must be overridden consistently.

**Real-World Scenario:**

```
// Without equals() override
User u1 = new User(1, "Alice");
User u2 = new User(1, "Alice");
map.put(u1, "admin");
map.get(u2); // Returns null! u1.equals(u2) is false (reference check)

// With equals() override based on id
map.get(u2); // Returns "admin" — correct!
```

**Common Mistakes:** Overriding `equals()` with wrong signature — `equals(User other)` instead of `equals(Object other)` creates an overload, not an override. Not checking `null` and type in `equals()`. Using `equals()` with fields not included in `hashCode()` — violates the contract.

**Closing:** `hashCode()` gets you to the door, `equals()` gets you through it — both must work together for HashMap to function correctly.

---

# Q16. Mutable key problem?

**Summary:** Using mutable objects as HashMap keys is dangerous — if a key's fields change after insertion, its hash changes, making the entry unreachable in its original bucket.

**Practical Explanation:** HashMap stores the entry in a bucket based on the key's `hashCode()` at insertion time. If you mutate the key afterward, `hashCode()` returns a different value, so `get()` looks in a different bucket — the entry is effectively lost. It's still in memory (memory leak) but unretrievable.

**When/Why Used:** This is a critical design consideration. Always use immutable keys — `String`, `Integer`, `Long`, or custom immutable objects. If you must use a mutable key, never modify it while it's in the map.

**Real-World Scenario:**

```
List<String> key = new ArrayList<>(List.of("a", "b"));
Map<List<String>, String> map = new HashMap<>();
map.put(key, "value");

System.out.println(map.get(key)); // "value" — works

key.add("c"); // MUTATED the key!
System.out.println(map.get(key)); // null — entry is LOST
System.out.println(map.size());   // 1 — entry still exists, unreachable
```

**Common Mistakes:** Using `Date`, `List`, `StringBuilder`, or any mutable object as a HashMap key without understanding the consequences. Also, this is a sneaky cause of memory leaks — entries accumulate but can never be retrieved or removed.

**Closing:** This is one of the most dangerous bugs in Java — always use immutable keys, and make it a code review rule on your team.

---

# Q17. Null key handling?

**Summary:** HashMap allows exactly one null key — it's always stored at bucket index 0 since `null.hashCode()` can't be called.

**Practical Explanation:** HashMap special-cases null keys. When the key is null, it skips `hashCode()` computation and directly uses bucket index 0. Only one null key is allowed — if you `put(null, "a")` then `put(null, "b")`, the value is replaced just like any duplicate key.

**When/Why Used:** Useful when "no key" is a valid state in your data. However, many modern Java developers consider null keys a code smell and prefer `Optional` or explicit sentinel values.

**Real-World Scenario:**

```
Map<String, String> config = new HashMap<>();
config.put(null, "default_value");  // Stored at bucket 0
config.put("key1", "value1");

String value = config.getOrDefault(someKey, config.get(null));
```

**Common Mistakes:** Assuming all Map implementations allow null keys — `ConcurrentHashMap`, `Hashtable`, and `TreeMap` (if using Comparator that doesn't handle null) do NOT allow null keys. Also, `map.get(key)` returning `null` is ambiguous — does the key not exist, or is the value `null`? Use `containsKey()` to disambiguate.

**Closing:** HashMap's null key support is a design decision, not a recommendation — in production code, avoid null keys and use `ConcurrentHashMap` when thread safety is needed, which disallows nulls entirely.

---

# Q18. Duplicate key insertion?

**Summary:** When you insert a duplicate key, HashMap replaces the old value with the new value and returns the previous value — no duplicate keys are ever stored.

**Practical Explanation:** On `put(key, value)`, HashMap finds the bucket, traverses the chain, and checks `equals()` on each existing key. If a match is found, it updates the value in that node and returns the old value. The key itself is NOT replaced — only the value.

**When/Why Used:** This behavior is fundamental to using HashMap as a lookup table. It's how we update cached values, overwrite configurations, and maintain the latest state for a given key.

**Real-World Scenario:**

```
Map<String, Integer> wordCount = new HashMap<>();
for (String word : words) {
    wordCount.put(word, wordCount.getOrDefault(word, 0) + 1);
    // Each put replaces the old count with count + 1
}

// Or using merge (cleaner)
wordCount.merge(word, 1, Integer::sum);
```

**Common Mistakes:** Not checking the return value of `put()` — it returns the previous value (or null if key was new), which is useful for detecting overwrites. Also, using `putIfAbsent()` when you actually want to update existing values, or using `put()` when you want insert-only behavior.

**Closing:** Understanding `put()` semantics — replace-on-duplicate — is basic but critical. Use `putIfAbsent()`, `computeIfAbsent()`, or `merge()` when you need different behavior.

---

# Q19. HashMap thread safety?

**Summary:** HashMap is NOT thread-safe — concurrent reads and writes without external synchronization can cause data corruption, infinite loops (Java 7), and lost updates.

**Practical Explanation:** HashMap has no internal locking. When multiple threads concurrently modify a HashMap:

- Entries can be lost (two threads writing to the same bucket).
- `size` counter can become inconsistent.
- Resize under concurrency can corrupt the internal structure.
- In Java 7, concurrent resize can cause infinite loops (circular linked list).

**When/Why Used:** Use HashMap only in single-threaded contexts or with external synchronization. For concurrent access, use `ConcurrentHashMap`.

**Real-World Scenario:** In a Spring singleton `@Service` bean (shared across all request threads), storing data in a plain `HashMap` field is a concurrency bug:

```
@Service
public class RateLimiter {
    // BUG — shared mutable state, no synchronization
    private Map<String, Integer> requestCounts = new HashMap<>();

    // FIX
    private Map<String, AtomicInteger> requestCounts = new ConcurrentHashMap<>();
}
```

**Common Mistakes:** Thinking "read-only HashMap is safe" — it IS safe if the map is fully populated before any thread reads it AND never modified afterward (effectively immutable). But even `Collections.synchronizedMap()` only synchronizes individual operations — compound operations (check-then-act) still need external locking. Prefer `ConcurrentHashMap`.

**Closing:** Never use HashMap in multi-threaded code with writes — `ConcurrentHashMap` exists for exactly this purpose and should be your default choice.

---

# Q20. Resize issue in Java 7?

**Summary:** In Java 7, concurrent resize of HashMap can cause an infinite loop — two threads simultaneously rehashing a linked list can create a circular reference, causing `get()` to loop forever.

**Practical Explanation:** Java 7 uses head insertion during rehashing. When two threads simultaneously resize:

1. Thread A reads the linked list: A → B → null.
2. Thread B completes resize first, reversing the list: B → A → null.
3. Thread A resumes with stale references, creates: A → B → A (cycle).
4. Any subsequent `get()` hitting this bucket enters an infinite loop — 100% CPU, thread never returns.

**When/Why Used:** This is a well-documented Java 7 bug that caused production outages at many companies. It's the #1 reason never to use HashMap concurrently.

**Real-World Scenario:** A production application's CPU suddenly spiked to 100%. Thread dumps showed a thread stuck in `HashMap.get()` — infinite loop. Root cause: a HashMap in a singleton service was being written to by multiple Tomcat request threads. The fix was simple — switch to `ConcurrentHashMap`.

**Common Mistakes:** Thinking Java 8 makes HashMap thread-safe — Java 8 fixed the infinite loop (tail insertion doesn't cause cycles) but HashMap is still not thread-safe. Concurrent writes can still lose data. Also, this bug only manifests under very specific timing — it can pass all testing and only appear under production load.

**Closing:** This is one of the most infamous bugs in Java history — it's a perfect example of why "it works in testing" doesn't mean "it's correct." Always use `ConcurrentHashMap` for shared maps.

---

# Q21. ConcurrentModificationException?

**Summary:** `ConcurrentModificationException` is thrown when a collection is structurally modified while being iterated — it's a fail-fast mechanism to detect unsafe concurrent modification.

**Practical Explanation:** HashMap maintains an internal `modCount` (modification count). When you create an iterator, it snapshots `modCount`. During iteration, if `modCount` changes (due to `put()`, `remove()`, `clear()`), the iterator throws `ConcurrentModificationException`. This happens even in single-threaded code.

**When/Why Used:** It's a safety net — not a guarantee. It detects most accidental modifications during iteration. "Concurrent" here doesn't necessarily mean multi-threaded — it means "at the same time as iteration."

**Real-World Scenario:**

```
// THROWS ConcurrentModificationException
for (Map.Entry<String, User> entry : userMap.entrySet()) {
    if (entry.getValue().isInactive()) {
        userMap.remove(entry.getKey()); // Modifies map during iteration!
    }
}

// FIX — use Iterator.remove()
Iterator<Map.Entry<String, User>> it = userMap.entrySet().iterator();
while (it.hasNext()) {
    if (it.next().getValue().isInactive()) {
        it.remove(); // Safe — iterator handles it
    }
}

// OR — use removeIf (Java 8+, cleanest)
userMap.entrySet().removeIf(e -> e.getValue().isInactive());
```

**Common Mistakes:** Catching and ignoring this exception — it indicates a real bug. Thinking it only happens with multi-threading — single-threaded code triggers it too. Also, note that `ConcurrentHashMap` NEVER throws this exception — it uses a weakly consistent iterator.

**Closing:** When you see this exception, don't suppress it — restructure your code to use `Iterator.remove()`, `removeIf()`, or collect keys to remove in a separate pass.

---

# Q22. Fail-fast iterator?

**Summary:** A fail-fast iterator immediately throws `ConcurrentModificationException` when it detects the underlying collection has been structurally modified — it fails quickly rather than producing unpredictable results.

**Practical Explanation:** HashMap, ArrayList, HashSet all use fail-fast iterators. They track `modCount` and check it on every `next()` call. If the count doesn't match the snapshot taken at iterator creation, it throws immediately. This is a "best-effort" mechanism — not guaranteed under concurrent modification.

**When/Why Used:** Fail-fast behavior helps catch bugs early during development. The alternative — silently returning wrong data or skipping entries — would be far worse.

**Real-World Scenario:**

```
// Fail-fast — HashMap iterator
Map<String, String> map = new HashMap<>();
// modification during iteration → ConcurrentModificationException

// Fail-safe — ConcurrentHashMap iterator
Map<String, String> map = new ConcurrentHashMap<>();
// modification during iteration → works fine (weakly consistent)
```

**Fail-safe alternatives:**

- `ConcurrentHashMap` — uses weakly consistent iterators
- `CopyOnWriteArrayList` — iterates over a snapshot
- `Collections.unmodifiableMap()` — prevents modification entirely

**Common Mistakes:** Relying on fail-fast for thread safety — it's a debugging aid, not a synchronization mechanism. The Javadoc explicitly says fail-fast behavior "cannot be guaranteed" and should only be used to detect bugs. Also, `for-each` loops hide the iterator — the exception can be confusing for developers who don't understand what's happening underneath.

**Closing:** Fail-fast is your friend during development — it surfaces bugs early. In concurrent environments, switch to fail-safe collections like `ConcurrentHashMap`.

---

# Q23. keySet vs entrySet?

**Summary:** `entrySet()` is more efficient for iterating over both keys and values because it avoids a separate `get()` lookup per key — `keySet()` requires an extra hash lookup for each value.

**Practical Explanation:**

- `keySet()` — returns `Set<K>`. To get values, you call `map.get(key)` for each key — that's an extra O(1) hash computation per entry.
- `entrySet()` — returns `Set<Map.Entry<K,V>>`. Each entry already holds both key and value — no additional lookup needed.

- `values()` — returns `Collection<V>`. Use when you only need values.

**When/Why Used:** Always use `entrySet()` when you need both keys and values during iteration. It's faster and more idiomatic.

**Real-World Scenario:**

```
// SLOWER — extra get() per key
for (String key : map.keySet()) {
    String value = map.get(key); // Redundant hash lookup
    process(key, value);
}

// FASTER — direct access to both key and value
for (Map.Entry<String, String> entry : map.entrySet()) {
    process(entry.getKey(), entry.getValue()); // No extra lookup
}

// MODERN — forEach with lambda
map.forEach((key, value) -> process(key, value));
```

**Common Mistakes:** Using `keySet()` + `get()` out of habit — it's O(1) per `get()` but still unnecessary overhead at scale. In a map with 1 million entries, that's 1 million extra hash computations. Also, modifications to `keySet()` or `entrySet()` reflect back to the map — `keySet().remove(key)` actually removes from the map.

**Closing:** `entrySet()` or `forEach()` — pick one and make it a team standard. There's never a reason to use `keySet()` + `get()` for iteration.

---

# Q24. Memory overhead?

**Summary:** HashMap has significant memory overhead — each entry creates a `Node` object with hash, key, value, and next pointer, plus the bucket array itself has empty slots.

**Practical Explanation:** For each entry, HashMap stores a `Node<K,V>` object containing:

- `int hash` (4 bytes)
- `K key` reference (8 bytes on 64-bit)
- `V value` reference (8 bytes)
- `Node<K,V> next` pointer (8 bytes)
- Object header (~16 bytes)

Total: ~44-48 bytes per Node, plus the actual key and value objects. The bucket array (`Node[]`) with default 0.75 load factor means ~25% of buckets are always empty.

**When/Why Used:** Memory awareness matters when storing millions of entries. A HashMap with 1 million entries uses ~50-80 MB just for the HashMap structure (excluding key/value objects). For memory-critical applications, consider alternatives.

**Real-World Scenario:** An in-memory cache with 10 million String-to-String entries consumed over 1 GB of heap just for the HashMap overhead. We switched to an off-heap solution (Chronicle Map) for the hot data and kept HashMap only for smaller working sets.

**Common Mistakes:** Not accounting for Node overhead when estimating memory — developers calculate `keys + values` and forget the per-entry Node cost. Also, small HashMaps (5-10 entries) have disproportionate overhead — the minimum bucket array is 16 slots. For very small maps, consider `Map.of()` (unmodifiable, optimized) or even simple arrays.

**Closing:** HashMap trades memory for speed — know the tradeoff, and for large-scale data, evaluate alternatives like EnumMap, array-based maps, or off-heap solutions.

---

# Q25. HashMap vs LinkedHashMap?

**Summary:** `HashMap` has no ordering guarantees; `LinkedHashMap` maintains insertion order (or access order) by linking entries with a doubly-linked list — at a small memory cost.

**Practical Explanation:** | Feature | HashMap | LinkedHashMap | |---------|---------|---------------| | Order | No guarantee | Insertion order (default) or access order | | Performance | Slightly faster | ~5-10% overhead for linked list maintenance | | Memory | Less | Extra prev/next pointers per entry | | Use case | General purpose | When order matters |

LinkedHashMap extends HashMap and adds `before`/`after` pointers to each entry, forming a doubly-linked list. Access-order mode (`new LinkedHashMap<>(16, 0.75f, true)`) moves accessed entries to the tail — perfect for LRU caches.

**When/Why Used:** JSON serialization where field order matters, maintaining config property order, implementing LRU caches, or when iteration order must match insertion order.

**Real-World Scenario:**

```
// LRU Cache using LinkedHashMap
Map<String, Data> lruCache = new LinkedHashMap<>(100, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry<String, Data> eldest) {
        return size() > MAX_CACHE_SIZE; // Auto-evict oldest
    }
};
```

**Common Mistakes:** Using HashMap and then wondering why JSON output has random field order. Assuming LinkedHashMap is significantly slower — the overhead is minimal. Forgetting access-order mode exists — it's the secret sauce for LRU caching.

**Closing:** Default to HashMap, but reach for LinkedHashMap whenever order matters — it's a drop-in replacement with predictable iteration.

---

# Q26. HashMap vs TreeMap?

**Summary:** `HashMap` is O(1) unordered; `TreeMap` is O(log n) and keeps keys sorted — choose based on whether you need sorted iteration or pure speed.

**Practical Explanation:** | Feature | HashMap | TreeMap | |---------|---------|---------| | Ordering | None | Natural order or Comparator | | get/put | O(1) average | O(log n) always | | Null keys | One allowed | Not allowed (with natural ordering) | | Implementation | Hash table | Red-black tree | | Memory | Node + array | TreeNode (left/right/parent/color) |

**When/Why Used:** HashMap for general-purpose key-value storage (99% of cases). TreeMap when you need sorted keys, range queries (`subMap()`, `headMap()`, `tailMap()`), or first/last operations (`firstKey()`, `lastKey()`).

**Real-World Scenario:**

```
// TreeMap for sorted leaderboard
TreeMap<Integer, String> leaderboard = new TreeMap<>(Comparator.reverseOrder());
leaderboard.put(score, playerName);

// Get top 10 players
leaderboard.entrySet().stream().limit(10).forEach(System.out::println);

// Range query — scores between 80 and 100
SortedMap<Integer, String> topScorers = leaderboard.subMap(80, 101);
```

**Common Mistakes:** Using TreeMap when you don't need sorting — O(log n) is slower than O(1) for every operation. Using HashMap then sorting keys separately when TreeMap gives you sorted iteration for free. Not implementing `Comparable` or providing a `Comparator` — `TreeMap` throws `ClassCastException` at runtime.

**Closing:** HashMap is the default; TreeMap is specialized — use it when you genuinely need sorted keys or range operations.

---

# Q27. HashMap vs Hashtable?

**Summary:** `Hashtable` is synchronized and legacy (Java 1.0); `HashMap` is unsynchronized and modern — there's virtually no reason to use `Hashtable` in modern Java.

**Practical Explanation:** | Feature | HashMap | Hashtable | |---------|---------|----------| | Thread-safe | No | Yes (synchronized) | | Null key/value | Allowed | NOT allowed | | Performance | Faster | Slower (synchronization overhead) | | Iterator | Fail-fast | Fail-fast (Enumerator is fail-safe) | | Inheritance | AbstractMap | Dictionary (legacy) | | Since | Java 1.2 | Java 1.0 |

**When/Why Used:** Never use `Hashtable` in new code. For thread-safe maps, use `ConcurrentHashMap` — it's faster than `Hashtable` because it uses fine-grained locking instead of synchronizing every method.

**Real-World Scenario:** Legacy codebases sometimes still have `Hashtable`. During modernization, we replace:

```
// Legacy
Hashtable<String, String> config = new Hashtable<>();

// Modern — if thread safety needed
ConcurrentHashMap<String, String> config = new ConcurrentHashMap<>();

// Modern — if single-threaded
HashMap<String, String> config = new HashMap<>();
```

**Common Mistakes:** Using `Hashtable` "for safety" — `ConcurrentHashMap` is both safer and faster. Using `Collections.synchronizedMap(new HashMap<>())` — it's basically the same as Hashtable (method-level synchronization) and has the same compound-operation problems.

**Closing:** `Hashtable` is dead — if you see it in code, it's a refactoring opportunity. `ConcurrentHashMap` is its modern, superior replacement.

---

# Q28. Why HashMap not synchronized?

**Summary:** HashMap is not synchronized by design — adding synchronization to every method would penalize the majority of use cases (single-threaded) for the minority (multi-threaded).

**Practical Explanation:** Most HashMap usage is thread-local — within a method, within a request, within a single thread. Adding `synchronized` to every `get()` and `put()` adds monitor acquisition/release overhead even when no contention exists. Java's philosophy is "pay only for what you use."

**When/Why Used:** When the JDK team redesigned collections in Java 1.2, they made the deliberate choice to unsynchronize everything (replacing `Vector` with `ArrayList`, `Hashtable` with `HashMap`). Developers opt-in to synchronization only when needed.

**Real-World Scenario:** A benchmarked comparison on a single thread:

- `HashMap.get()`: ~10-20 nanoseconds
- `Hashtable.get()`: ~40-60 nanoseconds
- `ConcurrentHashMap.get()`: ~15-25 nanoseconds (CAS, no lock for reads)

In a loop of 10 million operations, that synchronization overhead adds up to hundreds of milliseconds for zero benefit.

**Common Mistakes:** Wrapping HashMap with `Collections.synchronizedMap()` as a reflex — it adds the same overhead as Hashtable. Better to analyze: if reads dominate, `ConcurrentHashMap` is ideal. If you need a thread-local map, plain `HashMap` is fastest.

**Closing:** Non-synchronized by default was the right design call — it lets developers choose the appropriate concurrency strategy rather than paying a universal performance tax.

---

# Q29. Why ConcurrentHashMap?

**Summary:** `ConcurrentHashMap` provides thread-safe, high-performance concurrent access without locking the entire map — it's the standard choice for shared maps in multi-threaded Java applications.

**Practical Explanation:** Unlike `Hashtable` (which locks the entire map for every operation), `ConcurrentHashMap` uses fine-grained locking:

- **Java 7:** Segment-based locking (16 segments by default, 16 concurrent writers).
- **Java 8+:** Bucket-level locking with CAS (Compare-And-Swap) for lock-free reads, synchronized only on individual buckets for writes.

Reads are completely lock-free (volatile reads). This means hundreds of threads can read concurrently without blocking.

**When/Why Used:** Any shared Map in a multi-threaded application — caches, rate limiters, session stores, metrics counters, shared configuration.

**Real-World Scenario:**

```
@Service
public class RateLimiter {
    private final ConcurrentHashMap<String, AtomicInteger> counters = new ConcurrentHashMap<>();

    public boolean allowRequest(String clientId) {
        AtomicInteger count = counters.computeIfAbsent(clientId, k -> new AtomicInteger(0));
        return count.incrementAndGet() <= MAX_REQUESTS_PER_MINUTE;
    }
}
```

**Common Mistakes:** Using compound operations non-atomically — `if (!map.containsKey(key)) map.put(key, value)` is NOT atomic even with `ConcurrentHashMap`. Use `putIfAbsent()` or `computeIfAbsent()` instead. Also, iterators are weakly consistent — they reflect some but not necessarily all updates made after iterator creation.

**Closing:** `ConcurrentHashMap` is one of the most important classes in `java.util.concurrent` — it should be your default choice for any shared map.

---

# Q30. Java 7 ConcurrentHashMap?

**Summary:** Java 7's `ConcurrentHashMap` uses segment-based locking — the map is divided into 16 segments, each independently lockable, allowing up to 16 concurrent write operations.

**Practical Explanation:** The internal structure is an array of `Segment` objects, each being essentially a mini-HashMap with its own lock. When a thread writes, it only locks the specific segment that contains the target bucket. Other segments remain accessible to other threads.

- Default: 16 segments (concurrencyLevel=16).

- Each segment has its own `HashEntry[]` array, count, and threshold.
- Reads are lock-free (using volatile reads on `HashEntry.value`).
- Writes lock only the target segment.

**When/Why Used:** This was the gold standard for concurrent maps from Java 5 through Java 7. The segment approach was a significant improvement over `Hashtable`'s single-lock model.

**Real-World Scenario:** With 16 segments, up to 16 threads can write simultaneously (as long as they target different segments). In a web server with 200 request threads, this meant far less contention than `Hashtable`, though still some blocking when two threads targeted the same segment.

**Common Mistakes:** Assuming `size()` is exact — it requires locking all segments and can be expensive. Setting `concurrencyLevel` too high wastes memory. Also, segment-based locking was replaced in Java 8 because it had memory overhead (each segment is a full ReentrantLock) and the fixed 16-segment limit created uneven distribution.

**Closing:** Java 7's segment locking was clever but had limitations — Java 8's per-bucket approach is strictly better in both memory usage and concurrency.

---

# Q31. Java 8 ConcurrentHashMap?

**Summary:** Java 8's `ConcurrentHashMap` dropped segment-based locking in favor of per-bucket `synchronized` blocks + CAS operations — offering finer granularity, better memory efficiency, and higher concurrency.

**Practical Explanation:** Key changes in Java 8:

- **No more Segments** — the internal structure is a flat `Node[]` array, same as HashMap.
- **Reads:** Completely lock-free using `volatile` reads (Unsafe/VarHandle).
- **Writes:** CAS (Compare-And-Swap) for the first node in an empty bucket; `synchronized` on the first node for existing buckets.
- **Treeification:** Same as HashMap — buckets with 8+ nodes convert to red-black trees.
- **Concurrency:** Can scale to the number of buckets (thousands+), not limited to 16.

**When/Why Used:** This is the current implementation in Java 8+ and is significantly better than Java 7's version. More concurrent writers, less memory overhead, and better scalability.

**Real-World Scenario:**

```
// Lock-free read
Node<K,V> e = tabAt(tab, i); // volatile read, no lock

// CAS for empty bucket
casTabAt(tab, i, null, newNode); // atomic, no lock

// Synchronized only when bucket already occupied
synchronized (firstNodeInBucket) {
    // insert/update within this bucket only
}
```

**Common Mistakes:** Thinking `ConcurrentHashMap` serializes all writes — it only blocks writes to the SAME bucket. Two threads writing to different buckets proceed fully in parallel. Also, `size()` is still an approximation under concurrent modification — use `mappingCount()` for a `long` result on very large maps.

**Closing:** Java 8's `ConcurrentHashMap` is a masterclass in concurrent data structure design — per-bucket locking + CAS gives near-HashMap performance with full thread safety.

---

# Q32. Why no null in ConcurrentHashMap?

**Summary:** `ConcurrentHashMap` disallows null keys and values because `null` creates ambiguity in concurrent contexts — `get()` returning `null` could mean "key not found" or "value is null," and you can't distinguish these atomically.

**Practical Explanation:** With `HashMap`, you can call `containsKey()` after `get()` to check. But in a concurrent map, between your `get()` and `containsKey()` calls, another thread might insert or remove the key — the result is unreliable. Doug Lea (the author) chose to eliminate this ambiguity entirely by banning null.

```
// With HashMap (single-threaded, safe)
if (map.get(key) == null) {
    if (map.containsKey(key)) { /* value is null */ }
    else { /* key doesn't exist */ }
}

// With ConcurrentHashMap — between get() and containsKey(),
// another thread might change the map! Ambiguous and unsafe.
```

**When/Why Used:** This is a design decision you must know when migrating from `HashMap` to `ConcurrentHashMap`. If your code stores null values, you'll need to use a sentinel object instead.

**Real-World Scenario:**

```
// Migration from HashMap to ConcurrentHashMap
// Before
map.put("config", null); // null means "use default"

// After — use sentinel
private static final String USE_DEFAULT = "__DEFAULT__";
concurrentMap.put("config", USE_DEFAULT);
```

**Common Mistakes:** Attempting `concurrentMap.put(key, null)` — throws `NullPointerException` at runtime, not compile time. Not realizing that `HashMap.getOrDefault()` and `ConcurrentHashMap.getOrDefault()` behave differently with null values.

**Closing:** The no-null rule in `ConcurrentHashMap` is a conscious trade-off — clarity over flexibility. It forces cleaner API design and eliminates an entire category of concurrency bugs.

---

# Q33. computeIfAbsent use?

**Summary:** `computeIfAbsent()` atomically checks if a key exists and computes the value only if absent — it's the thread-safe, idiomatic way to implement lazy initialization in a Map.

**Practical Explanation:** `map.computeIfAbsent(key, k -> expensiveComputation(k))` does:

1. Check if key exists.
2. If absent, compute the value using the provided function.
3. Store and return it.
4. If present, return the existing value.

In `ConcurrentHashMap`, this entire operation is atomic — no race condition between check and insert.

**When/Why Used:** Caching, multimap construction, lazy initialization, memoization — any "get or create" pattern.

**Real-World Scenario:**

```
// Building a multimap (key → list of values)
Map<String, List<Order>> ordersByCustomer = new HashMap<>();
for (Order order : orders) {
    ordersByCustomer.computeIfAbsent(order.getCustomerId(), k -> new ArrayList<>())
        .add(order);
}

// Thread-safe cache with ConcurrentHashMap
ConcurrentHashMap<String, ExpensiveObject> cache = new ConcurrentHashMap<>();
ExpensiveObject obj = cache.computeIfAbsent(key, k -> createExpensiveObject(k));
// Guaranteed: createExpensiveObject called at most once per key
```

**Common Mistakes:** Confusing `computeIfAbsent()` with `putIfAbsent()` — `putIfAbsent()` requires you to compute the value upfront (even if it won't be used), while `computeIfAbsent()` only computes when needed. Also, the mapping function should not modify the map itself — it can cause deadlock in `ConcurrentHashMap`.

**Closing:** `computeIfAbsent()` is one of the most useful Map methods — it replaces the verbose check-then-put pattern with a single atomic, readable call.

---

# Q34. putIfAbsent use?

**Summary:** `putIfAbsent()` inserts a key-value pair only if the key is not already present — it's the atomic "insert if new" operation that prevents overwriting existing values.

**Practical Explanation:** `map.putIfAbsent(key, value)` does:

- If key doesn't exist → inserts the pair and returns `null`.
- If key exists → does nothing and returns the existing value.

Unlike `computeIfAbsent()`, the value is computed eagerly — even if the key already exists, you pay the cost of creating the value.

**When/Why Used:** Setting default values, first-write-wins semantics, preventing duplicate registrations.

**Real-World Scenario:**

```
// Register handler only if not already registered
ConcurrentHashMap<String, EventHandler> handlers = new ConcurrentHashMap<>();
EventHandler existing = handlers.putIfAbsent("orderCreated", new OrderHandler());
if (existing != null) {
    log.warn("Handler already registered for orderCreated");
}

// vs computeIfAbsent — value computed lazily
handlers.computeIfAbsent("orderCreated", event -> new OrderHandler());
// OrderHandler only created if key is absent (more efficient)
```

**Common Mistakes:** Using `putIfAbsent()` when the value is expensive to create — use `computeIfAbsent()` instead for lazy computation. Not using the return value — it tells you whether the insert happened. Also, `putIfAbsent()` returns `null` on success, which is unintuitive.

**Closing:** Use `putIfAbsent()` when you already have the value in hand; use `computeIfAbsent()` when the value is expensive to compute or should only be created on demand.

---

# Q35. HashMap in caching?

**Summary:** HashMap is commonly used as a simple in-memory cache for fast O(1) key-based lookups — but production caches need eviction, expiration, and thread-safety on top.

**Practical Explanation:** At its core, a cache is a key-value store with fast lookups — exactly what HashMap provides. But a raw HashMap lacks eviction (unbounded growth), TTL (entries never expire), thread-safety, and metrics. Production caches use specialized libraries built on similar principles.

**When/Why Used:** Simple, single-threaded, bounded caches for method-local lookups. For anything more, use Caffeine, Guava Cache, or distributed caches (Redis, Hazelcast).

**Real-World Scenario:**

```
// Simple in-memory cache (suitable for small, static data)
private final Map<String, Config> configCache = new HashMap<>();

public Config getConfig(String key) {
    return configCache.computeIfAbsent(key, k -> loadFromDatabase(k));
}

// Production cache — use Caffeine
Cache<String, Config> cache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterWrite(Duration.ofMinutes(5))
    .recordStats()
    .build();
```

**Common Mistakes:** Using a plain HashMap as a production cache without eviction — it grows unbounded and causes OOM. Not considering thread safety — singleton service + HashMap + concurrent requests = data corruption. Not adding TTL — stale data served indefinitely.

**Closing:** HashMap is the foundation of caching, but always use a proper caching library (Caffeine is the gold standard) for production systems.

---

# Q36. LRU cache role?

**Summary:** An LRU (Least Recently Used) cache evicts the least recently accessed entry when the cache is full — it's implemented using a HashMap for O(1) lookup combined with a doubly-linked list for O(1) eviction ordering.

**Practical Explanation:** The structure:

- **HashMap** maps keys to linked list nodes → O(1) lookup.
- **Doubly-linked list** tracks access order → the head is least recently used, tail is most recently used.
- On `get()`: move the node to the tail (most recent).
- On `put()` when full: remove the head node (least recent) and insert the new node at tail.

All operations are O(1).

**When/Why Used:** Database query result caching, API response caching, session management, connection pooling — anywhere you have limited memory and want to keep the most useful data.

**Real-World Scenario:**

```
// Simple LRU using LinkedHashMap
Map<String, Data> lruCache = new LinkedHashMap<>(maxSize, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry<String, Data> eldest) {
        return size() > maxSize;
    }
};

// Or implement from scratch (common interview question)
class LRUCache<K, V> {
    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final DoublyLinkedList<K, V> list = new DoublyLinkedList<>();
    // get(), put() both O(1)
}
```

**Common Mistakes:** Implementing LRU with only a HashMap (no ordering) or only a LinkedList (O(n) lookup). Using `Collections.synchronizedMap(LinkedHashMap)` for concurrent LRU — use Caffeine instead. Not considering cache sizing properly — too small = poor hit rate, too large = memory waste.

**Closing:** LRU is one of the most common interview questions AND one of the most practical data structures — knowing how HashMap + doubly-linked list work together is essential.

---

# Q37. Memory leak via HashMap?

**Summary:** HashMap can cause memory leaks when entries accumulate but are never removed — strong references to keys/values prevent garbage collection, even when the data is logically no longer needed.

**Practical Explanation:** Common memory leak patterns with HashMap:

1. **Unbounded growth** — entries added but never removed (missing eviction policy).
2. **Mutable keys** — mutated after insertion, become unreachable but not GC'd.
3. **Forgotten references** — static/singleton HashMap holds references to objects that should have been released.
4. **Listener/callback maps** — registrations without deregistrations.

**When/Why Used:** Memory leak detection is a critical debugging skill. Use heap dump analysis (Eclipse MAT, JProfiler) to identify retained objects.

**Real-World Scenario:**

```
// MEMORY LEAK — session map never cleaned up
@Service
public class SessionManager {
    private final Map<String, UserSession> sessions = new HashMap<>();

    public void createSession(String token, UserSession session) {
        sessions.put(token, session); // Entries added...
    }
    // No removeSession() or expiration! Map grows forever.

    // FIX — use WeakHashMap or add TTL-based eviction
    private final Map<String, UserSession> sessions =
        Collections.synchronizedMap(new WeakHashMap<>());
    // Or better: use Caffeine with expireAfterAccess
}
```

**Common Mistakes:** Using `static HashMap` without lifecycle management. Not profiling memory in production. Thinking `WeakHashMap` solves everything — it only weakly references keys, not values (values are strongly referenced). Also, `ThreadLocal` + thread pools = classic leak — entries survive across requests.

**Closing:** Memory leaks with HashMap are silent killers — the application degrades over days/weeks until OOM. Regular heap dump analysis and proper eviction policies are your defense.

---

# Q38. Optimize HashMap?

**Summary:** Optimize HashMap by setting proper initial capacity to avoid resizing, writing good `hashCode()` methods to minimize collisions, and choosing the right load factor.

**Practical Explanation:** Key optimization strategies:

1. **Initial capacity:** If you know the expected size, set `capacity = expectedSize / loadFactor + 1`. Prevents expensive resize operations.
2. **hashCode() quality:** Use all meaningful fields, distribute evenly. `Objects.hash()` is reliable. Avoid `return constant`.
3. **Load factor:** 0.75 is optimal for most cases. Lower means fewer collisions but more memory.
4. **Key type:** Use immutable keys with good hash distribution — `String`, `Integer`, `Long`.

**When/Why Used:** Optimization matters for large HashMaps (100K+ entries) or hot-path code where HashMap operations are called millions of times.

**Real-World Scenario:**

```
// Before optimization — 20 resizes during bulk load
Map<String, Data> map = new HashMap<>(); // default capacity 16
for (int i = 0; i < 1_000_000; i++) {
    map.put(keys[i], data[i]);
}

// After optimization — zero resizes
Map<String, Data> map = new HashMap<>(1_400_000); // 1M / 0.75 + buffer
for (int i = 0; i < 1_000_000; i++) {
    map.put(keys[i], data[i]);
}
// Result: ~30% faster bulk loading, no GC pressure from discarded arrays
```

**Common Mistakes:** Over-optimizing small maps (under 1000 entries) — the JVM handles them fine with defaults. Micro-tuning load factor without profiling. Using `HashMap.size()` to calculate new HashMap capacity — you need to account for load factor.

**Closing:** Pre-sizing and good `hashCode()` are the two biggest wins — they're simple to implement and yield measurable performance improvements for large datasets.

---

# Q39. Millions of records?

**Summary:** When storing millions of records in a HashMap, pre-size the capacity, ensure excellent hash distribution, tune JVM heap and GC, and consider whether HashMap is still the right data structure.

**Practical Explanation:** At scale, every per-entry overhead multiplies:

- 10M entries × ~48 bytes/Node = ~480 MB just for Nodes.
- Plus keys + values + bucket array.
- GC pressure increases significantly.

Strategies:

1. **Pre-size:** `new HashMap<>((int)(expectedSize / 0.75) + 1)` — avoid all resizes.
2. **Tune GC:** Use G1GC or ZGC, increase heap (`-Xmx`).
3. **Minimize object creation:** Use primitive-specialized maps (Eclipse Collections `IntObjectHashMap`, Koloboke).
4. **Consider alternatives:** Off-heap maps (Chronicle Map), databases, Redis.

**When/Why Used:** Big data processing, large in-memory caches, analytics, batch jobs processing millions of records.

**Real-World Scenario:**

```
// Storing 50M user sessions — standard HashMap is too expensive
// Option 1: Primitive-specialized map (avoids boxing)
IntObjectHashMap<Session> sessions = new IntObjectHashMap<>(50_000_000);

// Option 2: Off-heap (Chronicle Map)
ChronicleMap<Long, Session> sessions = ChronicleMap
    .of(Long.class, Session.class)
    .entries(50_000_000)
    .averageValueSize(256)
    .createPersistedTo(new File("/data/sessions.dat"));

// Option 3: Just use Redis
// sessions stored externally, no heap pressure
```

**Common Mistakes:** Loading everything into memory when only a subset is actively used — consider pagination or lazy loading. Not monitoring GC — large HashMaps cause long GC pauses. Auto-boxing primitives as keys (`int` → `Integer`) wastes significant memory at scale.

**Closing:** At millions of records, you're doing capacity planning, not just coding — the right answer often involves architectural decisions beyond just HashMap configuration.

---

# Q40. HashMap vs Redis?

**Summary:** HashMap is an in-memory, single-JVM data structure; Redis is a distributed, persistent, feature-rich external cache — use HashMap for local, single-instance data and Redis for shared, distributed state.

**Practical Explanation:** | Feature | HashMap | Redis | |---------|---------|-------| | Scope | Single JVM | Distributed across network | | Persistence | Lost on restart | Configurable (RDB/AOF) | | Data types | Key-Value only | Strings, Lists, Sets, Hashes, Sorted Sets, Streams | | TTL/Expiration | Manual | Built-in | | Memory limit | JVM heap | Configurable, eviction policies | | Network | No network hop | Network latency (~0.5-1ms) | | Scalability | Single node | Clustering, replication, sentinel |

**When/Why Used:**

- **HashMap:** Local caching within a single service instance, method-local computations, small lookup tables that don't need sharing.
- **Redis:** Session management across multiple instances, distributed rate limiting, shared cache invalidation, pub/sub messaging, leaderboards.

**Real-World Scenario:**

```
// L1 cache — HashMap (local, fastest)
private final Map<String, Product> l1Cache = new ConcurrentHashMap<>();

// L2 cache — Redis (shared across instances)
@Cacheable(value = "products", key = "#productId")
public Product getProduct(String productId) {
    return productRepository.findById(productId);
}

// Multi-level caching strategy
public Product getProduct(String id) {
    Product p = l1Cache.get(id);           // ~10ns, local
    if (p == null) {
        p = redisTemplate.opsForValue().get("product:" + id);  // ~1ms, network
        if (p == null) {
            p = productRepository.findById(id);  // ~10ms, database
            redisTemplate.opsForValue().set("product:" + id, p, Duration.ofMinutes(30));
        }
        l1Cache.put(id, p);
    }
    return p;
}
```

**Common Mistakes:** Using HashMap for data that needs to survive restarts — it's purely in-memory. Using Redis for data that never needs sharing — unnecessary network latency and infrastructure complexity. Not considering cache invalidation across instances — local HashMap caches can serve stale data when another instance updates the source.

**Closing:** HashMap and Redis are complementary, not competitors — the best architectures use both in a tiered caching strategy (L1 local + L2 distributed).

---

*End of HashMap Deep Dive — 40 Interview Q&A*

# Spring Boot Internals — Interview Answers (Senior Java Engineer Perspective)

---

## Q1. What is Spring Boot?

Spring Boot is an opinionated framework built on top of the Spring ecosystem that lets you build production-ready applications with minimal configuration.

In practice, before Spring Boot, setting up a Spring project meant writing tons of XML or Java config — datasource beans, view resolvers, dispatcher servlets, etc. Spring Boot eliminates all that boilerplate. It gives you sensible defaults out of the box: embedded Tomcat, auto-configured beans, and starter dependencies.

**Real-world scenario:** When I need to spin up a new microservice, I go to Spring Initializr, pick my starters (Web, JPA, Security), and within minutes I have a running REST API — no XML, no manual server setup.

**Common mistake:** Developers sometimes think Spring Boot is a different framework from Spring. It's not — it's just a layer on top that makes Spring easier to use.

Spring Boot is essentially the modern way to do Spring development, and in any enterprise project today, it's the default choice.

---

# Q2. Why Spring Boot?

Spring Boot exists because traditional Spring setup was painful — too much boilerplate, too many XML files, and manual server configuration.

The key reasons we use Spring Boot in real projects:

- **Auto-configuration** — it detects what's on your classpath and configures beans automatically
- **Embedded server** — no need to deploy a WAR to an external Tomcat; just run a JAR
- **Starter dependencies** — one dependency pulls in everything you need (e.g., `spring-boot-starter-web` gives you Spring MVC, Jackson, Tomcat, validation)
- **Production-ready features** — health checks, metrics, externalized config out of the box

**Real-world scenario:** In my last project, we migrated from a legacy Spring MVC app with 15+ XML config files to Spring Boot. We cut the configuration code by 80% and deployment became a simple `java -jar app.jar`.

**Edge case:** Spring Boot's opinionated defaults work great 90% of the time, but sometimes you need to fight those defaults for custom use cases — knowing how to override them is key.

Spring Boot dramatically reduces time-to-production, which is why it's the industry standard for Java backend services.

---

# Q3. What is auto-configuration?

Auto-configuration is Spring Boot's mechanism to automatically configure beans based on what libraries are on your classpath — so you don't have to manually define them.

For example, if Spring Boot sees `spring-boot-starter-data-jpa` and an H2 driver on the classpath, it will automatically configure a `DataSource`, `EntityManagerFactory`, and `TransactionManager` without you writing a single `@Bean` definition.

**How it works:** Spring Boot ships with hundreds of auto-configuration classes (like `DataSourceAutoConfiguration`, `JacksonAutoConfiguration`). Each one uses `@Conditional` annotations to check: "Is a certain class present? Is a certain property set? Has the user already defined this bean?" If conditions are met, the bean is created automatically.

**Real-world scenario:** When I add `spring-boot-starter-data-redis` to my project, Spring Boot automatically configures a `RedisTemplate` and `RedisConnectionFactory`. I just inject `RedisTemplate` and start using it.

**Common mistake:** Assuming auto-configuration is magic and not understanding what beans it creates. Use `--debug` flag or `spring.boot.autoconfiguration` report to see what's being configured and what's being skipped.

Auto-configuration is the heart of Spring Boot — understanding it separates junior developers from senior ones.

---

# Q4. How does auto-configuration work internally?

Internally, auto-configuration works through a combination of `@EnableAutoConfiguration`, the `spring.factories` / `AutoConfiguration.imports` file, and `@Conditional` annotations.

Here's the flow:

1. `@SpringBootApplication` includes `@EnableAutoConfiguration`
2. This triggers `AutoConfigurationImportSelector`, which loads all auto-configuration class names from `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (or `spring.factories` in older versions)
3. Each auto-config class is annotated with conditions like `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`
4. Spring evaluates these conditions — only classes whose conditions pass get loaded into the context

**Real-world scenario:** When debugging why a bean wasn't being created in one of our services, I checked the auto-config report (`/actuator/conditions`) and found that `@ConditionalOnClass(RedisClient.class)` was failing because we had the wrong Redis client dependency.

**Common mistake:** Not understanding the ordering — `@AutoConfigureAfter` and `@AutoConfigureBefore` control the order auto-config classes are processed. If your custom auto-config depends on another, you must declare the ordering.

This is the kind of internal knowledge that helps you debug production issues fast.

---

# Q5. What is `spring.factories`?

`spring.factories` is a metadata file located at `META-INF/spring.factories` that Spring Boot uses to discover and load auto-configuration classes, initializers, and listeners.

It's essentially a key-value mapping where the key is a fully qualified interface name and the value is a comma-separated list of implementation classes. For auto-configuration, the key is `org.springframework.boot.autoconfigure.EnableAutoConfiguration`.

```
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
  com.example.MyAutoConfiguration,\
  com.example.AnotherAutoConfiguration
```

**Real-world scenario:** When I built a custom Spring Boot starter for our company's internal logging library, I registered my auto-configuration class in `spring.factories` so that any project using our starter would automatically get the logging beans configured.

**Important note:** Starting from Spring Boot 3.x, the preferred mechanism has shifted to `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` — a simpler file with one class per line. But `spring.factories` still works and you'll encounter it in many existing projects.

**Common mistake:** Putting your app's configuration classes in `spring.factories` instead of just using `@Configuration`. `spring.factories` is meant for library/starter auto-configurations, not for your application's own beans.

Understanding `spring.factories` is essential if you ever need to build reusable starters or debug auto-config loading issues.

---

# Q6. What is `@EnableAutoConfiguration`?

`@EnableAutoConfiguration` is the annotation that tells Spring Boot to start its auto-configuration mechanism — scanning and loading all eligible auto-configuration classes.

Under the hood, it imports `AutoConfigurationImportSelector`, which reads the list of auto-config classes from `spring.factories` / `AutoConfiguration.imports`, evaluates all `@Conditional` annotations, and registers the qualifying beans into the application context.

**Real-world scenario:** You rarely use `@EnableAutoConfiguration` directly because `@SpringBootApplication` already includes it. But when I needed fine-grained control — for example, excluding certain auto-configs — I used:

```
@EnableAutoConfiguration(exclude = {DataSourceAutoConfiguration.class})
```

This was useful when our service didn't need a database but had JPA on the classpath transitively.

**Common mistake:** Using `@EnableAutoConfiguration` alongside `@SpringBootApplication` — that's redundant. `@SpringBootApplication` is already a meta-annotation that includes it.

This annotation is the trigger for everything that makes Spring Boot "magic" — it's the entry point to the auto-config pipeline.

---

# Q7. What is `@SpringBootApplication`?

`@SpringBootApplication` is a convenience meta-annotation that combines three core annotations into one: `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`.

- **`@Configuration`** — marks the class as a source of bean definitions
- **`@EnableAutoConfiguration`** — triggers auto-configuration
- **`@ComponentScan`** — scans the package (and sub-packages) of the annotated class for `@Component`, `@Service`, `@Repository`, `@Controller`, etc.

**Real-world scenario:** In every Spring Boot project, your main class looks like:

```
@SpringBootApplication
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

**Common mistake:** Placing your main class in a deeply nested package like `com.company.project.config`. Since `@ComponentScan` scans from the annotated class's package downward, beans in `com.company.project.service` would NOT be picked up. Always place your main class at the root package.

**Edge case:** You can customize scanning using `scanBasePackages`:

```
@SpringBootApplication(scanBasePackages = "com.company")
```

It's a simple annotation, but understanding what it's composed of helps you debug component scanning and auto-config issues.

---

# Q8. How does a Spring Boot application start?

When you call `SpringApplication.run()`, a well-defined startup sequence kicks off that bootstraps the entire application.

**The internal flow:**

1. **Create `SpringApplication` instance** — detects the application type (Servlet, Reactive, or None)
2. **Load `SpringApplicationRunListeners`** — fires startup events
3. **Prepare the Environment** — reads `application.properties/yml`, system properties, env variables
4. **Create the ApplicationContext** — `AnnotationConfigServletWebServerApplicationContext` for web apps
5. **Refresh the context** — this is where bean definitions are loaded, auto-config runs, dependencies are injected, and the embedded server starts
6. **Call `ApplicationRunner` / `CommandLineRunner`** — runs any startup logic you've defined
7. **Application is ready** — fires `ApplicationReadyEvent`

**Real-world scenario:** I've used `ApplicationRunner` to pre-load cache data on startup and `ApplicationReadyEvent` listeners to send a health notification to our monitoring system when a service comes up.

**Common mistake:** Putting heavy initialization logic in `@PostConstruct` — this blocks the entire context refresh. Use `ApplicationRunner` or async event listeners instead for non-critical startup tasks.

Understanding this flow is critical when you need to debug slow startups or ordering issues between beans.

---

# Q9. What is an embedded server in Spring Boot?

An embedded server means the web server (Tomcat, Jetty, or Undertow) runs inside your application as a dependency — you don't deploy to an external server.

In traditional Java EE, you'd build a WAR and deploy it to a standalone Tomcat. With Spring Boot, the server is just another JAR on the classpath. When your app starts, Spring Boot auto-configures and starts the server programmatically.

**Real-world scenario:** In our microservices architecture, every service runs as a self-contained JAR with embedded Tomcat. Deployment is just `java -jar order-service.jar`. This simplifies containerization — our Dockerfile is literally:

```
FROM eclipse-temurin:17
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

**Switching servers** is as simple as excluding Tomcat and adding Jetty:

```
<dependency>
    <groupId>spring-boot-starter-web</groupId>
    <exclusions>
        <exclusion>spring-boot-starter-tomcat</exclusion>
    </exclusions>
</dependency>
<dependency>spring-boot-starter-jetty</dependency>
```

**Common mistake:** Not tuning the embedded server for production — thread pool size, connection timeouts, and max connections still matter. Defaults are fine for dev, not for high-traffic production.

Embedded servers are a key enabler for the microservices + Docker deployment model that's standard today.

---

# Q10. What are starter dependencies?

Starter dependencies are curated, pre-configured bundles of dependencies that Spring Boot provides so you don't have to manually manage individual library versions.

Instead of adding Spring MVC, Jackson, Tomcat, Hibernate Validator, and logging dependencies one by one (and worrying about version compatibility), you just add:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

**Common starters I use daily:**

- `spring-boot-starter-web` — REST APIs
- `spring-boot-starter-data-jpa` — database with Hibernate
- `spring-boot-starter-security` — authentication/authorization
- `spring-boot-starter-test` — JUnit, Mockito, MockMvc all-in-one
- `spring-boot-starter-actuator` — production monitoring

**Real-world scenario:** When building a new microservice, I add 3-4 starters and I'm ready to go. The BOM (`spring-boot-dependencies`) ensures all transitive versions are compatible — no more "dependency hell."

**Common mistake:** Adding individual libraries that conflict with what a starter already provides. For example, adding a different Jackson version alongside `spring-boot-starter-web` can cause serialization issues.

Starters are one of the biggest productivity boosters in Spring Boot — they handle dependency management so you can focus on business logic.

---

# Q11. How do you override auto-configuration?

The simplest way is to define your own `@Bean` — Spring Boot's auto-configuration is designed to back off when it detects a user-defined bean of the same type.

Most auto-config classes use `@ConditionalOnMissingBean`, which means: "Only create this bean if the user hasn't already defined one."

**Real-world scenario:** Spring Boot auto-configures a `DataSource` using `application.properties`. But in one project, we needed a custom `DataSource` with connection pool tuning and multi-tenancy routing:

```
@Configuration
public class DataSourceConfig {
    @Bean
    public DataSource dataSource() {
        HikariDataSource ds = new HikariDataSource();
        ds.setMaximumPoolSize(50);
        ds.setConnectionTimeout(3000);
        // custom routing logic
        return new TenantRoutingDataSource(ds);
    }
}
```

Spring Boot sees this bean and skips `DataSourceAutoConfiguration` for that bean type.

**Other ways to override:**

- Override properties in `application.properties` (e.g., `server.port=9090`)
- Use `@EnableAutoConfiguration(exclude = ...)` to completely disable a specific auto-config
- Use `@AutoConfigureBefore/@AutoConfigureAfter` for ordering your config

**Common mistake:** Defining a bean with a different name but same type, then wondering why you have two beans and getting `NoUniqueBeanDefinitionException`. Use `@Primary` if you need to keep both.

Knowing how to override auto-config cleanly is essential — it's where you draw the line between Boot's defaults and your custom requirements.

---

# Q12. How do you disable auto-configuration?

You can disable specific auto-configuration classes using the `exclude` attribute on `@SpringBootApplication` or `@EnableAutoConfiguration`.

```
@SpringBootApplication(exclude = {
    DataSourceAutoConfiguration.class,
    SecurityAutoConfiguration.class
})
public class MyApplication { }
```

Alternatively, you can use the property-based approach:

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

**Real-world scenario:** In a microservice that only processes messages from Kafka (no database), JPA was on the classpath as a transitive dependency. The app failed to start because `DataSourceAutoConfiguration` couldn't find a datasource URL. Excluding it fixed the issue instantly.

**When to use which approach:**

- `exclude` in annotation — when you always want it disabled for that app
- `spring.autoconfigure.exclude` in properties — when it varies per environment or profile

**Common mistake:** Excluding a parent auto-config but not its related ones. For example, excluding `DataSourceAutoConfiguration` but not `HibernateJpaAutoConfiguration` — the latter may still fail. Check the auto-config dependency chain.

**Pro tip:** Run with `--debug` to see the full auto-configuration report — it shows positive matches (applied) and negative matches (skipped), which tells you exactly what to exclude.

Disabling auto-config is a common requirement in microservice architectures where transitive dependencies pull in things you don't need.

---

# Q13. What are Profiles in Spring Boot?

Profiles are Spring's mechanism for environment-specific configuration — allowing you to have different beans and properties for dev, staging, and production environments.

In practice, you create separate config files:

- `application-dev.yml` — local database, debug logging
- `application-staging.yml` — staging database, moderate logging
- `application-prod.yml` — production database, minimal logging, SSL enabled

You can also conditionally load beans:

```
@Configuration
@Profile("prod")
public class ProdCacheConfig {
    @Bean
    public CacheManager cacheManager() {
        return new RedisCacheManager(...); // Redis in prod
    }
}

@Configuration
@Profile("dev")
public class DevCacheConfig {
    @Bean
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager(); // In-memory for dev
    }
}
```

**Real-world scenario:** In our production setup, we use profiles to switch between AWS S3 (prod) and local filesystem storage (dev) for file uploads, and between real payment gateways (prod) and sandbox/mock implementations (dev).

**Common mistake:** Hardcoding environment-specific values without profiles and relying on code changes for each deployment. Also, accidentally running with the wrong profile — always verify with the startup log: `The following profiles are active: prod`.

Profiles are fundamental to any professional Spring Boot project — they keep your code clean and your deployments safe.

---

# Q14. How do you activate profiles?

You activate profiles using `spring.profiles.active`, and there are multiple ways to set it depending on your deployment strategy.

**Common approaches (in order of precedence, high to low):**

1. **Command-line argument** (most common in production):

   ```
   java -jar app.jar --spring.profiles.active=prod
   ```

2. **Environment variable** (ideal for Docker/Kubernetes):

```
SPRING_PROFILES_ACTIVE=prod java -jar app.jar
```

3. **JVM system property:**

```
java -Dspring.profiles.active=prod -jar app.jar
```

4. **In `application.properties`** (for default profile):

```
spring.profiles.active=dev
```

5. **Programmatically:**

```
SpringApplication app = new SpringApplication(MyApp.class);
app.setAdditionalProfiles("prod");
app.run(args);
```

**Real-world scenario:** In our Kubernetes deployments, we set `SPRING_PROFILES_ACTIVE` as an environment variable in the deployment YAML. This way, the same Docker image runs in any environment — only the profile changes.

**Common mistake:** Setting `spring.profiles.active=dev` in `application.properties` and forgetting to override it in production. The property file value is lowest priority, so command-line or env vars will override it, but it's still risky.

**Pro tip:** You can activate multiple profiles: `spring.profiles.active=prod,redis,aws`

Profile activation is a deployment concern, not a code concern — keep it externalized.

---

# Q15. What is externalized configuration?

Externalized configuration means keeping your application's configurable values outside the source code — so you can change behavior without recompiling or redeploying.

Spring Boot supports multiple external config sources with a well-defined precedence order (highest to lowest):

1. Command-line arguments
2. JVM system properties
3. OS environment variables
4. Profile-specific `application-{profile}.properties`
5. `application.properties` / `application.yml`
6. `@PropertySource` annotations
7. Default properties

**Real-world scenario:** In our microservices, database URLs, API keys, feature flags, and thread pool sizes are all externalized. In Kubernetes, we use ConfigMaps for non-sensitive config and Secrets for credentials:

```
# Kubernetes ConfigMap
data:
  SPRING_DATASOURCE_URL: jdbc:postgresql://db-host:5432/orders
  APP_CACHE_TTL: "300"
```

Spring Boot automatically maps `APP_CACHE_TTL` environment variable to `app.cache.ttl` property (relaxed binding).

**Common mistake:** Hardcoding URLs, timeouts, or feature toggles in Java code. If it might change between environments or needs runtime tuning, externalize it.

**Edge case:** Property precedence can bite you — an environment variable will override your `application.yml` value, which can be confusing during debugging.

Externalized configuration is a Twelve-Factor App principle — it's non-negotiable in production-grade applications.

---

# Q16. `application.yml` vs `application.properties`?

Both serve the same purpose — externalizing configuration — but `application.yml` uses YAML's hierarchical structure while `.properties` uses flat key-value pairs.

**`application.properties` (flat):**

```
spring.datasource.url=jdbc:mysql://localhost:3306/db
spring.datasource.username=root
spring.datasource.hikari.maximum-pool-size=20
```

**`application.yml` (hierarchical):**

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/db
    username: root
    hikari:
      maximum-pool-size: 20
```

**Key differences:**

- YAML supports nested structures — less repetition, more readable for deeply nested configs
- YAML supports multi-document files (multiple profiles in one file using `---`)
- Properties files are simpler and less error-prone (no indentation issues)
- YAML supports lists and maps more naturally

**Real-world scenario:** In most of my projects, we use `application.yml` because microservice configs tend to be deeply nested (datasource, security, actuator, custom app properties). YAML makes this much more readable.

**Common mistake:** YAML indentation errors — a misplaced space can silently change which property you're setting. Also, mixing tabs and spaces in YAML will break your application. Use an IDE with YAML support.

**Pro tip:** You can use both in the same project — properties take precedence over YAML. But pick one and be consistent.

Choose YAML for complex configs, properties for simple ones — consistency within the team matters most.

---

# Q17. What is Spring Boot Actuator?

Actuator provides production-ready operational endpoints that let you monitor, manage, and inspect your running Spring Boot application.

Out of the box, it gives you endpoints like:

- `/actuator/health` — application health status (DB, disk, custom checks)
- `/actuator/metrics` — JVM memory, CPU, HTTP request stats, custom metrics
- `/actuator/info` — application info (build version, git commit)
- `/actuator/env` — all configuration properties
- `/actuator/loggers` — view and change log levels at runtime
- `/actuator/threaddump` — thread dump for debugging

**Real-world scenario:** In our production setup, `/health` is used by Kubernetes for liveness and readiness probes. Prometheus scrapes `/actuator/prometheus` for metrics, and we've written custom health indicators for downstream service checks:

```java
@Component
public class PaymentGatewayHealthIndicator implements HealthIndicator {
    public Health health() {
        boolean reachable = paymentClient.ping();
        return reachable ? Health.up().build()
                         : Health.down().withDetail("error", "Gateway unreachable").build();
    }
}
```

**Common mistake:** Exposing all actuator endpoints publicly in production without security. Endpoints like `/env` and `/heapdump` can leak sensitive data.

Actuator is not optional in production — it's how you achieve observability in Spring Boot services.

---

# Q18. How do you secure Actuator endpoints?

You secure Actuator by restricting which endpoints are exposed and who can access them, using a combination of exposure configuration and Spring Security.

**Step 1: Control exposure** — By default, only `/health` is exposed over HTTP. Selectively expose what you need:

```
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, prometheus
        # Never expose: env, heapdump, shutdown in prod
  endpoint:
    health:
      show-details: when-authorized
```

**Step 2: Secure with Spring Security:**

```
@Configuration
public class ActuatorSecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.requestMatcher(EndpointRequest.toAnyEndpoint())
            .authorizeRequests()
            .requestMatchers(EndpointRequest.to("health", "info")).permitAll()
            .anyRequest().hasRole("ACTUATOR_ADMIN")
            .and().httpBasic();
        return http.build();
    }
}
```

**Step 3: Change the base path** (security by obscurity as additional layer):

```
management:
  server:
    port: 9090  # Separate port, not exposed publicly
  endpoints:
    web:
      base-path: /internal/manage
```

**Real-world scenario:** In our Kubernetes setup, we run actuator on a separate port (9090) that's only accessible within the cluster — external load balancers only route to port 8080. Health endpoints are open for K8s probes, everything else requires authentication.

**Common mistake:** Using `include: "*"` to expose everything and forgetting that endpoints like `/heapdump` can dump your entire JVM memory (including secrets) and `/shutdown` can literally kill your application.

Treat Actuator endpoints like admin panels — lock them down with the principle of least privilege.

---

# Q19. What are Spring Boot DevTools?

DevTools is a Spring Boot module that improves the developer experience by providing automatic restarts, live reload, and development-specific defaults.

Add it with:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
```

**Key features:**

- **Automatic restart** — when classpath files change (code save), the app restarts using a custom classloader (much faster than a cold restart)
- **LiveReload** — triggers browser refresh when resources change (templates, static files)
- **Relaxed caching** — disables template caching, enables debug logging
- **Remote debugging support** — allows DevTools features over HTTP for remote apps

**Real-world scenario:** During development of a REST API, every time I save a Java file in IntelliJ, DevTools automatically restarts the context in ~2 seconds instead of 15+ seconds for a full restart. For Thymeleaf template changes, it's instant with LiveReload.

**Important:** DevTools is automatically disabled in production — it detects when you run from a packaged JAR (`java -jar`) and deactivates itself. The `optional=true` flag also prevents it from being a transitive dependency.

**Common mistake:** Adding DevTools without `optional=true` or proper scope — it could accidentally end up in production builds. Also, confusing DevTools restart with JRebel-style hot reload — DevTools still restarts the context, just faster.

DevTools is a must-have during development — the feedback loop improvement alone makes it worth it.

---

# Q20. Hot reload vs restart?

These are two different strategies for applying code changes during development — **restart** recreates the Spring ApplicationContext using a fast classloader, while **hot reload (live reload)** refreshes specific resources without restarting the context.

**Restart (DevTools):**

- Triggered when Java classes change on the classpath
- Uses two classloaders: a **base classloader** (for third-party JARs, rarely changes) and a **restart classloader** (for your project classes)
- Only the restart classloader is discarded and recreated — making it much faster (2-5 seconds vs 15-30 seconds cold start)
- All beans are re-created, context is fully refreshed

**Hot Reload / Live Reload:**

- Triggered when static resources or templates change (HTML, CSS, JS)
- No context restart — just the changed resource is picked up
- Browser auto-refreshes via LiveReload protocol
- Near-instant feedback

**Real-world scenario:** When working on a full-stack Spring Boot + Thymeleaf app, I modify a Java controller (triggers restart in ~3 seconds) vs modifying a Thymeleaf template (hot reloaded instantly without restart).

**True Hot Reload (JRebel/DCEVM):**

- Tools like JRebel can hot-swap method bodies, add fields/methods without any restart
- More powerful but commercial (JRebel) or requires special JVM setup (DCEVM)

**Common mistake:** Expecting DevTools to hot-swap code like JRebel — it doesn't. It's a fast restart, not true hot code replacement. Adding new methods or changing method signatures always requires a restart.

Understand the distinction — restart for code changes, hot reload for resources. Use the right tool for the right job.

---

# Q21. WAR vs JAR deployment?

JAR deployment packages your application with an embedded server as a standalone executable, while WAR deployment produces a web archive meant to be deployed on an external application server.

**JAR (Modern, recommended):**

```
java -jar myapp.jar
```

- Self-contained — embedded Tomcat/Jetty inside
- Perfect for microservices, Docker, Kubernetes
- Spring Boot's default packaging

**WAR (Legacy/Enterprise):**

- Deployed to external Tomcat, WildFly, WebSphere
- Required when organization mandates shared application servers
- Needs `SpringBootServletInitializer`:

```
public class MyApp extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
        return builder.sources(MyApp.class);
    }
}
```

**Real-world scenario:** In a greenfield microservice project, we always use JAR. But I've worked with a banking client where compliance required deployment on a hardened WebSphere server — WAR was the only option. Spring Boot supports both, so the same codebase can be packaged either way.

**Common mistake:** Choosing WAR "just because that's how we've always done it." Unless there's a specific infrastructure constraint, JAR is simpler, more portable, and aligned with modern deployment practices.

Go with JAR unless you have a concrete reason for WAR — in 2024+, JAR is the standard.

---

# Q22. What is `@ConfigurationProperties`?

`@ConfigurationProperties` is a type-safe way to bind a group of related configuration properties to a POJO, instead of injecting them one by one with `@Value`.

```
# application.yml
app:
  payment:
    gateway-url: https://api.payment.com
    timeout: 5000
    retry-count: 3
    enabled: true
```

```
@Component
@ConfigurationProperties(prefix = "app.payment")
public class PaymentProperties {
    private String gatewayUrl;
    private int timeout;
    private int retryCount;
    private boolean enabled;
    // getters and setters
}
```

Now you inject `PaymentProperties` wherever needed — clean, validated, and testable.

**Why use it over `@Value`:**

- Groups related config logically
- Supports validation with `@Validated` + JSR-303 annotations (`@NotNull`, `@Min`, `@Max`)
- IDE auto-completion when you add `spring-boot-configuration-processor`
- Immutable support with constructor binding
- Much easier to test — just create the POJO in your test

**Real-world scenario:** In our e-commerce platform, each external integration (payment, shipping, notification) has its own `@ConfigurationProperties` class. This keeps config organized and validates at startup — if a required URL is missing, the app fails fast.

**Common mistake:** Forgetting to enable it — either annotate with `@Component` or use `@EnableConfigurationProperties(PaymentProperties.class)` on a config class. Without this, the binding won't happen.

For anything beyond a single property, always prefer `@ConfigurationProperties` over `@Value` — it's the professional approach.

---

# Q23. What is `@Value`?

`@Value` is a Spring annotation that injects a single externalized property value directly into a field, constructor parameter, or method parameter.

```
@Service
public class NotificationService {
    @Value("${app.notification.sender-email}")
    private String senderEmail;

    @Value("${app.notification.max-retries:3}")  // default value = 3
    private int maxRetries;

    @Value("${app.feature.sms-enabled:false}")
    private boolean smsEnabled;
}
```

**Key features:**

- Supports default values with `:` syntax
- Supports SpEL (Spring Expression Language): `@Value("#{2 * 10}")`
- Can inject system properties: `@Value("${java.home}")`
- Works with constructor injection too

**When to use `@Value` vs `@ConfigurationProperties`:**

- **`@Value`** — for 1-2 isolated properties (a feature flag, a single URL)
- **`@ConfigurationProperties`** — for a group of related properties (database config, API client settings)

**Real-world scenario:** I use `@Value` for simple feature toggles:

```
@Value("${app.feature.new-checkout:false}")
private boolean newCheckoutEnabled;
```

**Common mistakes:**

- Missing the `${}` syntax — writing `@Value("app.name")` instead of `@Value("${app.name}")` injects the literal string
- Not providing a default value and the property is missing — app fails to start with `IllegalArgumentException`
- Using `@Value` in a class not managed by Spring (missing `@Component`) — injection won't work

`@Value` is fine for quick, simple property injection, but for anything structured, graduate to `@ConfigurationProperties`.

---

# Q24. What is `@ConditionalOnClass`?

`@ConditionalOnClass` is a Spring Boot condition annotation that tells Spring to only load a bean or configuration class if a specified class exists on the classpath.

```
@Configuration
@ConditionalOnClass(RedisTemplate.class)
public class RedisAutoConfiguration {
    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory factory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);
        return template;
    }
}
```

This configuration only activates if `RedisTemplate.class` is on the classpath — meaning the Redis dependency is present.

**Why it matters:** This is the backbone of Spring Boot's auto-configuration. It allows Spring Boot to ship hundreds of auto-config classes but only activate the ones relevant to your project.

**Real-world scenario:** When I built a custom Spring Boot starter for our internal caching library, I used `@ConditionalOnClass` to provide different implementations:

```
@Bean
@ConditionalOnClass(name = "com.hazelcast.core.HazelcastInstance")
public CacheManager hazelcastCacheManager() { ... }

@Bean
@ConditionalOnClass(name = "redis.clients.jedis.Jedis")
public CacheManager redisCacheManager() { ... }
```

The starter auto-detects which cache provider is available and configures accordingly.

**Common mistake:** Using `@ConditionalOnClass` with a class that's always on the classpath (like `String.class`) — the condition is useless. Also, note the `name` attribute (string-based) is safer when the class might not be present, avoiding `ClassNotFoundException` at annotation processing time.

This is a fundamental building block for writing library auto-configurations and understanding how Spring Boot decides what to configure.

---

# Q25. What is `@ConditionalOnProperty`?

`@ConditionalOnProperty` is a condition annotation that loads a bean or configuration only when a specific property is set (or has a specific value) in your configuration.

```
@Configuration
@ConditionalOnProperty(name = "app.cache.enabled", havingValue = "true", matchIfMissing = false)
public class CacheConfig {
    @Bean
    public CacheManager cacheManager() {
        return new RedisCacheManager(...);
    }
}
```

This config only loads if `app.cache.enabled=true` is in your properties.

**Key attributes:**

- `name` / `prefix` — the property key to check
- `havingValue` — the expected value (defaults to anything non-false)
- `matchIfMissing` — what happens if the property is absent (default: `false` — don't load)

**Real-world scenario:** We use this extensively for feature flags and conditional integrations:

```
// Only enable Kafka consumer if the feature is toggled on
@ConditionalOnProperty(name = "app.kafka.consumer.enabled", havingValue = "true")
@Component
public class OrderEventConsumer { ... }

// Enable scheduled jobs only in the scheduler instance
@ConditionalOnProperty(name = "app.scheduler.enabled", havingValue = "true", matchIfMissing = false)
@Configuration
@EnableScheduling
public class SchedulerConfig { ... }
```

This lets us deploy the same codebase but toggle behaviors per environment — one instance processes HTTP, another runs scheduled jobs.

**Common mistake:** Setting `matchIfMissing = true` when you actually want the feature OFF by default. Also, not understanding that `havingValue` is case-sensitive — `"True"` won't match if the property is `"true"`.

`@ConditionalOnProperty` is one of the most powerful tools for building flexible, configurable applications — I use it in almost every project.

# SPRING BOOT INTERVIEW QUESTIONS - SENIOR LEVEL ANSWERS (5-8 YOE)

## BASIC QUESTIONS (1-20)

---

### Question 1: What is Spring Boot?

**1. Definition:** Spring Boot is an opinionated framework built on top of Spring Framework that simplifies the bootstrapping and development of production-ready Spring applications through auto-configuration, embedded servers, and starter dependencies.

**2. Why it matters:** In production, development velocity and consistency are critical. Spring Boot eliminates boilerplate configuration, enforces best practices, and provides production-ready features (metrics, health checks) out of the box—reducing time-to-market significantly.

**3. Internal working:**

- Spring Boot uses `SpringApplication.run()` which initializes `ApplicationContext`
- Auto-configuration classes are loaded via `spring.factories` (or `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` in Spring Boot 3.x)
- `@Conditional` annotations determine which beans to create based on classpath, properties, and existing beans
- Embedded server (Tomcat/Jetty/Undertow) is initialized as part of the application context

**4. Real-world example:** In a payment gateway microservice, Spring Boot allowed our team to spin up a new service in hours instead of days. Auto-configured DataSource, connection pooling (HikariCP), and actuator endpoints meant we focused on business logic while getting production-ready infrastructure for free.

**5. Pitfalls:**

- Blindly accepting auto-configuration without understanding what's being configured
- Not excluding unnecessary auto-configurations, leading to bloated applications
- Ignoring actuator security in production environments

**6. Follow-up questions:**

- How would you debug what auto-configurations are being applied?
- How do you customize embedded server thread pools for high-traffic scenarios?
- What's the difference between Spring Boot 2.x and 3.x auto-configuration loading?

**7. Senior summary:** "Spring Boot is an opinionated layer over Spring that provides auto-configuration, embedded servers, and production-ready features. It dramatically reduces boilerplate while still allowing full customization when needed. I leverage it to build microservices quickly while maintaining production-grade standards."

---

# Question 2: Why Spring Boot over Spring MVC?

**1. Definition:** Spring MVC is just the web layer of Spring Framework requiring extensive XML/Java configuration. Spring Boot is a complete solution that auto-configures Spring MVC along with all supporting infrastructure.

**2. Why it matters:** In enterprise environments, configuration consistency across teams and environments is crucial. Spring Boot's convention-over-configuration approach ensures standardized setups while reducing human error in configuration.

**3. Internal working:** Spring MVC alone requires:

- Manual DispatcherServlet configuration
- ViewResolver setup
- HandlerMapping configuration
- External server deployment (WAR to Tomcat)

Spring Boot auto-configures all of this via `WebMvcAutoConfiguration`, plus adds:

- Embedded server
- Default error handling
- Static resource handling
- JSON serialization (Jackson)

**4. Real-world example:** Migrating a legacy Spring MVC banking application (500+ lines of XML config) to Spring Boot reduced configuration to ~50 lines of properties. Deployment shifted from WAR files on external Tomcat to self-contained JARs, simplifying our CI/CD pipeline.

**5. Pitfalls:**

- Assuming Spring Boot replaces Spring—it's still Spring underneath
- Over-customizing and fighting against Boot's conventions
- Not understanding that you can still use XML config if needed

**6. Follow-up questions:**

- When would you choose Spring MVC without Boot?
- How do you override Spring Boot's default MVC configurations?

**7. Senior summary:** "Spring Boot eliminates the configuration overhead of Spring MVC by auto-configuring the web layer, providing embedded servers, and establishing sensible defaults. It's not a replacement but an enhancement that lets teams focus on business logic rather than plumbing."

---

# Question 3: What problems does Spring Boot solve?

**1. Definition:** Spring Boot addresses: (1) Configuration complexity, (2) Dependency management hell, (3) Deployment complexity, (4) Production readiness, and (5) Development-to-production parity.

**2. Why it matters:** Enterprise applications often fail not due to business logic but due to misconfiguration, dependency conflicts, or environment differences. Spring Boot systematically eliminates these failure modes.

**3. Internal working:**

- **Starter POMs**: Curated, tested dependency sets (no version conflicts)
- **Auto-configuration**: Conditional bean creation based on classpath scanning
- **Embedded servers**: Same runtime in dev and prod
- **Externalized config**: Property sources hierarchy for environment-specific overrides
- **Actuator**: Production monitoring built-in

**4. Real-world example:** In an e-commerce platform, we had "works on my machine" issues constantly. After Spring Boot adoption:

- Starter dependencies eliminated JAR hell
- Embedded Tomcat meant identical behavior across environments
- Actuator gave us health checks for Kubernetes probes

**5. Pitfalls:**

- Treating starters as magic—not understanding transitive dependencies
- Not leveraging externalized configuration properly (hardcoding values)
- Ignoring the actuator for production monitoring

**6. Follow-up questions:**

- How do you handle dependency conflicts when a starter brings in unwanted versions?
- How does Spring Boot's property source hierarchy work?

**7. Senior summary:** "Spring Boot solves configuration complexity through auto-configuration, dependency management through starters, deployment complexity through embedded servers, and production readiness through actuator. It essentially codifies best practices so teams don't reinvent the wheel."

---

# Question 4: What is auto-configuration?

**1. Definition:** Auto-configuration is Spring Boot's mechanism to automatically configure beans based on classpath contents, existing beans, and properties—eliminating explicit configuration for common scenarios.

**2. Why it matters:** It enables the "just add dependency and it works" experience while still allowing full override capability. This dramatically accelerates development without sacrificing flexibility.

**3. Internal working:**

1. `@EnableAutoConfiguration` triggers scanning of `META-INF/spring.factories` (or `.imports` file in 3.x)
2. Each auto-configuration class uses `@Conditional*` annotations:
    - `@ConditionalOnClass`: Only if class exists on classpath
    - `@ConditionalOnMissingBean`: Only if you haven't defined this bean
    - `@ConditionalOnProperty`: Only if property is set
3. Order controlled via `@AutoConfigureBefore`/`@AutoConfigureAfter`
4. Debug with `--debug` flag or `debug=true` property

**4. Real-world example:** Adding `spring-boot-starter-data-redis` to a session caching service:

- `RedisAutoConfiguration` detects Jedis/Lettuce on classpath
- Creates `RedisConnectionFactory` with sensible defaults
- `RedisTemplate` bean auto-created
- Override by simply defining your own `RedisConnectionFactory` bean

**5. Pitfalls:**

- Not knowing which auto-configurations are active (use actuator `/conditions` endpoint)
- Creating beans that conflict with auto-configured ones without proper `@Primary`
- Assuming auto-configuration means zero configuration—production often needs tuning

**6. Follow-up questions:**

- How would you create your own auto-configuration for a custom library?
- Explain the role of `@ConditionalOnMissingBean` in auto-configuration classes.

**7. Senior summary:** "Auto-configuration uses conditional bean registration to automatically configure components based on classpath, properties, and existing beans. It follows a 'back-off' pattern—if you define a bean, auto-configuration doesn't override it. I always check the conditions report to understand what's being auto-configured."

---

# Question 5: What is @SpringBootApplication?

**1. Definition:** `@SpringBootApplication` is a meta-annotation combining `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`—serving as the entry point for Spring Boot applications.

**2. Why it matters:** It establishes the component scanning base package and triggers auto-configuration. Misplacing this annotation is a common source of "bean not found" errors.

**3. Internal working:**

```
@SpringBootConfiguration  // Same as @Configuration
@EnableAutoConfiguration  // Triggers auto-config loading
@ComponentScan           // Scans current package and below
public @interface SpringBootApplication
```

Key behaviors:

- Component scanning starts from the annotated class's package
- `exclude` and `excludeName` attributes allow disabling specific auto-configurations
- `scanBasePackages` overrides default scanning behavior

**4. Real-world example:** In a multi-module Maven project, placing `@SpringBootApplication` in `com.company.app` while services were in `com.company.services` caused beans to not be found. Solution: Either move the main class to `com.company` or explicitly set `scanBasePackages`.

**5. Pitfalls:**

- Placing main class in a sub-package, missing sibling package components
- Not understanding that it's just a convenience annotation
- Using both `@SpringBootApplication` and explicit `@ComponentScan` incorrectly

**6. Follow-up questions:**

- How would you exclude a specific auto-configuration?
- What happens if you have two classes with `@SpringBootApplication`?

**7. Senior summary:** "@SpringBootApplication combines @Configuration, @EnableAutoConfiguration, and @ComponentScan. It should be placed in a root package so component scanning finds all beans. I use its exclude attribute to disable unwanted auto-configurations like DataSourceAutoConfiguration when not needed."

---

# Question 6: Difference between Spring and Spring Boot?

**1. Definition:** Spring is the core dependency injection and enterprise integration framework. Spring Boot is an opinionated project that simplifies Spring application setup with auto-configuration, starters, and embedded servers.

**2. Why it matters:** Understanding this distinction is crucial because Spring Boot doesn't replace Spring—it enhances it. All Spring concepts (DI, AOP, transactions) work identically in Boot.

**3. Internal working:**

| Aspect | Spring | Spring Boot |
|---|---|---|
| Configuration | Manual (XML/Java) | Auto + Manual |
| Dependencies | Individual selection | Curated starters |
| Server | External container | Embedded |
| Production features | Manual setup | Actuator built-in |
| Build | WAR typically | JAR typically |

**4. Real-world example:** A legacy Spring application required 15+ config files, explicit dependency versions, and external Tomcat. The same functionality in Spring Boot: one `application.yml`, one starter dependency, and `mvn spring-boot:run`.

**5. Pitfalls:**

- Thinking Spring Boot is a different framework (it's still Spring)
- Believing Spring Boot can't do something because it's "opinionated"
- Not leveraging Spring expertise when using Boot

**6. Follow-up questions:**

- Can you use XML configuration in Spring Boot?
- How do you migrate a Spring MVC app to Spring Boot?

**7. Senior summary:** "Spring is the foundational DI and enterprise framework. Spring Boot is a productivity layer that provides auto-configuration, embedded servers, and production features. The core programming model is identical—Boot just eliminates boilerplate. Any Spring knowledge directly applies to Boot."

---

# Question 7: What is embedded server?

**1. Definition:** An embedded server is a servlet container (Tomcat, Jetty, Undertow) packaged within the application JAR, starting as part of the application rather than requiring external deployment.

**2. Why it matters:** Embedded servers enable the "build once, run anywhere" promise. The same JAR runs identically on a developer laptop and in Kubernetes, eliminating environment-specific deployment issues.

**3. Internal working:**

1. Spring Boot's `spring-boot-starter-web` includes `spring-boot-starter-tomcat` by default
2. `ServletWebServerApplicationContext` detects embedded server on classpath
3. `TomcatServletWebServerFactory` creates and configures Tomcat instance
4. Server starts as part of `ApplicationContext.refresh()`
5. Shutdown hooks ensure graceful termination

**4. Real-world example:** Our microservices platform runs 200+ Spring Boot services in Kubernetes. Each service is a self-contained JAR with embedded Tomcat. Docker images are simply `FROM openjdk` + `COPY app.jar`. No Tomcat installation, no WAR deployment, no server configuration drift.

**5. Pitfalls:**

- Not tuning thread pools for production load (defaults may be insufficient)
- Ignoring graceful shutdown configuration (`server.shutdown=graceful`)
- Not configuring appropriate connection timeouts

**6. Follow-up questions:**

- How do you switch from Tomcat to Undertow?
- How do you configure thread pool sizes for high-concurrency scenarios?

**7. Senior summary:** "Embedded servers package the servlet container within the application JAR, eliminating external deployment. This enables consistent dev-to-prod parity and simplifies containerization. I always tune thread pools, connection limits, and enable graceful shutdown for production deployments."

---

# Question 8: Which servers Spring Boot supports?

**1. Definition:** Spring Boot supports Tomcat (default), Jetty, and Undertow as embedded servlet containers, plus Netty for reactive applications.

**2. Why it matters:** Different servers have different performance characteristics. Undertow excels at high concurrency, Jetty at low memory footprint, Tomcat at ecosystem compatibility.

**3. Internal working:**

| Server | Best For | Key Characteristic |
|---|---|---|
| Tomcat | General purpose, ecosystem | Mature, well-documented |
| Jetty | Low memory, HTTP/2 | Lightweight, async |
| Undertow | High concurrency | Non-blocking, fast |
| Netty | Reactive/WebFlux | Event-loop, non-blocking |

Switching servers:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-undertow</artifactId>
</dependency>
```

**4. Real-world example:** A real-time bidding platform needed to handle 50K concurrent connections. Switched from Tomcat to Undertow, reducing memory per connection and improving p99 latency by 30%.

**5. Pitfalls:**

- Switching servers without benchmarking for your specific workload
- Assuming Undertow/Jetty are always faster (depends on use case)
- Not adjusting configuration when switching (each has different tuning parameters)

**6. Follow-up questions:**

- When would you choose Netty over servlet-based servers?
- How do thread models differ between Tomcat and Undertow?

**7. Senior summary:** "Spring Boot supports Tomcat, Jetty, Undertow for servlet workloads, and Netty for reactive. Server choice depends on workload: Tomcat for compatibility, Undertow for high concurrency, Netty for reactive. I benchmark with realistic load before committing to a server choice."

---

# Question 9: Default port of Spring Boot?

**1. Definition:** Spring Boot's default server port is 8080 for web applications and 8443 for HTTPS when SSL is configured.

**2. Why it matters:** Understanding default ports is essential for container orchestration, firewall rules, and load balancer configuration. Port conflicts are a common deployment issue.

**3. Internal working:**

- Port configured via `server.port` property
- `0` means random available port (useful for testing)
- `-1` disables the web server entirely
- At startup, Spring Boot binds to the port during `WebServerInitializedEvent`

**4. Real-world example:** In Kubernetes, we override `server.port=8080` to maintain consistency but use different service ports (80) at the Service level. Health probes hit `8080/actuator/health`. For local development with multiple services, we use random ports (`server.port=0`) and service discovery.

**5. Pitfalls:**

- Hardcoding port instead of externalizing it
- Not handling port-already-in-use errors gracefully
- Forgetting management port separation (`management.server.port`)

**6. Follow-up questions:**

- How do you run multiple Spring Boot apps locally without port conflicts?
- What's the purpose of `management.server.port`?

**7. Senior summary:** "Default is 8080, changed via `server.port` property. In production, I externalize this through environment variables. For microservices, I often use port 0 with service discovery or separate management port for actuator endpoints for security isolation."

---

# Question 10: How to change server port?

**1. Definition:** Server port is changed via `server.port` property in `application.properties/yml`, command-line argument, or environment variable.

**2. Why it matters:** Port configuration strategy affects deployment flexibility. Proper externalization enables the same artifact to run in different environments without rebuilding.

**3. Internal working:** Property source hierarchy (highest to lowest priority):

1. Command line: `--server.port=9090`
2. OS environment: `SERVER_PORT=9090`
3. `application-{profile}.properties`
4. `application.properties`
5. Default (8080)

```
# application.yml
server:
  port: ${PORT:8080}  # Environment variable with fallback
```

**4. Real-world example:** Our deployment strategy:

- Local dev: `application-local.yml` with port 8080
- Docker: `SERVER_PORT` env var from orchestrator
- Kubernetes: ConfigMap sets port, but container always exposes 8080 internally
- Cloud Run/Heroku: `PORT` env var set by platform

**5. Pitfalls:**

- Hardcoding ports in properties files committed to source control
- Not understanding property source precedence
- Using different ports in different environments unnecessarily

**6. Follow-up questions:**

- How does Spring Boot relax property binding work?
- What's the difference between `server.port` and `management.server.port`?

**7. Senior summary:** "I use environment variables with fallbacks: `server.port=${PORT:8080}`. This allows platform-specific ports (Cloud Run, Kubernetes) while maintaining sensible defaults. Never hardcode ports in committed configuration—externalize for deployment flexibility."

---

# Question 11: What is starter dependency?

**1. Definition:** Starter dependencies are curated, pre-tested sets of transitive dependencies that provide all required libraries for a specific functionality with compatible versions.

**2. Why it matters:** Dependency management is a major source of production issues. Starters eliminate version conflicts and ensure all required libraries are included—no more missing dependencies at runtime.

**3. Internal working:** A starter typically contains:

- No code (usually)
- `pom.xml` with dependencies
- Auto-configuration trigger

Example `spring-boot-starter-data-jpa` includes:

- Hibernate (ORM)
- Spring Data JPA
- HikariCP (connection pool)
- Spring JDBC
- Transaction management

The BOM (`spring-boot-dependencies`) manages all versions centrally.

**4. Real-world example:** Adding database support to a service:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

This single line brings in ~15 compatible dependencies. Without starters, you'd manually manage versions and likely face `NoSuchMethodError` due to version mismatches.

**5. Pitfalls:**

- Not understanding transitive dependencies (use `mvn dependency:tree`)
- Overriding starter versions without testing compatibility
- Including multiple starters with conflicting dependencies

**6. Follow-up questions:**

- How do you override a transitive dependency version in a starter?
- How would you create a custom starter for your organization?

**7. Senior summary:** "Starters are curated dependency sets that provide everything needed for specific functionality with tested-compatible versions. They eliminate JAR hell and ensure consistency. I always check the dependency tree to understand what's being pulled in and exclude unnecessary transitive dependencies."

---

# Question 12: Why spring-boot-starter-parent?

**1. Definition:** `spring-boot-starter-parent` is a parent POM that provides dependency management, plugin configuration, and resource filtering defaults for Spring Boot projects.

**2. Why it matters:** Consistent build configuration across projects reduces maintenance burden. The parent POM ensures all Spring Boot projects in an organization build identically.

**3. Internal working:** Parent provides:

- `spring-boot-dependencies` BOM (version management)
- Plugin configurations (compiler, surefire, resources)
- Property defaults (`java.version`, encoding)
- Resource filtering for `application.properties`
- Executable JAR packaging

Alternative without parent:

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-dependencies</artifactId>
            <version>3.2.0</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

**4. Real-world example:** Enterprise projects often have a corporate parent POM. We import Spring Boot BOM instead:

```
<parent>
    <groupId>com.company</groupId>
    <artifactId>corporate-parent</artifactId>
</parent>
<!-- Import Spring Boot BOM -->
```

This maintains corporate governance while getting Spring Boot dependency management.

**5. Pitfalls:**

- Overriding parent properties without understanding impact
- Not realizing you can use BOM import instead of parent
- Forgetting to configure executable JAR manually when not using parent

**6. Follow-up questions:**

- How do you use Spring Boot without the parent POM?
- What does the `repackage` goal do?

**7. Senior summary:** "The parent POM provides version management, build plugins, and resource filtering. In enterprise settings where a corporate parent is required, I import the `spring-boot-dependencies` BOM instead. This gives version management without mandating the parent."

---

# Question 13: application.properties vs application.yml

**1. Definition:** Both are configuration files; `.properties` uses key=value format while `.yml` uses hierarchical YAML syntax. They're functionally equivalent—Spring Boot reads both.

**2. Why it matters:** Configuration readability and maintainability affect operations. YAML's hierarchical structure reduces repetition for nested properties but introduces whitespace sensitivity issues.

**3. Internal working:** Processing order:

1. `application.properties` outside JAR
2. `application.yml` outside JAR
3. `application.properties` inside JAR
4. `application.yml` inside JAR

Both support:

- Profile-specific files (`application-prod.yml`)
- Placeholder resolution (`${VARIABLE:default}`)
- Spring profiles in YAML (`---` separator)

**4. Real-world example:**

```
# application.yml - Complex nested config
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 30000
```

vs

```
# application.properties - Same config
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=30000
```

YAML is cleaner for nested config; properties better for simple key-values.

**5. Pitfalls:**

- YAML whitespace/indentation errors (invisible bugs)
- Mixing both formats causing confusion
- Not knowing both can coexist (properties takes precedence for same key)

**6. Follow-up questions:**

- How do you define multiple profiles in a single YAML file?
- What's the loading order when both files exist?

**7. Senior summary:** "Functionally equivalent—I prefer YAML for complex nested configurations due to readability, but properties for simple settings. YAML's whitespace sensitivity can cause subtle bugs. In teams, I enforce one format for consistency."

---

# Question 14: How to run Spring Boot application?

**1. Definition:** Spring Boot applications run via: IDE run, Maven/Gradle plugin, executable JAR, or as a deployed WAR (legacy).

**2. Why it matters:** Different execution methods suit different stages (dev, CI, production). Understanding each ensures efficient development workflows and proper production deployment.

**3. Internal working:**

| Method | Use Case | Command |
|---|---|---|
| IDE | Development | Run/Debug main class |
| Maven | Dev/CI | `mvn spring-boot:run` |
| Gradle | Dev/CI | `./gradlew bootRun` |
| JAR | Production | `java -jar app.jar` |
| WAR | Legacy servers | Deploy to external Tomcat |

The `spring-boot-maven-plugin` creates an uber-JAR with:

- Application classes in `BOOT-INF/classes`
- Dependencies in `BOOT-INF/lib`
- Spring Boot loader in root

**4. Real-world example:** Our CI/CD pipeline:

1. Build: `mvn clean package -DskipTests`
2. Test: `mvn verify`
3. Docker: `COPY target/*.jar app.jar`
4. Run: `java -XX:+UseContainerSupport -jar app.jar`

JVM flags for containers: `-XX:MaxRAMPercentage=75.0`

**5. Pitfalls:**

- Using `mvn spring-boot:run` in production (meant for dev)
- Not setting proper JVM memory flags for containers
- Missing the executable permission on JAR in some environments

**6. Follow-up questions:**

- How do you pass JVM arguments when running the JAR?
- What's the difference between the executable JAR and regular JAR?

**7. Senior summary:** "Development uses IDE or Maven plugin for hot reload. Production uses executable JAR with proper JVM flags. For containers, I ensure JVM respects container memory limits with `-XX:+UseContainerSupport`. The JAR is self-contained—no external dependencies needed."

---

# Question 15: What is CommandLineRunner?

**1. Definition:** `CommandLineRunner` is a functional interface that executes code after the application context is initialized, receiving command-line arguments as a String array.

**2. Why it matters:** Initialization logic (cache warming, data loading, connectivity checks) must run at the right lifecycle point. CommandLineRunner ensures dependencies are injected before execution.

**3. Internal working:**

```
@Component
public class StartupRunner implements CommandLineRunner {
    @Override
    public void run(String... args) {
        // Executed after ApplicationContext is ready
        // args contains command-line arguments
    }
}
```

Execution order:

1. ApplicationContext created
2. All beans initialized
3. `CommandLineRunner` and `ApplicationRunner` beans executed
4. Application fully started

Multiple runners ordered via `@Order` annotation.

**4. Real-world example:** In a pricing service:

```
@Component
@Order(1)
public class CacheWarmer implements CommandLineRunner {
    @Autowired
    private PricingCache cache;

    @Override
    public void run(String... args) {
        log.info("Warming pricing cache...");
        cache.loadAllPrices();  // Prevent cold-start latency
    }
}
```

**5. Pitfalls:**

- Putting long-running tasks that block application startup
- Not handling exceptions (crashes the app if uncaught)
- Confusing with `@PostConstruct` (different lifecycle point)

**6. Follow-up questions:**

- What's the difference between `CommandLineRunner` and `@PostConstruct`?
- How do you order multiple `CommandLineRunner` beans?

**7. Senior summary:** "CommandLineRunner executes after full context initialization with access to command-line args. I use it for startup tasks like cache warming or connectivity verification. Unlike @PostConstruct, it runs after ALL beans are ready, preventing dependency issues."

---

# Question 16: What is ApplicationRunner?

**1. Definition:** `ApplicationRunner` is similar to `CommandLineRunner` but receives `ApplicationArguments` instead of a raw String array, providing parsed access to option arguments.

**2. Why it matters:** When command-line arguments follow conventions (`--key=value`), `ApplicationRunner` provides cleaner access than manual parsing.

**3. Internal working:**

```
@Component
public class AppRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        // --server.port=8080 --debug
        args.getOptionNames();           // [server.port, debug]
        args.getOptionValues("debug");   // []
        args.containsOption("debug");    // true
        args.getNonOptionArgs();         // remaining args
    }
}
```

**4. Real-world example:** A data migration tool:

```
@Component
public class MigrationRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        if (args.containsOption("migrate")) {
            String version = args.getOptionValues("version").get(0);
            migrationService.migrateTo(version);
        }
    }
}
```

Run: `java -jar app.jar --migrate --version=2.0`

**5. Pitfalls:**

- Using when simple string args suffice (over-engineering)
- Not understanding it runs at the same lifecycle point as CommandLineRunner
- Forgetting that non-option args are separate from option args

**6. Follow-up questions:**

- When would you choose ApplicationRunner over CommandLineRunner?
- Can you have both in the same application?

**7. Senior summary:** "ApplicationRunner provides parsed command-line arguments with option/non-option separation. I use it when the app accepts `--key=value` style arguments. For simple args, CommandLineRunner is sufficient. Both run at the same lifecycle point."

---

# Question 17: What is Actuator?

**1. Definition:** Spring Boot Actuator provides production-ready features: health checks, metrics, environment info, HTTP tracing, and management endpoints for monitoring and managing applications.

**2. Why it matters:** Production observability is non-negotiable. Actuator provides standardized endpoints that integrate with monitoring tools (Prometheus, Grafana), orchestrators (Kubernetes), and APM solutions.

**3. Internal working:** Key endpoints: | Endpoint | Purpose | |---------|--------| | `/health` | Liveness/readiness probes | | `/metrics` | Micrometer metrics | | `/info` | Build info, git commit | | `/env` | Environment properties | | `/loggers` | Dynamic log level changes | | `/threaddump` | Thread analysis |

Secured by default in Spring Boot 2.x+. Exposing:

```
management:
  endpoints:
    web:
      exposure:
        include: health,metrics,info
  endpoint:
    health:
      show-details: when_authorized
```

**4. Real-world example:** Kubernetes integration:

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
```

Custom health indicator for database:

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        return database.isAvailable()
            ? Health.up().build()
            : Health.down().withDetail("error", "DB unreachable").build();
    }
}
```

**5. Pitfalls:**

- Exposing sensitive endpoints (env, heapdump) without authentication
- Not separating management port from application port
- Heavy health checks affecting application performance

**6. Follow-up questions:**

- How do you secure actuator endpoints?
- How would you add custom metrics to Micrometer?

**7. Senior summary:** "Actuator provides production observability: health probes, metrics, dynamic log levels. I always configure it with security, separate management port, and integrate with Prometheus/Grafana. Health endpoints are critical for Kubernetes liveness/readiness probes."

---

# Question 18: How to enable Actuator endpoints?

**1. Definition:** Add `spring-boot-starter-actuator` dependency and configure exposure via `management.endpoints.web.exposure.include` property.

**2. Why it matters:** Endpoints are disabled by default for security. Explicitly enabling only necessary endpoints follows the principle of least privilege.

**3. Internal working:**

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
        exclude: env,heapdump
      base-path: /actuator  # default
  server:
    port: 8081  # Separate management port
  endpoint:
    health:
      show-details: when_authorized
      probes:
        enabled: true  # Kubernetes probes
```

Security configuration:

```
@Bean
public SecurityFilterChain actuatorSecurity(HttpSecurity http) {
    http.securityMatcher("/actuator/**")
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/actuator/health/**").permitAll()
            .requestMatchers("/actuator/**").hasRole("ADMIN"));
    return http.build();
}
```

**4. Real-world example:** Production configuration for a payment service:

- Health: Public (for load balancer)
- Metrics: Internal network only (Prometheus scraping)
- Env/heapdump: Admin role only, separate management port
- Loggers: Enabled for dynamic debug level changes

**5. Pitfalls:**

- Using `include: *` in production (security risk)
- Not separating management port in sensitive environments
- Forgetting that `/shutdown` endpoint can kill your app

**6. Follow-up questions:**

- How do you secure the `/env` endpoint which shows properties?
- How would you enable the shutdown endpoint safely?

**7. Senior summary:** "Add actuator starter, explicitly enable needed endpoints via `management.endpoints.web.exposure.include`. Use separate management port in production, secure sensitive endpoints. Only `/health` should be public; everything else requires authentication."

---

# Question 19: Difference between @Component and @Bean?

**1. Definition:** `@Component` is a class-level annotation for auto-detection via component scanning. `@Bean` is a method-level annotation in `@Configuration` classes for explicit bean definition.

**2. Why it matters:** Choosing the right approach affects code organization, testability, and third-party library integration. Understanding both is essential for proper Spring development.

**3. Internal working:**

| Aspect | @Component | @Bean |
|---|---|---|
| Level | Class | Method |
| Detection | Component scanning | Explicit declaration |
| Third-party | Cannot annotate | Can wrap any class |
| Customization | Limited | Full control |
| Conditional | @Conditional on class | @Conditional on method |

```
// @Component - class you own
@Component
public class EmailService { }

// @Bean - third-party or complex instantiation
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
            .setConnectTimeout(Duration.ofSeconds(5))
            .build();
    }
}
```

**4. Real-world example:** A payment gateway integration:

```
// Can't add @Component to Stripe SDK classes
@Configuration
public class PaymentConfig {
    @Bean
    public StripeClient stripeClient(@Value("${stripe.api-key}") String apiKey) {
        return new StripeClient(apiKey);  // Third-party class
    }
}
```

**5. Pitfalls:**

- Using @Component for everything, losing explicit configuration benefits
- Forgetting that @Bean methods are proxied (don't call directly from same class)
- Not understanding @Bean method interception requires @Configuration (not @Component)

**6. Follow-up questions:**

- What's the difference between @Configuration and @Component with @Bean methods?
- How does CGLIB proxying affect @Bean method calls?

**7. Senior summary:** "@Component for your own classes discovered via scanning, @Bean for third-party classes or complex instantiation logic. @Bean in @Configuration classes enables proxied method calls for singleton semantics. I use @Bean when I need explicit control over bean creation."

---

# Question 20: What is Dependency Injection?

**1. Definition:** Dependency Injection (DI) is a design pattern where objects receive their dependencies from external sources rather than creating them, enabling loose coupling and testability.

**2. Why it matters:** DI is foundational to testable, maintainable code. It enables mocking in tests, swapping implementations without code changes, and following SOLID principles.

**3. Internal working:** Spring DI mechanisms:

1. **Constructor injection** (preferred): Dependencies as constructor params
2. **Setter injection**: Via setter methods
3. **Field injection**: Via `@Autowired` on fields (discouraged)

Spring's IoC container:

1. Creates beans based on configuration
2. Resolves dependencies via type matching
3. Handles circular dependencies (for setter/field injection)
4. Manages bean lifecycle

```
// Constructor injection - preferred
@Service
public class OrderService {
    private final PaymentService paymentService;
    private final InventoryService inventoryService;

    public OrderService(PaymentService payment, InventoryService inventory) {
        this.paymentService = payment;
        this.inventoryService = inventory;
    }
}
```

**4. Real-world example:** Testing with DI:

```
@Test
void shouldProcessOrder() {
    // Easy mocking because of DI
    PaymentService mockPayment = mock(PaymentService.class);
    InventoryService mockInventory = mock(InventoryService.class);

    OrderService service = new OrderService(mockPayment, mockInventory);
    // Test without real payment processing
}
```

**5. Pitfalls:**

- Field injection making classes untestable without reflection
- Circular dependencies indicating design problems
- Over-injection (too many dependencies = class doing too much)

**6. Follow-up questions:**

- Why is constructor injection preferred over field injection?
- How does Spring handle circular dependencies?

**7. Senior summary:** ""DI inverts control—objects receive dependencies rather than creating them. This enables testability, loose coupling, and configuration flexibility. I always use constructor injection for mandatory dependencies as it makes dependencies explicit and enables easy testing without Spring context."

---

# INTERMEDIATE QUESTIONS (21-40) - Most rejections happen here

---

## Question 21: What is auto-configuration internally?

**1. Definition:** Internally, auto-configuration is a mechanism where Spring Boot loads configuration classes from `META-INF/spring.factories` (or `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` in 3.x) and conditionally creates beans based on classpath, properties, and existing beans.

**2. Why it matters:** Understanding the internals helps debug "why is this bean created/not created" issues—a common production debugging scenario.

**3. Internal working:**

1. `@EnableAutoConfiguration` imports `AutoConfigurationImportSelector`
2. Selector reads auto-configuration class names from:
   - Spring Boot 2.x: `META-INF/spring.factories`
   - Spring Boot 3.x: `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`
3. Classes filtered by `@Conditional*` annotations
4. Remaining classes processed by `ConfigurationClassParser`
5. `@AutoConfigureOrder`, `@AutoConfigureBefore`, `@AutoConfigureAfter` control ordering

Debug with:

```
java -jar app.jar --debug
# Or
logging.level.org.springframework.boot.autoconfigure=DEBUG
```

**4. Real-world example:** Debugging why HikariCP wasn't being configured:

```
# Check conditions report
/actuator/conditions

# Found: DataSourceAutoConfiguration
#   - @ConditionalOnClass found HikariDataSource
#   - @ConditionalOnMissingBean - MATCHED (no existing DataSource)
#   BUT spring.datasource.url was missing!
```

Fix: Added `spring.datasource.url` property.

**5. Pitfalls:**

- Not checking the conditions report when beans aren't created
- Creating a bean that shadows auto-configured one unintentionally
- Not understanding the order dependencies between auto-configurations

**6. Follow-up questions:**

- How would you create your own auto-configuration?
- What's the purpose of @AutoConfigureAfter?

**7. Senior summary:** "Auto-configuration loads classes from spring.factories/imports files, filters them through @Conditional annotations, and creates beans that don't conflict with user-defined ones. I use `--debug` flag and `/actuator/conditions` to diagnose configuration issues."

---

# Question 22: How does Spring Boot decide which beans to create?

**1. Definition:** Spring Boot uses `@Conditional*` annotations to evaluate conditions at configuration time, creating beans only when all conditions are met.

**2. Why it matters:** This is the core mechanism enabling "just add dependency and it works." Understanding conditions explains why a bean exists or doesn't exist.

**3. Internal working:** Condition evaluation hierarchy:

```
@ConditionalOnClass(DataSource.class)      // Class on classpath?
@ConditionalOnProperty(name="spring.datasource.url")  // Property set?
@ConditionalOnMissingBean(DataSource.class)  // User didn't define one?
public DataSource dataSource() { ... }
```

Key conditionals: | Annotation | Checks | |----------|--------| | `@ConditionalOnClass` | Class exists on classpath | | `@ConditionalOnMissingClass` | Class doesn't exist | | `@ConditionalOnBean` | Bean exists in context | | `@ConditionalOnMissingBean` | Bean doesn't exist | | `@ConditionalOnProperty` | Property has specific value | | `@ConditionalOnResource` | Resource exists | | `@ConditionalOnWebApplication` | Is a web app |

**4. Real-world example:** Custom conditional for feature flags:

```
@Retention(RetentionPolicy.RUNTIME)
@Conditional(OnFeatureCondition.class)
public @interface ConditionalOnFeature {
    String value();
}

public class OnFeatureCondition implements Condition {
    @Override
    public boolean matches(ConditionContext context, AnnotatedTypeMetadata metadata) {
        String feature = metadata.getAnnotationAttributes(
            ConditionalOnFeature.class.getName()).get("value").toString();
        return context.getEnvironment()
            .getProperty("features." + feature, Boolean.class, false);
    }
}
```

**5. Pitfalls:**

- Conditions evaluated at configuration time, not runtime
- Order of condition evaluation matters
- `@ConditionalOnMissingBean` checked after user configs processed

**6. Follow-up questions:**

- How would you create a custom condition?
- What's the difference between @Conditional and @Profile?

**7. Senior summary:** "Spring Boot evaluates @Conditional annotations during configuration processing. The 'back-off' pattern uses @ConditionalOnMissingBean to let user beans take precedence. Conditions are evaluated once at startup—not dynamically. Debug with conditions actuator endpoint."

---

# Question 23: What is spring.factories?

**1. Definition:** `spring.factories` is a properties file in `META-INF/` that maps interface types to implementation classes, enabling service discovery for auto-configuration, listeners, and other extension points.

**2. Why it matters:** It's the extension mechanism for Spring Boot. Custom starters, auto-configurations, and framework extensions all use this file.

**3. Internal working:** Location: `META-INF/spring.factories`

```
# Auto-configuration
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
  com.example.MyAutoConfiguration,\
  com.example.AnotherAutoConfiguration

# Application listeners
org.springframework.context.ApplicationListener=\
  com.example.MyListener

# Failure analyzers
org.springframework.boot.diagnostics.FailureAnalyzer=\
  com.example.MyFailureAnalyzer
```

**Spring Boot 3.x change:** Auto-configurations moved to: `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`

```
com.example.MyAutoConfiguration
com.example.AnotherAutoConfiguration
```

**4. Real-world example:** Creating a company-wide observability starter:

```
# META-INF/spring.factories
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
  com.company.observability.TracingAutoConfiguration,\
  com.company.observability.MetricsAutoConfiguration
```

Now any project adding this dependency gets tracing and metrics configured automatically.

**5. Pitfalls:**

- Format is strict—no spaces around `=` or after `\`
- Spring Boot 3.x migration requires moving to .imports file
- Multiple JARs can contribute to same key (they merge)

**6. Follow-up questions:**

- How do you migrate spring.factories to Spring Boot 3.x?
- What other extension points use spring.factories?

**7. Senior summary:** "spring.factories enables service discovery for auto-configuration and extensions. It maps interface types to implementations loaded at startup. In Spring Boot 3.x, auto-configurations moved to dedicated .imports files. I use this for creating reusable starters."

---

# Question 24: What is @EnableAutoConfiguration?

**1. Definition:** `@EnableAutoConfiguration` triggers Spring Boot's auto-configuration mechanism by importing `AutoConfigurationImportSelector`, which loads and applies auto-configuration classes.

**2. Why it matters:** It's the switch that enables the "magic" of Spring Boot. Understanding it helps when you need to disable or customize auto-configuration behavior.

**3. Internal working:**

```
@Target(ElementType.TYPE)
@Import(AutoConfigurationImportSelector.class)
public @interface EnableAutoConfiguration {
    String[] exclude() default {};
    String[] excludeName() default {};
}
```

Processing flow:

1. `@Import` triggers `AutoConfigurationImportSelector`
2. Selector calls `getCandidateConfigurations()`
3. Loads classes from spring.factories/imports
4. Filters through exclusions and conditions
5. Orders configurations
6. Returns list for Spring to process

**4. Real-world example:** Disabling specific auto-configurations:

```
@SpringBootApplication(exclude = {
    DataSourceAutoConfiguration.class,
    SecurityAutoConfiguration.class
})
public class Application { }
```

Or via properties:

```
spring:
  autoconfigure:
    exclude:
      - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

**5. Pitfalls:**

- Excluding too much, breaking dependent auto-configurations
- Not realizing @SpringBootApplication includes this
- Excluding by class name that might change between versions

**6. Follow-up questions:**

- How do you find out which auto-configurations are being applied?
- What's the difference between exclude and @ConditionalOnMissingBean?

**7. Senior summary:** "@EnableAutoConfiguration imports AutoConfigurationImportSelector which loads configurations from spring.factories. It's included in @SpringBootApplication. Use exclude attribute or spring.autoconfigure.exclude property to disable unwanted configurations."

---

# Question 25: What is @ConfigurationProperties?

**1. Definition:** `@ConfigurationProperties` binds external configuration (properties/YAML) to a type-safe Java object, enabling structured configuration with validation and IDE support.

**2. Why it matters:** Type-safe configuration prevents runtime errors from typos. It enables validation at startup, IDE auto-completion, and documentation generation.

**3. Internal working:**

```
@ConfigurationProperties(prefix = "payment.gateway")
@Validated
public class PaymentProperties {
    @NotBlank
    private String apiKey;

    @Min(1000) @Max(30000)
    private int timeout = 5000;

    private Retry retry = new Retry();

    public static class Retry {
        private int maxAttempts = 3;
        private Duration backoff = Duration.ofSeconds(1);
    }
    // getters and setters
}
```

```
payment:
  gateway:
    api-key: ${PAYMENT_API_KEY}
    timeout: 10000
    retry:
      max-attempts: 5
      backoff: 2s
```

Enable with:

- `@EnableConfigurationProperties(PaymentProperties.class)` or
- `@ConfigurationPropertiesScan`

**4. Real-world example:** A rate limiting configuration:

```
@ConfigurationProperties(prefix = "ratelimit")
public record RateLimitProperties(
    boolean enabled,
    int requestsPerMinute,
    Map<String, Integer> endpointLimits
) {}
```

Benefits: Immutable (record), validated, documented, IDE support.

**5. Pitfalls:**

- Forgetting to enable via @EnableConfigurationProperties
- Using @Value for complex nested config (hard to maintain)
- Not adding spring-boot-configuration-processor for IDE support

**6. Follow-up questions:**

- How does relaxed binding work?
- How do you generate configuration metadata for IDE support?

**7. Senior summary:** "@ConfigurationProperties provides type-safe, validated configuration binding with IDE support. I use it for any non-trivial configuration instead of @Value. Add spring-boot-configuration-processor for metadata generation. Records make properties immutable."

---

## Question 26: Difference between @Value and @ConfigurationProperties?

**1. Definition:** `@Value` injects individual properties into fields. `@ConfigurationProperties` binds a group of properties to a structured object.

**2. Why it matters:** Choosing the right approach affects maintainability. @Value is fine for simple cases; complex configuration demands @ConfigurationProperties.

**3. Internal working:**

| Aspect | @Value | @ConfigurationProperties |
|---|---|---|
| Granularity | Single value | Grouped properties |
| Type safety | SpEL parsing | Compile-time binding |
| Validation | Manual | @Validated integration |
| Relaxed binding | No | Yes (kebab-case, etc.) |
| IDE support | Limited | Full with processor |
| Meta-data | None | Generated |

```
// @Value - simple, single values
@Value("${app.name}")
private String appName;

@Value("${app.timeout:5000}")
private int timeout;

// @ConfigurationProperties - structured
@ConfigurationProperties(prefix = "app")
public class AppProperties {
    private String name;
    private int timeout;
    private Security security;
    // Hierarchical, validated, documented
}
```

**4. Real-world example:** A service with multiple integrations:

```
// Bad - scattered @Value across codebase
@Value("${stripe.api-key}") private String stripeKey;
@Value("${stripe.timeout}") private int stripeTimeout;
@Value("${stripe.retry}") private int stripeRetry;

// Good - centralized configuration
@ConfigurationProperties(prefix = "stripe")
public class StripeProperties {
    private String apiKey;
    private Duration timeout;
    private int retry;
    // All Stripe config in one place
}
```

**5. Pitfalls:**

- Using @Value for more than 2-3 related properties
- Missing properties with @Value fails silently with defaults
- SpEL expressions in @Value can be complex and error-prone

**6. Follow-up questions:**

- Can you use SpEL expressions in @ConfigurationProperties?
- How does relaxed binding work (app-name vs appName)?

**7. Senior summary:** "@Value for simple, single values; @ConfigurationProperties for structured configuration. I use @ConfigurationProperties for anything beyond basic settings—it provides validation, relaxed binding, IDE support, and keeps related configuration together."

---

# Question 27: How profiles work in Spring Boot?

**1. Definition:** Profiles are named logical groups of configuration that can be activated to customize application behavior for different environments (dev, staging, prod).

**2. Why it matters:** Same artifact, different environments. Profiles enable environment-specific configuration without code changes or rebuilding.

**3. Internal working:** Profile-specific files:

- `application-dev.yml`
- `application-prod.yml`
- `application-{profile}.yml`

Loading order (later overrides earlier):

1. `application.yml` (base)
2. `application-{active-profile}.yml`
3. External configuration

Profile-specific beans:

```
@Configuration
@Profile("dev")
public class DevConfig {
    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder().build();
    }
}

@Configuration
@Profile("prod")
public class ProdConfig {
    @Bean
    public DataSource dataSource() {
        return createProductionPool();
    }
}
```

**4. Real-world example:** Multi-environment setup:

```
# application.yml (base)
spring:
  application:
    name: payment-service

# application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:dev
logging:
  level:
    root: DEBUG

# application-prod.yml
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST}/payments
logging:
  level:
    root: WARN
```

**5. Pitfalls:**

- Hardcoding profile in code instead of externalizing
- Too many profile-specific differences (indicates deployment issues)
- Sensitive data in profile files committed to source control

**6. Follow-up questions:**

- How do you activate multiple profiles simultaneously?
- What's the difference between @Profile and @ConditionalOnProperty?

**7. Senior summary:** "Profiles group configuration by environment. I use base application.yml for common settings, profile-specific files for overrides. Activate via spring.profiles.active environment variable in production. Keep profile differences minimal—mostly connection strings and log levels."

---

# Question 28: How to activate profiles?

**1. Definition:** Profiles are activated via `spring.profiles.active` property, which can be set through environment variables, command-line arguments, properties files, or programmatically.

**2. Why it matters:** Profile activation strategy affects deployment flexibility. Externalizing profile selection enables the same artifact to run anywhere.

**3. Internal working:** Activation methods (priority order):

```
# 1. Command line argument
java -jar app.jar --spring.profiles.active=prod

# 2. JVM system property
java -Dspring.profiles.active=prod -jar app.jar

# 3. Environment variable
SPRING_PROFILES_ACTIVE=prod java -jar app.jar

# 4. In application.properties (not recommended for prod profile)
spring.profiles.active=dev

# 5. Programmatically
SpringApplication app = new SpringApplication(MyApp.class);
app.setAdditionalProfiles("prod");
app.run(args);
```

Multiple profiles:

```
SPRING_PROFILES_ACTIVE=prod,metrics,feature-x
```

Profile groups (Spring Boot 2.4+):

```
spring:
  profiles:
    group:
      prod: [prod-db, prod-security, prod-logging]
```

**4. Real-world example:** Kubernetes deployment:

```
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      containers:
        - name: payment-service
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "prod,kubernetes"
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: db-config
                  key: host
```

**5. Pitfalls:**

- Setting profile in application.properties defeats the purpose
- Forgetting that profile order matters for property conflicts
- Using profiles for feature flags (use @ConditionalOnProperty instead)

**6. Follow-up questions:**

- What happens when multiple active profiles define the same property?
- How do profile groups work?

**7. Senior summary:** "Activate via SPRING_PROFILES_ACTIVE environment variable in production—never hardcode in properties. Multiple profiles can be active simultaneously, with later profiles overriding earlier ones. I use profile groups to bundle related configurations."

---

# Question 29: What is @Profile?

**1. Definition:** `@Profile` is an annotation that marks a component, configuration, or bean to only be registered when specific profile(s) are active.

**2. Why it matters:** It enables environment-specific beans without runtime conditionals. Different implementations can be swapped based on environment.

**3. Internal working:**

```
@Service
@Profile("dev")
public class MockPaymentService implements PaymentService {
    // Returns success without calling real gateway
}

@Service
@Profile("prod")
public class RealPaymentService implements PaymentService {
    // Calls actual payment gateway
}
```

Expression support:

```
@Profile("!prod")           // NOT prod
@Profile({"dev", "test"})   // dev OR test
@Profile("prod & feature-x") // prod AND feature-x (Spring 5.1+)
```

**4. Real-world example:** A notification service with environment-specific implementations:

```
@Service
@Profile("prod")
public class SmsNotificationService implements NotificationService {
    @Autowired
    private TwilioClient twilio;
    // Real SMS sending
}

@Service
@Profile("!prod")
public class LoggingNotificationService implements NotificationService {
    public void send(Notification n) {
        log.info("Would send: {}", n);  // Log instead of send
    }
}
```

**5. Pitfalls:**

- Creating too many profile-specific beans (consider strategy pattern)
- Using @Profile when @ConditionalOnProperty is more appropriate
- Forgetting that missing profile means bean never registers

**6. Follow-up questions:**

- Can you use NOT logic with @Profile?
- When would you prefer @ConditionalOnProperty over @Profile?

**7. Senior summary:** "@Profile conditionally registers beans based on active profiles. I use it for environment-specific implementations like mock services in dev. For feature flags or fine-grained control, I prefer @ConditionalOnProperty as it's more flexible."

---

# Question 30: How to override auto-configured beans?

**1. Definition:** Define your own bean of the same type—Spring Boot's auto-configuration uses `@ConditionalOnMissingBean` and will back off when it finds your bean.

**2. Why it matters:** Production often requires customization beyond defaults. Understanding the override mechanism enables fine-tuning without disabling entire auto-configurations.

**3. Internal working:** Auto-configuration pattern:

```
@Configuration
@ConditionalOnClass(DataSource.class)
public class DataSourceAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean  // Backs off if user defines one
    public DataSource dataSource(DataSourceProperties properties) {
        return createDefaultDataSource(properties);
    }
}
```

Your override:

```
@Configuration
public class CustomDataSourceConfig {

    @Bean  // No @ConditionalOnMissingBean - always created
    public DataSource dataSource() {
        HikariConfig config = new HikariConfig();
        config.setMaximumPoolSize(50);
        config.setConnectionTimeout(5000);
        // Custom configuration
        return new HikariDataSource(config);
    }
}
```

**4. Real-world example:** Custom RestTemplate with timeouts and interceptors:

```
@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
            .setConnectTimeout(Duration.ofSeconds(5))
            .setReadTimeout(Duration.ofSeconds(30))
            .additionalInterceptors(new LoggingInterceptor())
            .errorHandler(new CustomErrorHandler())
            .build();
    }
}
```

This overrides any auto-configured RestTemplate.

**5. Pitfalls:**

- Partially overriding—some auto-config beans may still be created
- Not checking what the auto-configuration provides before overriding
- Breaking other auto-configurations that depend on the default bean

**6. Follow-up questions:**

- What if you only want to customize, not replace, an auto-configured bean?
- How does @Primary work with auto-configuration?

**7. Senior summary:** "Define your own bean of the same type—auto-configuration backs off via @ConditionalOnMissingBean. For customization without full replacement, use customizer callbacks like WebMvcConfigurer or RestTemplateCustomizer. Always understand what you're overriding."

---

# Question 31: What is externalized configuration?

**1. Definition:** Externalized configuration is the practice of keeping configuration outside the application code, allowing the same build to run in different environments without modification.

**2. Why it matters:** It's a 12-factor app principle. Same artifact in dev, staging, and prod eliminates "works in dev, fails in prod" issues.

**3. Internal working:** Property source hierarchy (highest to lowest priority):

1. Command-line arguments (`--server.port=9090`)
2. JNDI attributes
3. Java System properties (`-Dserver.port=9090`)
4. OS environment variables (`SERVER_PORT=9090`)
5. `application-{profile}.properties` outside JAR
6. `application-{profile}.properties` inside JAR
7. `application.properties` outside JAR
8. `application.properties` inside JAR
9. `@PropertySource` annotations
10. Default properties

External locations:

- `./config/` subdirectory
- Current directory
- `classpath:/config/`
- `classpath:/`

**4. Real-world example:** Kubernetes ConfigMap and Secrets:

```
# ConfigMap for non-sensitive config
spring:
  datasource:
    url: jdbc:postgresql://db:5432/app

---
# Secret mounted as environment variables
DB_USERNAME: (from Secret)
DB_PASSWORD: (from Secret)
```

Application using both:

```
spring:
  datasource:
    url: ${DATABASE_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
```

**5. Pitfalls:**

- Not understanding property source precedence
- Secrets in properties files committed to git
- Overcomplicating with too many property sources

**6. Follow-up questions:**

- How do you reference secrets securely in configuration?
- What's the config directory search order?

**7. Senior summary:** "Externalized configuration separates config from code. Spring Boot's property source hierarchy allows overrides at every level—from defaults to command-line. I use environment variables for secrets and config files for non-sensitive settings."

---

# Question 32: How environment variables work in Spring Boot?

**1. Definition:** Environment variables are automatically mapped to Spring properties using relaxed binding—`MY_APP_TIMEOUT` becomes `my.app.timeout` or `my-app-timeout`.

**2. Why it matters:** Environment variables are the standard configuration mechanism in containerized environments. Understanding the mapping enables proper property naming.

**3. Internal working:** Relaxed binding rules:

```
Environment Variable    →  Property Name
---------------------------------------------
SERVER_PORT            →  server.port
SPRING_DATASOURCE_URL  →  spring.datasource.url
MY_APP_RETRY_COUNT     →  my.app.retry-count
                          my.app.retryCount
                          my-app-retry-count
```

Conventions:

- Replace `.` with `_`
- Replace `-` with `_`
- Use UPPERCASE
- List/array items: `MY_PROP_0=value1`, `MY_PROP_1=value2`

```
# All equivalent in application.yml
my:
  app:
    retry-count: 3      # Kebab case
    retryCount: 3       # Camel case
    retry_count: 3      # Underscore
```

**4. Real-world example:** Docker Compose with environment variables:

```
services:
  payment-service:
    image: payment-service:latest
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/payments
      - SPRING_DATASOURCE_USERNAME=${DB_USER}
      - SPRING_DATASOURCE_PASSWORD=${DB_PASS}
      - PAYMENT_GATEWAY_API_KEY=${STRIPE_KEY}
      - PAYMENT_GATEWAY_TIMEOUT=10000
```

**5. Pitfalls:**

- Case sensitivity on Linux vs. Windows
- Forgetting the underscore replacement for dots
- Environment variables with special characters not working

**6. Follow-up questions:**

- How do you bind environment variables to list properties?
- What's the precedence between env vars and property files?

**7. Senior summary:** "Environment variables map to properties via relaxed binding: uppercase, underscores replacing dots. `DATABASE_URL` becomes `database.url`. They take precedence over property files but not command-line arguments. Essential for container-based deployments."

---

# Question 33: What is @ConditionalOnProperty?

**1. Definition:** `@ConditionalOnProperty` is a condition that registers a bean only if a specific property has a specific value (or exists/doesn't exist).

**2. Why it matters:** It enables feature flags, optional features, and configuration-driven architecture without code changes.

**3. Internal working:**

```
@Bean
@ConditionalOnProperty(
    name = "feature.new-checkout",
    havingValue = "true",
    matchIfMissing = false
)
public CheckoutService newCheckoutService() {
    return new NewCheckoutService();
}

@Bean
@ConditionalOnProperty(
    name = "feature.new-checkout",
    havingValue = "false",
    matchIfMissing = true  // Default when property missing
)
public CheckoutService legacyCheckoutService() {
    return new LegacyCheckoutService();
}
```

Attributes:

- `name`/`value`: Property name(s)
- `havingValue`: Expected value (default: not "false")
- `matchIfMissing`: Match if property undefined
- `prefix`: Property prefix

**4. Real-world example:** Feature toggling in production:

```
@Configuration
public class FeatureConfig {

    @Bean
    @ConditionalOnProperty(
        prefix = "features",
        name = "async-processing",
        havingValue = "true"
    )
    public OrderProcessor asyncProcessor() {
        return new AsyncOrderProcessor();
    }

    @Bean
    @ConditionalOnProperty(
        prefix = "features",
        name = "async-processing",
        havingValue = "false",
        matchIfMissing = true
    )
    public OrderProcessor syncProcessor() {
        return new SyncOrderProcessor();
    }
}
```

Flip features without redeployment: `features.async-processing=true`

**5. Pitfalls:**

- Forgetting `matchIfMissing` logic (defaults to false)
- Using for complex boolean expressions (consider custom Condition)
- Property typos silently falling to default

**6. Follow-up questions:**

- How do you implement AND/OR logic with multiple properties?
- What's the difference between @ConditionalOnProperty and @Profile?

**7. Senior summary:** "@ConditionalOnProperty enables property-driven bean registration for feature flags and optional components. I use it over @Profile for fine-grained control and runtime flexibility. Set matchIfMissing carefully—it determines default behavior."

---

# Question 34: What is @ConditionalOnClass?

**1. Definition:** `@ConditionalOnClass` registers a bean only if specified class(es) are present on the classpath at configuration time.

**2. Why it matters:** It enables optional integrations—if a library is present, configure it automatically. This is the foundation of starter dependencies.

**3. Internal working:**

```
@Configuration
@ConditionalOnClass(RedisTemplate.class)  // Only if Redis client on classpath
public class RedisAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public RedisTemplate<String, Object> redisTemplate(
            RedisConnectionFactory factory) {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);
        return template;
    }
}
```

Works with:

- `value`: Class array (must be loadable)

- `name`: Class names as strings (for optional dependencies)

**4. Real-world example:** Supporting multiple cache providers:

```
@Configuration
public class CacheAutoConfiguration {

    @Configuration
    @ConditionalOnClass(RedisCache.class)
    static class RedisCacheConfig {
        @Bean
        public CacheManager cacheManager(RedisConnectionFactory cf) {
            return RedisCacheManager.create(cf);
        }
    }

    @Configuration
    @ConditionalOnClass(CaffeineCacheManager.class)
    @ConditionalOnMissingBean(CacheManager.class)
    static class CaffeineCacheConfig {
        @Bean
        public CacheManager cacheManager() {
            return new CaffeineCacheManager();
        }
    }
}
```

Add redis dependency → Redis cache auto-configured.

**5. Pitfalls:**

- Using `value` for classes from optional dependencies (use `name` instead)
- Not combining with @ConditionalOnMissingBean for proper back-off
- Class loading issues with modular applications

**6. Follow-up questions:**

- Why use `name` attribute vs `value` attribute?
- How do you handle optional dependencies that may not be on classpath?

**7. Senior summary:** "@ConditionalOnClass enables classpath-driven configuration—if library present, configure it. Combined with @ConditionalOnMissingBean, it's the pattern for auto-configuration. Use `name` attribute for optional dependencies to avoid ClassNotFoundException."

---

# Question 35: What is @ConditionalOnMissingBean?

**1. Definition:** `@ConditionalOnMissingBean` registers a bean only if no bean of the specified type is already defined, enabling the "back-off" pattern for auto-configuration.

**2. Why it matters:** This is what makes auto-configuration customizable. User beans always win—auto-configuration provides defaults only when needed.

**3. Internal working:**

```
@Configuration
public class JacksonAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean  // User's ObjectMapper takes precedence
    public ObjectMapper objectMapper(Jackson2ObjectMapperBuilder builder) {
        return builder.build();  // Default configuration
    }
}

// User's configuration - no @ConditionalOnMissingBean
@Configuration
public class CustomJacksonConfig {

    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper()
            .setSerializationInclusion(JsonInclude.Include.NON_NULL)
            .enable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
    }
}
```

Attributes:

- `value`/`type`: Bean class(es)
- `name`: Bean name(s)
- `annotation`: Beans with specific annotation
- `search`: SearchStrategy for bean lookup scope

**4. Real-world example:** Custom transaction manager overriding default:

```
// Auto-configuration provides default
@Bean
@ConditionalOnMissingBean(PlatformTransactionManager.class)
public PlatformTransactionManager transactionManager(DataSource ds) {
    return new DataSourceTransactionManager(ds);
}

// Your custom transaction manager
@Bean
@Primary  // Good practice when overriding
public PlatformTransactionManager transactionManager(
        EntityManagerFactory emf) {
    JpaTransactionManager tm = new JpaTransactionManager(emf);
    tm.setDefaultTimeout(30);
    return tm;
}
```

**5. Pitfalls:**

- Condition checked after user configs but before other auto-configs
- Bean name matching is exact (case-sensitive)
- @Primary doesn't affect @ConditionalOnMissingBean evaluation

**6. Follow-up questions:**

- When is the condition evaluated relative to user beans?
- How does SearchStrategy affect bean lookup?

**7. Senior summary:** "@ConditionalOnMissingBean implements the back-off pattern—auto-configuration yields to user-defined beans. It's evaluated after @Configuration classes but before other auto-configurations. Define your bean without this annotation to override defaults."

---

# Question 36: How to disable auto-configuration?

**1. Definition:** Disable via `@SpringBootApplication(exclude=...)`, `spring.autoconfigure.exclude` property, or by defining your own beans that trigger `@ConditionalOnMissingBean`.

**2. Why it matters:** Not all auto-configurations are needed. Disabling unnecessary ones reduces startup time, memory footprint, and potential conflicts.

**3. Internal working:**

```
// Method 1: Exclude in annotation
@SpringBootApplication(exclude = {
    DataSourceAutoConfiguration.class,
    HibernateJpaAutoConfiguration.class
})
public class Application { }

// Method 2: Exclude by name (for optional dependencies)
@SpringBootApplication(excludeName = {
    "org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration"
})
```

```
# Method 3: Properties file
spring:
  autoconfigure:
    exclude:
      - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
      - org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```

**4. Real-world example:** API gateway that doesn't need database:

```
@SpringBootApplication(exclude = {
    DataSourceAutoConfiguration.class,
    HibernateJpaAutoConfiguration.class,
    FlywayAutoConfiguration.class,
    LiquibaseAutoConfiguration.class
})
public class GatewayApplication { }
```

Check what's being configured:

```
java -jar app.jar --debug | grep -i "autoconfiguration"
```

**5. Pitfalls:**

- Excluding a configuration that others depend on
- Not knowing which auto-configuration enables what
- Using excludeName for classes that might be renamed

**6. Follow-up questions:**

- How do you find out which auto-configurations are active?
- What's the impact of excluding SecurityAutoConfiguration?

**7. Senior summary:** "Disable via exclude attribute in @SpringBootApplication or spring.autoconfigure.exclude property. Use --debug flag to see what's being configured. Be careful about dependencies between auto-configurations—excluding one might break another."

---

# Question 37: What is exclude in @SpringBootApplication?

**1. Definition:** The `exclude` attribute in `@SpringBootApplication` disables specific auto-configuration classes, preventing them from being applied.

**2. Why it matters:** Explicit exclusion is clearer than relying on conditions. It documents intent and prevents auto-configuration from being applied accidentally.

**3. Internal working:**

```
@SpringBootApplication(
    exclude = {
        DataSourceAutoConfiguration.class,
        SecurityAutoConfiguration.class
    },
    excludeName = {
        "org.springframework.boot.autoconfigure.amqp.RabbitAutoConfiguration"
    }
)
public class Application { }
```

`exclude` vs `excludeName`:

- `exclude`: Type-safe, compile-time checked
- `excludeName`: String-based, for optional/conditional dependencies

Internally, exclusions are processed by `AutoConfigurationImportSelector` before conditions are evaluated.

**4. Real-world example:** Microservice with custom security:

```
@SpringBootApplication(exclude = {
    // Disable default security, we use custom OAuth2
    SecurityAutoConfiguration.class,
    UserDetailsServiceAutoConfiguration.class,

    // Disable default error handling
    ErrorMvcAutoConfiguration.class
})
public class PaymentService { }
```

**5. Pitfalls:**

- Using exclude for classes not on classpath throws exception
- Forgetting dependent auto-configurations still run
- Over-excluding, then wondering why things don't work

**6. Follow-up questions:**

- When would you use excludeName instead of exclude?
- How do you find all available auto-configuration classes?

**7. Senior summary:** "exclude in @SpringBootApplication prevents specific auto-configurations from running. Use type-safe exclude for required dependencies, excludeName for optional ones. It's more explicit than relying on conditions to disable features."

---

# Question 38: What is DevTools?

**1. Definition:** Spring Boot DevTools is a module that provides development-time features like automatic restart, live reload, and sensible development defaults.

**2. Why it matters:** Developer productivity directly impacts delivery speed. Hot reload and automatic restart eliminate the manual restart cycle, saving significant development time.

**3. Internal working:** Key features:

1. **Automatic restart**: Monitors classpath, restarts on change
2. **LiveReload**: Browser refresh on resource changes
3. **Property defaults**: Disables caching, enables debug features
4. **Remote debugging**: Development on remote servers

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
```

Restart vs. Reload:

- Base classloader: Third-party JARs (not reloaded)

- Restart classloader: Your code (reloaded on change)

**4. Real-world example:** Development workflow:

```
# application-dev.yml
spring:
  devtools:
    restart:
      enabled: true
      additional-paths: src/main/resources
      exclude: static/**,templates/**
    livereload:
      enabled: true
```

Change Java code → auto-restart (~2s) Change HTML/CSS → browser auto-refresh

**5. Pitfalls:**

- DevTools in production (it's auto-disabled in JARs, but verify)
- Slow restarts due to large classpath (use excludes)
- Conflicts with other hot-reload tools (JRebel)

**6. Follow-up questions:**

- How is DevTools automatically disabled in production?
- What's the difference between restart and hot-swap?

**7. Senior summary:** "DevTools accelerates development with auto-restart, live reload, and dev-friendly defaults. It uses two classloaders to restart only your code, not dependencies. Automatically disabled in production JARs. Essential for modern Spring Boot development."

---

# Question 39: Hot reload vs Restart

**1. Definition:** Hot reload (hot swap) replaces individual classes in a running JVM. Restart loads a new classloader with all application classes. Hot reload is faster but limited; restart is slower but handles all changes.

**2. Why it matters:** Understanding the difference helps set realistic expectations. Hot reload fails silently for structural changes, causing confusion.

**3. Internal working:**

| Aspect | Hot Reload (JVM HotSwap) | DevTools Restart |
|---|---|---|
| Speed | ~100ms | ~2-5s |
| What works | Method body changes | All changes |
| What fails | Signature changes, new classes | Nothing |
| Mechanism | JVM debug agent | Classloader swap |
| State | Preserved | Lost |

DevTools restart:

1. Monitors classpath for changes
2. Throws away restart classloader
3. Creates new restart classloader
4. Reloads all application classes
5. Reinitializes ApplicationContext

**4. Real-world example:** Development scenarios:

```
// Hot reload works: Change method implementation
public String greet() {
    return "Hello";  // Change to "Hi" - hot reload works
}

// Hot reload fails: Add new method
public String newMethod() {  // Requires restart
    return "New";
}

// Hot reload fails: Change method signature
public String greet(String name) {  // Requires restart
    return "Hello " + name;
}
```

For complex changes, consider JRebel (commercial) for true hot reload.

**5. Pitfalls:**

- Expecting hot reload to handle structural changes
- Not realizing DevTools restart loses in-memory state
- Debug mode hot swap ≠ DevTools restart

**6. Follow-up questions:**

- How does JRebel differ from DevTools?
- When would restart fail to pick up changes?

**7. Senior summary:** "Hot reload (JVM HotSwap) is fast but only handles method body changes. DevTools restart is slower but handles all changes by swapping classloaders. For development, DevTools restart is reliable; for maximum productivity with no restart, consider JRebel."

---

# Question 40: Difference between WAR and JAR deployment?

**1. Definition:** JAR deployment packages an embedded server inside the application (self-contained). WAR deployment requires an external servlet container like Tomcat or WebLogic.

**2. Why it matters:** Deployment strategy affects infrastructure, operations, and scalability. JAR is the modern approach for microservices and containers; WAR is for enterprise environments requiring external servers.

**3. Internal working:**

| Aspect | JAR (Embedded) | WAR (External) |
|---|---|---|
| Server | Inside app | External Tomcat/WebLogic |
| Startup | `java -jar app.jar` | Deploy to container |
| Port config | application.properties | server.xml |
| Clustering | App-level | Container-level |
| Memory | Per-app JVM | Shared JVM |
| Docker | Simple | Complex |

WAR conversion:

```
@SpringBootApplication
public class Application extends SpringBootServletInitializer {

    @Override
    protected SpringApplicationBuilder configure(
            SpringApplicationBuilder builder) {
        return builder.sources(Application.class);
    }

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

```
<packaging>war</packaging>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>  <!-- Container provides this -->
</dependency>
```

**4. Real-world example:** Enterprise migration:

- Legacy: 10 WARs on 2 WebLogic clusters
- Modern: 10 JARs in Kubernetes, each with own resources
- Benefit: Independent scaling, isolated failures, simpler deployments

Some enterprises still require WAR for:

- Shared container management
- Legacy SSO integration
- Compliance requirements

**5. Pitfalls:**

- Mixing embedded and provided server dependencies
- Forgetting SpringBootServletInitializer for WAR
- Container classloader conflicts

**6. Follow-up questions:**

- Can a Spring Boot WAR still run standalone?
- How do you handle logging conflicts between container and app?

**7. Senior summary:** ""JAR with embedded server is the modern default—self-contained, container-friendly, independently scalable. WAR is for enterprise environments requiring external servers. Spring Boot WARs can run both ways. I default to JAR unless there's a specific enterprise requirement."

---

# REST + MVC QUESTIONS (41-61)

---

## Question 41: Difference between @Controller and @RestController?

**1. Definition:** `@Controller` returns view names for template resolution. `@RestController` combines `@Controller` + `@ResponseBody`, returning data directly serialized to JSON/XML.

**2. Why it matters:** Using the wrong annotation causes confusing errors—view resolver failures or unwanted response wrapping. Proper usage defines your API contract.

**3. Internal working:**

```
@Controller
public class ViewController {
    @GetMapping("/page")
    public String page(Model model) {
        model.addAttribute("data", getData());
        return "page";  // Returns view name → ViewResolver → HTML
    }

    @GetMapping("/api/data")
    @ResponseBody  // Explicit - return JSON
    public Data getData() {
        return new Data();
    }
}

@RestController  // = @Controller + @ResponseBody on all methods
public class ApiController {
    @GetMapping("/api/data")
    public Data getData() {
        return new Data();  // Automatically serialized to JSON
    }
}
```

`@RestController` internally:

```
@Controller
@ResponseBody
public @interface RestController { }
```

**4. Real-world example:** Mixed application with UI and API:

```
@Controller
@RequestMapping("/admin")
public class AdminViewController {
    @GetMapping("/dashboard")
    public String dashboard(Model model) {
        model.addAttribute("stats", statsService.get());
        return "admin/dashboard";  // Thymeleaf template
    }
}

@RestController
@RequestMapping("/api/v1/admin")
public class AdminApiController {
    @GetMapping("/stats")
    public ResponseEntity<Stats> getStats() {
        return ResponseEntity.ok(statsService.get());
    }
}
```

**5. Pitfalls:**

- Using @Controller for REST APIs and forgetting @ResponseBody
- Using @RestController for MVC views (returns literal string, not view)
- Not understanding that @RestController affects ALL methods in class

**6. Follow-up questions:**

- Can you mix @ResponseBody and view returns in @Controller?
- How does content negotiation work with @RestController?

**7. Senior summary:** "@RestController is @Controller + @ResponseBody applied to all methods—for REST APIs returning data. @Controller is for MVC applications returning view names. I use @RestController for pure APIs, @Controller for template-based UIs, sometimes mixing both in the same application."

---

# Question 42: Difference between @RequestParam and @PathVariable?

**1. Definition:** `@PathVariable` extracts values from URI path segments (`/users/{id}`). `@RequestParam` extracts values from query parameters (`/users?id=123`).

**2. Why it matters:** RESTful design uses path variables for resource identification and query params for filtering/pagination. Proper usage affects API discoverability and cacheability.

**3. Internal working:**

```
// PathVariable - part of resource identifier
@GetMapping("/users/{userId}/orders/{orderId}")
public Order getOrder(
    @PathVariable Long userId,
    @PathVariable("orderId") Long order) {  // Custom name binding
    return orderService.find(userId, order);
}

// RequestParam - filtering, pagination
@GetMapping("/orders")
public Page<Order> getOrders(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(required = false) String status,
    @RequestParam List<String> tags) {  // Multi-value: ?tags=a&tags=b
    return orderService.findAll(page, size, status, tags);
}
```

Matrix variables (less common):

```
// /cars;color=red;year=2020
@GetMapping("/cars")
public List<Car> getCars(@MatrixVariable String color, @MatrixVariable int year)
```

**4. Real-world example:** E-commerce product API:

```
@RestController
@RequestMapping("/api/v1/products")
public class ProductController {

    // GET /products/123 - specific product
    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) { }

    // GET /products?category=electronics&minPrice=100&sort=price
    @GetMapping
    public Page<Product> searchProducts(
        @RequestParam(required = false) String category,
        @RequestParam(required = false) BigDecimal minPrice,
        @RequestParam(defaultValue = "name") String sort,
        Pageable pageable) { }
}
```

**5. Pitfalls:**

- Using query params for resource identification (not RESTful)
- Forgetting `required = false` for optional query params
- Path variable name mismatch with method parameter

**6. Follow-up questions:**

- How do you handle optional path variables?
- What happens if a required @RequestParam is missing?

**7. Senior summary:** "@PathVariable for resource identification in URL path, @RequestParam for query parameters used in filtering and pagination. Path variables are mandatory by default; query params can have defaults. Follow REST conventions—identifiers in path, filters in query string."

---

# Question 43: What is @RequestBody?

**1. Definition:** `@RequestBody` binds the HTTP request body to a method parameter, deserializing JSON/XML content into a Java object using configured HttpMessageConverters.

**2. Why it matters:** It's the primary mechanism for receiving structured data in REST APIs. Understanding its deserialization behavior is crucial for proper API design and error handling.

**3. Internal working:**

```
@PostMapping("/orders")
public Order createOrder(@RequestBody @Valid OrderRequest request) {
    // request is deserialized from JSON body
    return orderService.create(request);
}
```

Processing flow:

1. Request received with Content-Type header
2. `RequestResponseBodyMethodProcessor` handles @RequestBody
3. Appropriate `HttpMessageConverter` selected (Jackson for JSON)
4. Body deserialized to target type
5. If @Valid present, validation triggered
6. Object passed to method

**4. Real-world example:** Complex order creation:

```
public record OrderRequest(
    @NotNull Long customerId,
    @NotEmpty List<OrderItem> items,
    @Valid ShippingAddress address,
    String couponCode
) {}

@PostMapping("/orders")
public ResponseEntity<Order> createOrder(
    @RequestBody @Valid OrderRequest request) {

    Order order = orderService.create(request);
    return ResponseEntity
        .created(URI.create("/orders/" + order.getId()))
        .body(order);
}
```

Request:

```
{
  "customerId": 123,
  "items": [{"productId": 456, "quantity": 2}],
  "address": {"street": "123 Main", "city": "NYC"}
}
```

**5. Pitfalls:**

- Missing Content-Type header causing 415 error
- Not handling deserialization exceptions globally
- Forgetting @Valid for nested object validation
- Request body can only be read once (streaming)

**6. Follow-up questions:**

- How do you handle deserialization errors gracefully?
- Can you have multiple @RequestBody parameters?

**7. Senior summary:** "@RequestBody deserializes HTTP body to Java objects using HttpMessageConverters. Always combine with @Valid for input validation. Handle deserialization errors with @ExceptionHandler. The body is consumed once—can't read multiple times."

---

# Question 44: What is @ResponseBody?

**1. Definition:** `@ResponseBody` indicates that the method return value should be serialized directly to the HTTP response body, not interpreted as a view name.

**2. Why it matters:** It's what makes REST APIs return JSON instead of trying to render templates. Understanding the serialization pipeline helps debug response issues.

**3. Internal working:**

```
@Controller
public class MixedController {

    @GetMapping("/api/user")
    @ResponseBody  // Return as JSON
    public User getUser() {
        return userService.getCurrent();
    }

    @GetMapping("/user")
    public String userPage() {
        return "user/profile";  // View name
    }
}
```

Serialization flow:

1. Method returns object
2. `RequestResponseBodyMethodProcessor` intercepts
3. Content negotiation determines format (Accept header)
4. `HttpMessageConverter` serializes (Jackson → JSON)
5. Response written with Content-Type header

@RestController makes this implicit for all methods.

**4. Real-world example:** Custom serialization control:

```
@Controller
public class ReportController {

    @GetMapping("/report/json")
    @ResponseBody
    public Report getReportJson() {
        return reportService.generate();  // JSON
    }

    @GetMapping("/report/html")
    public String getReportHtml(Model model) {
        model.addAttribute("report", reportService.generate());
        return "report/view";  // HTML template
    }

    @GetMapping(value = "/report/csv", produces = "text/csv")
    @ResponseBody
    public byte[] getReportCsv() {
        return reportService.generateCsv();  // CSV bytes
    }
}
```

**5. Pitfalls:**

- Returning void with @ResponseBody (empty response, not void)
- Not setting produces attribute for non-JSON formats
- Circular references in objects causing serialization failure

**6. Follow-up questions:**

- How does content negotiation work?
- How do you customize JSON serialization for specific endpoints?

**7. Senior summary:** "@ResponseBody bypasses view resolution, serializing return values directly to response body. Content type determined by Accept header and produces attribute. Implicit in @RestController. Use for REST endpoints returning data, not views."

---

# Question 45: How JSON conversion happens?

**1. Definition:** Spring Boot uses Jackson's `ObjectMapper` through `MappingJackson2HttpMessageConverter` to serialize objects to JSON and deserialize JSON to objects.

**2. Why it matters:** JSON handling affects performance, security, and API contract. Understanding Jackson configuration helps solve serialization issues and optimize performance.

**3. Internal working:**

```
Request → HttpMessageConverter → Object
Object → HttpMessageConverter → Response

For JSON:
MappingJackson2HttpMessageConverter wraps ObjectMapper

ObjectMapper configuration:
- SerializationFeature (null handling, date format)
- DeserializationFeature (unknown properties, null handling)
- Modules (JavaTimeModule for dates)
```

Spring Boot auto-configuration:

```yaml
spring:
  jackson:
    serialization:
      write-dates-as-timestamps: false
    deserialization:
      fail-on-unknown-properties: false
    default-property-inclusion: non_null
    date-format: yyyy-MM-dd'T'HH:mm:ss.SSSZ
```

**4. Real-world example:** Custom ObjectMapper for API:

```java
@Configuration
public class JacksonConfig {

    @Bean
    @Primary
    public ObjectMapper objectMapper() {
        return JsonMapper.builder()
            .addModule(new JavaTimeModule())
            .addModule(new Jdk8Module())
            .serializationInclusion(JsonInclude.Include.NON_NULL)
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
            .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
            .enable(DeserializationFeature.FAIL_ON_NULL_FOR_PRIMITIVES)
            .build();
    }
}
```

**5. Pitfalls:**

- Multiple ObjectMapper beans causing inconsistent serialization
- Not registering JavaTimeModule for Java 8 dates
- Circular references causing infinite loops
- @JsonIgnore vs @JsonProperty conflicts

**6. Follow-up questions:**

- How do you handle circular references?
- How do you use different ObjectMappers for different endpoints?

**7. Senior summary:** "Jackson ObjectMapper handles JSON conversion via MappingJackson2HttpMessageConverter. Configure via spring.jackson.* properties or custom ObjectMapper bean. Always register JavaTimeModule for dates, disable fail-on-unknown-properties for forward compatibility."

---

# Question 46: Role of Jackson in Spring Boot?

**1. Definition:** Jackson is the default JSON processing library in Spring Boot, providing serialization (object→JSON), deserialization (JSON→object), and data binding through `ObjectMapper`.

**2. Why it matters:** Jackson is involved in almost every REST request/response. Understanding its features enables proper API design, performance optimization, and security hardening.

**3. Internal working:** Jackson components:

- `ObjectMapper`: Main API for all operations
- `JsonParser`: Low-level streaming read
- `JsonGenerator`: Low-level streaming write
- `@JsonProperty`, `@JsonIgnore`, `@JsonFormat`: Annotations

Auto-configured by `JacksonAutoConfiguration`:

```
@Bean
@ConditionalOnMissingBean
public ObjectMapper jacksonObjectMapper(Jackson2ObjectMapperBuilder builder) {
    return builder.createXmlMapper(false).build();
}
```

**4. Real-world example:** API with custom serialization:

```
public class Order {
    private Long id;

    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    private LocalDateTime orderDate;

    @JsonIgnore  // Don't expose internal data
    private String internalReference;

    @JsonProperty("customer")  // Rename in JSON
    private String customerName;

    @JsonSerialize(using = MoneySerializer.class)
    private BigDecimal amount;
}

// Custom serializer
public class MoneySerializer extends JsonSerializer<BigDecimal> {
    @Override
    public void serialize(BigDecimal value, JsonGenerator gen,
                          SerializerProvider provider) throws IOException {
        gen.writeString(value.setScale(2, RoundingMode.HALF_UP).toString());
    }
}
```

**5. Pitfalls:**

- Exposing internal fields unintentionally (@JsonIgnore)
- Date format inconsistencies across APIs
- Performance issues with large objects (consider streaming)
- Security: polymorphic deserialization vulnerabilities

**6. Follow-up questions:**

- How do you handle polymorphic types?
- What's the performance difference between ObjectMapper and streaming API?

**7. Senior summary:** "Jackson is Spring Boot's default JSON library—ObjectMapper handles all serialization. Use annotations like @JsonFormat, @JsonIgnore, @JsonProperty for control. Configure globally via spring.jackson.* or custom ObjectMapper. Be aware of security implications with polymorphic deserialization."

---

# Question 47: What is HttpMessageConverter?

**1. Definition:** `HttpMessageConverter` is a strategy interface for converting HTTP request bodies to objects and objects to HTTP response bodies, supporting multiple formats (JSON, XML, form data).

**2. Why it matters:** Content negotiation relies on converters. Understanding the converter chain explains format selection and enables custom formats.

**3. Internal working:**

```
public interface HttpMessageConverter<T> {
    boolean canRead(Class<?> clazz, MediaType mediaType);
    boolean canWrite(Class<?> clazz, MediaType mediaType);
    List<MediaType> getSupportedMediaTypes();
    T read(Class<? extends T> clazz, HttpInputMessage input);
    void write(T t, MediaType contentType, HttpOutputMessage output);
}
```

Default converters (order matters):

1. `ByteArrayHttpMessageConverter` - byte[]
2. `StringHttpMessageConverter` - String
3. `FormHttpMessageConverter` - form data
4. `MappingJackson2HttpMessageConverter` - JSON
5. `MappingJackson2XmlHttpMessageConverter` - XML (if on classpath)

**4. Real-world example:** Custom converter for CSV:

```
@Component
public class CsvHttpMessageConverter
        extends AbstractHttpMessageConverter<List<Object>> {

    public CsvHttpMessageConverter() {
        super(new MediaType("text", "csv"));
    }

    @Override
    protected boolean supports(Class<?> clazz) {
        return List.class.isAssignableFrom(clazz);
    }

    @Override
    protected List<Object> readInternal(Class<? extends List<Object>> clazz,
                                        HttpInputMessage input) {
        // Parse CSV to List
    }

    @Override
    protected void writeInternal(List<Object> objects,
                                 HttpOutputMessage output) {
        // Write List as CSV
    }
}
```

Usage:

```
@GetMapping(value = "/export", produces = "text/csv")
public List<User> exportUsers() {
    return userService.findAll();
}
```

**5. Pitfalls:**

- Converter order affects selection (first match wins)
- Not configuring appropriate converters for content types
- Performance issues with large bodies (streaming converters)

**6. Follow-up questions:**

- How do you add a custom HttpMessageConverter?
- How does content negotiation use converters?

**7. Senior summary:** "HttpMessageConverter handles format conversion between HTTP bodies and Java objects. Spring Boot auto-configures converters for JSON, XML, form data. Custom converters enable proprietary formats. Converter selection based on Content-Type (read) and Accept headers (write)."

---

# Question 48: How to handle validation in Spring Boot?

**1. Definition:** Spring Boot uses Bean Validation (JSR-380) with Hibernate Validator, triggered via `@Valid` or `@Validated` annotations, with errors handled through `BindingResult` or exception handlers.

**2. Why it matters:** Input validation is the first line of defense against bad data. Proper validation prevents database errors, security issues, and business logic failures.

**3. Internal working:**

```
// DTO with validation
public record CreateUserRequest(
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    String email,

    @NotBlank @Size(min = 8, max = 100)
    String password,

    @Min(18) @Max(120)
    Integer age,

    @Valid  // Cascade validation to nested object
    Address address
) {}

// Controller
@PostMapping("/users")
public ResponseEntity<?> createUser(
    @RequestBody @Valid CreateUserRequest request) {
    // If validation fails, MethodArgumentNotValidException thrown
    return ResponseEntity.ok(userService.create(request));
}
```

Common annotations:

- `@NotNull`, `@NotBlank`, `@NotEmpty`
- `@Size`, `@Min`, `@Max`
- `@Email`, `@Pattern`
- `@Past`, `@Future`
- `@Valid` (cascading)

**4. Real-world example:** Global exception handling for validation:

```
@RestControllerAdvice
public class ValidationExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ValidationErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {

        Map<String, String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage,
                (e1, e2) -> e1  // Handle duplicates
            ));

        return ResponseEntity.badRequest()
            .body(new ValidationErrorResponse("Validation failed", errors));
    }
}
```

**5. Pitfalls:**

- Forgetting @Valid on nested objects
- Not handling validation exceptions globally
- Over-validating in controllers vs. service layer
- Using @NotNull when @NotBlank is needed for strings

**6. Follow-up questions:**

- How do you create custom validators?
- What's the difference between @Valid and @Validated?

**7. Senior summary:** "Use @Valid on @RequestBody to trigger Bean Validation. Handle MethodArgumentNotValidException in @ControllerAdvice for consistent error responses. Use @Valid for cascading. Custom validators via @Constraint. Validate at API boundary, trust internal code."

---

# Question 49: Difference between @Valid and @Validated?

**1. Definition:** `@Valid` is JSR-380 standard, triggers cascaded validation. `@Validated` is Spring's extension, enabling validation groups for conditional validation logic.

**2. Why it matters:** Validation groups enable different validation rules for create vs. update operations. Understanding both annotations enables sophisticated validation strategies.

**3. Internal working:**

```
// Validation groups
public interface OnCreate {}
public interface OnUpdate {}

public record UserRequest(
    @Null(groups = OnCreate.class)
    @NotNull(groups = OnUpdate.class)
    Long id,

    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    String name,

    @NotBlank(groups = OnCreate.class)
    String password
) {}

// Usage with @Validated
@RestController
@Validated  // Enable method-level validation
public class UserController {

    @PostMapping
    public User create(
        @RequestBody @Validated(OnCreate.class) UserRequest request) {
        // id must be null, password required
    }

    @PutMapping("/{id}")
    public User update(
        @PathVariable Long id,
        @RequestBody @Validated(OnUpdate.class) UserRequest request) {
        // id must not be null, password optional
    }
}
```

| Aspect | @Valid | @Validated |
|---|---|---|
| Origin | JSR-380 | Spring |
| Groups | No | Yes |
| Method params | Yes (nested) | Yes |
| Class-level | No | Yes |

**4. Real-world example:** Partial update validation:

```
@PatchMapping("/users/{id}")
public User partialUpdate(
    @PathVariable Long id,
    @RequestBody @Validated(OnPatch.class) UserPatchRequest request) {
    // Only validates fields present in patch
}
```

**5. Pitfalls:**

- Using @Valid when groups are needed
- Forgetting to specify groups (no validation runs)
- Complex group inheritance becoming unmaintainable

**6. Follow-up questions:**

- Can you use both @Valid and @Validated together?
- How do group sequences work?

**7. Senior summary:** "@Valid is standard JSR-380, sufficient for most cases. @Validated is Spring's extension adding validation groups for conditional validation. I use groups for create/update scenarios where different rules apply. Both trigger MethodArgumentNotValidException on failure."

---

# Question 50: How to create custom exception?

**1. Definition:** Custom exceptions extend `RuntimeException` (or appropriate base), carry domain-specific information, and are handled via `@ExceptionHandler` or `@ControllerAdvice` for proper HTTP responses.

**2. Why it matters:** Well-designed exceptions improve debugging, provide clear error messages to clients, and separate business logic from error handling.

**3. Internal working:**

```
// Custom exception
public class ResourceNotFoundException extends RuntimeException {
    private final String resourceType;
    private final Object resourceId;

    public ResourceNotFoundException(String resourceType, Object id) {
        super(String.format("%s with id %s not found", resourceType, id));
        this.resourceType = resourceType;
        this.resourceId = id;
    }
}

// Business exception with error code
public class PaymentFailedException extends RuntimeException {
    private final String errorCode;
    private final String providerMessage;

    public PaymentFailedException(String code, String provider, String msg) {
        super("Payment failed: " + msg);
        this.errorCode = code;
        this.providerMessage = msg;
    }
}
```

**4. Real-world example:** Complete exception handling:

```
// Domain exceptions
public class OrderException extends RuntimeException {
    private final ErrorCode errorCode;
    public enum ErrorCode { INVALID_STATE, INSUFFICIENT_STOCK, PAYMENT_FAILED }
}

// Handler
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("NOT_FOUND", ex.getMessage()));
    }

    @ExceptionHandler(OrderException.class)
    public ResponseEntity<ErrorResponse> handleOrderError(OrderException ex) {
        HttpStatus status = switch (ex.getErrorCode()) {
            case INVALID_STATE -> HttpStatus.CONFLICT;
            case INSUFFICIENT_STOCK -> HttpStatus.UNPROCESSABLE_ENTITY;
            case PAYMENT_FAILED -> HttpStatus.PAYMENT_REQUIRED;
        };
        return ResponseEntity.status(status)
            .body(new ErrorResponse(ex.getErrorCode().name(), ex.getMessage()));
    }
}

public record ErrorResponse(String code, String message, Instant timestamp) {
    public ErrorResponse(String code, String message) {
        this(code, message, Instant.now());
    }
}
```

**5. Pitfalls:**

- Exposing internal exception details to clients
- Not using appropriate HTTP status codes
- Creating too many exception classes (over-engineering)
- Catching and re-throwing without adding value

**6. Follow-up questions:**

- How do you log exceptions appropriately?
- How do you handle exceptions in async operations?

**7. Senior summary:** "Create domain exceptions extending RuntimeException with relevant context (error codes, resource info). Handle in @ControllerAdvice with proper HTTP status mapping. Never expose stack traces to clients. I categorize exceptions by HTTP status family for consistent handling."

---

# Question 51: What is @ControllerAdvice?

**1. Definition:** `@ControllerAdvice` is a specialization of `@Component` that enables global exception handling, model attribute binding, and data binding initialization across all controllers.

**2. Why it matters:** Centralized exception handling ensures consistent error responses, reduces code duplication, and makes error handling testable.

**3. Internal working:**

```
@RestControllerAdvice  // = @ControllerAdvice + @ResponseBody
public class GlobalExceptionHandler {

    // Exception handling
    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ErrorResponse> handleBusiness(BusinessException ex) {
        return ResponseEntity.badRequest().body(toError(ex));
    }

    // Global model attribute
    @ModelAttribute("currentUser")
    public User currentUser(@AuthenticationPrincipal UserDetails user) {
        return userService.findByUsername(user.getUsername());
    }

    // Data binder initialization
    @InitBinder
    public void initBinder(WebDataBinder binder) {
        binder.registerCustomEditor(LocalDate.class, new LocalDateEditor());
    }
}
```

Scoping options:

```
@ControllerAdvice(assignableTypes = {ApiController.class})
@ControllerAdvice(basePackages = "com.example.api")
@ControllerAdvice(annotations = RestController.class)
```

**4. Real-world example:** Comprehensive API error handling:

```
@RestControllerAdvice(annotations = RestController.class)
@Slf4j
public class ApiExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(
            MethodArgumentNotValidException ex) {
        Map<String, String> errors = extractFieldErrors(ex);
        return ResponseEntity.badRequest()
            .body(ApiError.validation(errors));
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(NOT_FOUND)
            .body(ApiError.notFound(ex.getMessage()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiError> handleUnexpected(Exception ex) {
        log.error("Unexpected error", ex);  // Log full stack trace
        return ResponseEntity.status(INTERNAL_SERVER_ERROR)
            .body(ApiError.internal("An unexpected error occurred"));
    }
}
```

**5. Pitfalls:**

- Multiple @ControllerAdvice with conflicting handlers (order matters)
- Not scoping @ControllerAdvice (applies to ALL controllers)
- Catching Exception.class without logging
- Missing ResponseBody in error handlers (use @RestControllerAdvice)

**6. Follow-up questions:**

- How do you order multiple @ControllerAdvice classes?
- How do you test @ControllerAdvice handlers?

**7. Senior summary:** "@ControllerAdvice enables global exception handling, model attributes, and binders. I use @RestControllerAdvice for APIs with scoping by package or annotation. Always log unexpected exceptions, never expose internal details. @Order controls precedence when multiple handlers exist."

---

# Question 52: What is @ControllerAdvice? (Repeated - Advanced)

**1. Definition:** Beyond basic exception handling, `@ControllerAdvice` is a cross-cutting mechanism that intercepts controller calls for AOP-like functionality without full AspectJ.

**2. Why it matters:** It enables separation of concerns—controllers focus on business logic while advice handles cross-cutting concerns like error handling, logging, and response modification.

**3. Internal working:** Processing order:

1. Request enters `DispatcherServlet`
2. `@ModelAttribute` methods from advice called
3. `@InitBinder` methods initialize binders
4. Controller method executed
5. If exception, `@ExceptionHandler` searched (controller first, then advice)

Multiple advice ordering:

```
@ControllerAdvice
@Order(1)  // Higher priority
public class SecurityExceptionHandler { }

@ControllerAdvice
@Order(2)  // Lower priority
public class BusinessExceptionHandler { }
```

**4. Real-world example:** Advanced use—response wrapping:

```
@RestControllerAdvice
public class ResponseWrapperAdvice implements ResponseBodyAdvice<Object> {

    @Override
    public boolean supports(MethodParameter returnType,
                            Class<? extends HttpMessageConverter<?>> converterType) {
        return !returnType.getParameterType().equals(ApiResponse.class);
    }

    @Override
    public Object beforeBodyWrite(Object body,
                                  MethodParameter returnType,
                                  MediaType selectedContentType,
                                  Class<? extends HttpMessageConverter<?>> converterType,
                                  ServerHttpRequest request,
                                  ServerHttpResponse response) {
        // Wrap all responses in standard format
        return ApiResponse.success(body);
    }
}
```

**5. Pitfalls:**

- ResponseBodyAdvice affecting all responses including errors
- Order conflicts between multiple advice classes
- Performance impact of complex advice logic

**6. Follow-up questions:**

- How do you implement response modification with @ControllerAdvice?
- What's RequestBodyAdvice used for?

**7. Senior summary:** "@ControllerAdvice is Spring's AOP-lite for web layer—handles exceptions, modifies requests/responses, adds global model attributes. Use ResponseBodyAdvice for response wrapping, RequestBodyAdvice for request preprocessing. Order with @Order when multiple advice classes exist."

---

# Question 53: What is global exception handling?

**1. Definition:** Global exception handling centralizes error handling logic in `@ControllerAdvice` classes, ensuring consistent error responses and reducing duplicate error handling code.

**2. Why it matters:** Consistent error responses improve API usability, simplify client integration, and make debugging easier. Centralized handling ensures security-sensitive exceptions don't leak information.

**3. Internal working:** Exception resolution order:

1. `@ExceptionHandler` in the controller
2. `@ExceptionHandler` in `@ControllerAdvice`
3. `DefaultHandlerExceptionResolver` (Spring defaults)
4. Response error handler

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex,
            HttpHeaders headers,
            HttpStatusCode status,
            WebRequest request) {
        // Override Spring's default validation handling
    }

    @ExceptionHandler(DataAccessException.class)
    public ResponseEntity<ErrorResponse> handleDataAccess(DataAccessException ex) {
        log.error("Database error", ex);
        return ResponseEntity.status(SERVICE_UNAVAILABLE)
            .body(ErrorResponse.of("DATABASE_ERROR", "Service temporarily unavailable"));
    }
}
```

**4. Real-world example:** Production-ready exception handling:

```
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ProblemDetail> handleAll(Exception ex, WebRequest request) {
        String traceId = MDC.get("traceId");
        log.error("Unhandled exception [traceId={}]", traceId, ex);

        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.INTERNAL_SERVER_ERROR,
            "An unexpected error occurred"
        );
        problem.setProperty("traceId", traceId);
        problem.setInstance(URI.create(request.getDescription(false)));

        return ResponseEntity.status(INTERNAL_SERVER_ERROR).body(problem);
    }
}
```

**5. Pitfalls:**

- Catching and swallowing exceptions without logging
- Returning internal exception messages to clients
- Not including correlation IDs in error responses
- Forgetting to handle async exceptions differently

**6. Follow-up questions:**

- How do you handle exceptions in @Async methods?
- What's ProblemDetail in Spring 6?

**7. Senior summary:** "Global exception handling via @ControllerAdvice centralizes error handling, ensuring consistent responses and proper logging. Include trace IDs for debugging, never expose internal details. I use ProblemDetail (RFC 7807) for standard error format in modern APIs."

---

# Question 54: What is ResponseEntity?

**1. Definition:** `ResponseEntity<T>` represents the full HTTP response—status code, headers, and body—giving complete control over the response sent to clients.

**2. Why it matters:** Fine-grained response control is essential for proper REST APIs—setting location headers, cache controls, custom status codes, and conditional responses.

**3. Internal working:**

```
@GetMapping("/orders/{id}")
public ResponseEntity<Order> getOrder(@PathVariable Long id) {
    return orderService.findById(id)
        .map(order -> ResponseEntity.ok()
            .header("X-Custom-Header", "value")
            .cacheControl(CacheControl.maxAge(1, TimeUnit.HOURS))
            .lastModified(order.getModifiedAt())
            .body(order))
        .orElse(ResponseEntity.notFound().build());
}

@PostMapping("/orders")
public ResponseEntity<Order> createOrder(@RequestBody @Valid OrderRequest request) {
    Order created = orderService.create(request);
    return ResponseEntity
        .created(URI.create("/orders/" + created.getId()))
        .body(created);
}
```

Builder methods:

- `ok()`, `created(URI)`, `accepted()`, `noContent()`
- `badRequest()`, `notFound()`, `status(HttpStatus)`
- `headers()`, `contentType()`, `body()`

**4. Real-world example:** Conditional requests for efficiency:

```
@GetMapping("/products/{id}")
public ResponseEntity<Product> getProduct(
        @PathVariable Long id,
        @RequestHeader(value = "If-None-Match", required = false) String ifNoneMatch) {

    Product product = productService.findById(id);
    String etag = "\"" + product.getVersion() + "\"";

    if (etag.equals(ifNoneMatch)) {
        return ResponseEntity.status(NOT_MODIFIED)
            .eTag(etag)
            .build();
    }

    return ResponseEntity.ok()
        .eTag(etag)
        .cacheControl(CacheControl.maxAge(1, TimeUnit.HOURS))
        .body(product);
}
```

**5. Pitfalls:**

- Overusing ResponseEntity when simple return types suffice
- Forgetting to include body in non-204 responses
- Not using proper status codes (201 for create, 204 for no content)

**6. Follow-up questions:**

- When would you return the POJO directly vs ResponseEntity?
- How do you handle conditional GETs with ETags?

**7. Senior summary:** "ResponseEntity provides complete HTTP response control—status, headers, body. Use for proper REST semantics: 201 Created with Location, 304 Not Modified, cache headers. For simple 200 OK responses, returning the object directly is cleaner."

---

# Question 55: How do you return proper HTTP status codes?

**1. Definition:** Proper status codes are returned via `ResponseEntity`, `@ResponseStatus` annotation, or exception handlers, following REST semantics for each operation type.

**2. Why it matters:** Status codes are the primary communication mechanism in REST. Clients rely on them for control flow—retry logic, error handling, cache behavior.

**3. Internal working:**

```
// Method 1: ResponseEntity
@PostMapping("/users")
public ResponseEntity<User> create(@RequestBody UserRequest request) {
    User user = userService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED)
        .location(URI.create("/users/" + user.getId()))
        .body(user);
}

// Method 2: @ResponseStatus on method
@DeleteMapping("/users/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
    userService.delete(id);
}

// Method 3: @ResponseStatus on exception
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException { }
```

Common status codes: | Code | Meaning | Use Case | |------|---------|----------| | 200 | OK | Successful GET, PUT | | 201 | Created | Successful POST | | 204 | No Content | Successful DELETE | | 400 | Bad Request | Validation failure | | 401 | Unauthorized | Not authenticated | | 403 | Forbidden | Not authorized | | 404 | Not Found | Resource doesn't exist | | 409 | Conflict | State conflict | | 422 | Unprocessable | Business rule failure | | 500 | Internal Error | Unexpected error |

**4. Real-world example:** Comprehensive status code handling:

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<Order> get(@PathVariable Long id) {
        return orderService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());  // 404
    }

    @PostMapping
    public ResponseEntity<Order> create(@RequestBody @Valid OrderRequest req) {
        Order order = orderService.create(req);
        return ResponseEntity.created(
            URI.create("/api/orders/" + order.getId()))
            .body(order);  // 201
    }

    @PostMapping("/{id}/cancel")
    public ResponseEntity<?> cancel(@PathVariable Long id) {
        try {
            orderService.cancel(id);
            return ResponseEntity.noContent().build();  // 204
        } catch (InvalidStateException e) {
            return ResponseEntity.status(HttpStatus.CONFLICT)  // 409
                .body(new ErrorResponse("INVALID_STATE", e.getMessage()));
        }
    }
}
```

**5. Pitfalls:**

- Using 200 for everything (lazy but wrong)
- 500 for client errors (should be 4xx)
- Not distinguishing 401 vs 403
- Missing Location header on 201

**6. Follow-up questions:**

- When would you use 422 vs 400?
- How do you handle 202 Accepted for async operations?

**7. Senior summary:** "Use ResponseEntity for explicit status control, @ResponseStatus for simple cases. Follow REST semantics: 201 for create with Location, 204 for delete, 4xx for client errors, 5xx for server errors. Proper status codes enable client retry logic and caching."

---

# Question 56: Difference between PUT and PATCH?

**1. Definition:** PUT replaces the entire resource with the request payload. PATCH applies partial modifications to a resource, updating only specified fields.

**2. Why it matters:** Choosing correctly affects bandwidth, idempotency, and conflict resolution. PUT is simpler but wasteful; PATCH is efficient but requires careful implementation.

**3. Internal working:**

```
// PUT - full replacement (idempotent)
@PutMapping("/users/{id}")
public User updateUser(@PathVariable Long id, @RequestBody @Valid User user) {
    user.setId(id);
    return userService.replace(user);  // Replace entire entity
}

// PATCH - partial update
@PatchMapping("/users/{id}")
public User patchUser(@PathVariable Long id, @RequestBody Map<String, Object> updates) {
    return userService.patch(id, updates);
}
```

| Aspect | PUT | PATCH |
|---|---|---|
| Idempotent | Yes | Not guaranteed |
| Body | Complete resource | Partial changes |
| Missing fields | Set to null/default | Unchanged |
| Bandwidth | Higher | Lower |

**4. Real-world example:** Implementing PATCH properly:

```
@PatchMapping("/users/{id}")
public ResponseEntity<User> patchUser(
        @PathVariable Long id,
        @RequestBody JsonMergePatch patch) {

    User user = userService.findById(id);
    User patched = applyPatch(patch, user);
    User saved = userService.save(patched);

    return ResponseEntity.ok(saved);
}

private User applyPatch(JsonMergePatch patch, User user) {
    ObjectMapper mapper = new ObjectMapper();
    JsonNode patched = patch.apply(mapper.valueToTree(user));
    return mapper.treeToValue(patched, User.class);
}
```

JSON Merge Patch (RFC 7396):

```
{
  "email": "new@email.com",
  "phone": null
}
```

Sets email, removes phone, leaves other fields unchanged.

**5. Pitfalls:**

- Using PUT when PATCH is appropriate (wasting bandwidth)
- PATCH without null handling (ambiguity)
- Not validating partial updates properly
- Assuming PATCH is idempotent (depends on implementation)

**6. Follow-up questions:**

- How do you handle null in PATCH—remove field or ignore?
- What's JSON Patch vs JSON Merge Patch?

**7. Senior summary:** "PUT replaces the entire resource, PATCH updates specific fields. Use JSON Merge Patch (RFC 7396) for simple PATCH operations. PUT is idempotent, PATCH may not be. I use PATCH for large resources where sending the full object is wasteful."

---

# Question 57: How does file upload work in Spring Boot?

**1. Definition:** File uploads use `multipart/form-data` content type, handled via `MultipartFile` parameter or `@RequestPart`, with configurable size limits and storage options.

**2. Why it matters:** File handling affects memory usage, security, and user experience. Proper implementation prevents OOM errors and ensures secure file processing.

**3. Internal working:**

```
@PostMapping(value = "/upload", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<FileResponse> uploadFile(
        @RequestParam("file") MultipartFile file,
        @RequestParam("metadata") String metadata) {

    if (file.isEmpty()) {
        throw new BadRequestException("File is empty");
    }

    String filename = StringUtils.cleanPath(file.getOriginalFilename());
    Path targetPath = uploadDir.resolve(filename);
    Files.copy(file.getInputStream(), targetPath, REPLACE_EXISTING);

    return ResponseEntity.ok(new FileResponse(filename, file.getSize()));
}
```

Configuration:

```
spring:
  servlet:
    multipart:
      enabled: true
      max-file-size: 10MB
      max-request-size: 50MB
      file-size-threshold: 2KB  # Threshold before writing to disk
      location: /tmp/uploads    # Temp storage location
```

**4. Real-world example:** Production file upload with validation:

```
@PostMapping("/documents")
public ResponseEntity<Document> uploadDocument(
        @RequestPart("file") MultipartFile file,
        @RequestPart("metadata") @Valid DocumentMetadata metadata) {

    // Validate file type
    String contentType = file.getContentType();
    if (!ALLOWED_TYPES.contains(contentType)) {
        throw new UnsupportedMediaTypeException("File type not allowed");
    }

    // Validate file size
    if (file.getSize() > MAX_FILE_SIZE) {
        throw new FileTooLargeException("Max size is " + MAX_FILE_SIZE);
    }

    // Scan for viruses (production requirement)
    virusScanner.scan(file.getInputStream());

    // Store in cloud storage
    String url = cloudStorage.upload(file, metadata);

    return ResponseEntity.created(URI.create(url))
        .body(new Document(metadata, url));
}
```

**5. Pitfalls:**

- Not limiting file size (OOM, disk exhaustion)
- Trusting client-provided filename (path traversal attack)
- Storing files in application directory
- Not scanning for malware

**6. Follow-up questions:**

- How do you handle large file uploads without memory issues?

- How do you implement resumable uploads?

**7. Senior summary:** "File uploads use MultipartFile with configurable size limits. Always validate file type, size, and sanitize filenames. For large files, stream to storage instead of loading into memory. In production, scan for malware and store in cloud storage, not local filesystem."

---

# Question 58: How multipart request handling works?

**1. Definition:** Multipart requests split HTTP body into multiple parts, each with its own content type, enabling mixed content uploads (files + JSON) in a single request.

**2. Why it matters:** Understanding multipart handling enables proper API design for complex uploads and debugging upload issues.

**3. Internal working:** Multipart structure:

```
POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

------WebKitFormBoundary
Content-Disposition: form-data; name="file"; filename="doc.pdf"
Content-Type: application/pdf

[binary data]
------WebKitFormBoundary
Content-Disposition: form-data; name="metadata"
Content-Type: application/json

{"title": "My Document"}
------WebKitFormBoundary--
```

Spring handling:

1. `MultipartResolver` parses request
2. Parts stored as `MultipartFile` or temp files
3. `@RequestPart` binds parts to parameters
4. After request, temp files cleaned up

```
@PostMapping("/products")
public Product createProduct(
        @RequestPart("image") MultipartFile image,
        @RequestPart("product") @Valid ProductRequest product) {
    // image is file part
    // product is JSON part deserialized
}
```

**4. Real-world example:** Mixed upload with multiple files:

```
@PostMapping("/listings")
public Listing createListing(
        @RequestPart("photos") List<MultipartFile> photos,
        @RequestPart("details") @Valid ListingDetails details,
        @RequestPart(value = "attachments", required = false) List<MultipartFile> attachments) {

    // Validate total upload size
    long totalSize = Stream.concat(photos.stream(),
            Optional.ofNullable(attachments).stream().flatMap(List::stream))
        .mapToLong(MultipartFile::getSize)
        .sum();

    if (totalSize > MAX_TOTAL_SIZE) {
        throw new RequestTooLargeException();
    }

    List<String> photoUrls = photos.stream()
        .map(storage::upload)
        .toList();

    return listingService.create(details, photoUrls);
}
```

**5. Pitfalls:**

- @RequestParam for JSON parts (doesn't deserialize)
- Forgetting temp file cleanup on errors
- Not handling partial upload failures
- Content-Type mismatch between part and expected type

**6. Follow-up questions:**

- How do you send JSON alongside a file in multipart?
- What's the difference between @RequestParam and @RequestPart?

**7. Senior summary:** "Multipart requests separate parts by boundary, each with its own Content-Type. Use @RequestPart for typed parts (files or JSON), @RequestParam for simple form fields. Spring's MultipartResolver handles parsing and temp file management. Clean up properly on errors."

---

# Question 59: How pagination works in Spring Boot?

**1. Definition:** Spring Data provides `Pageable` parameter binding and `Page<T>` results, automatically handling page number, size, and sorting for database queries.

**2. Why it matters:** Pagination is essential for performance and UX with large datasets. Proper implementation prevents memory issues and provides consistent API behavior.

**3. Internal working:**

```
@GetMapping("/products")
public Page<Product> getProducts(
        @PageableDefault(size = 20, sort = "name") Pageable pageable) {
    return productRepository.findAll(pageable);
}

// Request: GET /products?page=0&size=10&sort=price,desc&sort=name,asc
```

Configuration:

```yaml
spring:
  data:
    web:
      pageable:
        default-page-size: 20
        max-page-size: 100
        one-indexed-parameters: false  # page starts at 0
        page-parameter: page
        size-parameter: size
      sort:
        sort-parameter: sort
```

Page response includes:

- `content`: List of items
- `totalElements`: Total count
- `totalPages`: Number of pages
- `number`: Current page (0-indexed)
- `size`: Page size

**4. Real-world example:** Custom pagination response:

```java
@GetMapping("/orders")
public ResponseEntity<PagedResponse<OrderDto>> getOrders(
        @RequestParam(required = false) String status,
        Pageable pageable) {

    Page<Order> page = orderRepository.findByStatus(status, pageable);

    PagedResponse<OrderDto> response = new PagedResponse<>(
        page.getContent().stream().map(OrderDto::from).toList(),
        page.getNumber(),
        page.getSize(),
        page.getTotalElements(),
        page.getTotalPages()
    );

    return ResponseEntity.ok()
        .header("X-Total-Count", String.valueOf(page.getTotalElements()))
        .body(response);
}

public record PagedResponse<T>(
    List<T> data,
    int page,
    int size,
    long total,
    int totalPages
) {}
```

**5. Pitfalls:**

- Not limiting max page size (memory/performance issues)
- COUNT queries on large tables (use keyset pagination)
- Offset pagination performance degradation at high page numbers
- Exposing internal entity directly (use DTOs)

**6. Follow-up questions:**

- What's the difference between offset and keyset pagination?
- How do you handle pagination with complex joins?

**7. Senior summary:** "Spring Data's Pageable provides automatic page/size/sort binding. Configure max-page-size to prevent abuse. For large datasets, consider keyset pagination over offset due to performance. Always use DTOs in API responses, not entities."

---

## Question 60: How sorting works in Spring Boot?

**1. Definition:** Sorting is handled via `Sort` parameter or as part of `Pageable`, supporting multiple fields with configurable direction (ASC/DESC).

**2. Why it matters:** Proper sorting implementation affects query performance and user experience. Understanding the mechanisms prevents SQL injection and inefficient queries.

**3. Internal working:**

```
// Via Pageable
@GetMapping("/products")
public Page<Product> getProducts(Pageable pageable) {
    // Request: ?sort=price,desc&sort=name,asc
    return productRepository.findAll(pageable);
}

// Direct Sort parameter
@GetMapping("/products")
public List<Product> getProducts(Sort sort) {
    return productRepository.findAll(sort);
}

// Programmatic Sort
Sort sort = Sort.by(
    Sort.Order.desc("price"),
    Sort.Order.asc("name").ignoreCase()
);
```

Sort safety:

```
@GetMapping("/products")
public Page<Product> getProducts(
        @SortDefault.SortDefaults({
            @SortDefault(sort = "price", direction = DESC),
            @SortDefault(sort = "name", direction = ASC)
        }) Pageable pageable) {

    // Whitelist allowed sort fields
    Set<String> allowedFields = Set.of("price", "name", "createdAt");
    pageable.getSort().forEach(order -> {
        if (!allowedFields.contains(order.getProperty())) {
            throw new BadRequestException("Invalid sort field: " + order.getProperty());
        }
    });

    return productRepository.findAll(pageable);
}
```

**4. Real-world example:** Complex sorting with null handling:

```
@GetMapping("/users")
public Page<User> getUsers(
        @RequestParam(required = false) String sortBy,
        @RequestParam(defaultValue = "ASC") Sort.Direction direction,
        Pageable pageable) {

    Sort sort = sortBy != null
        ? Sort.by(direction, sortBy).and(Sort.by("id"))  // Stable sort
        : Sort.by("createdAt").descending();

    Pageable sortedPageable = PageRequest.of(
        pageable.getPageNumber(),
        pageable.getPageSize(),
        sort
    );

    return userRepository.findAll(sortedPageable);
}
```

**5. Pitfalls:**

- Allowing arbitrary sort fields (SQL injection risk)
- Sorting on non-indexed columns (performance)
- Unstable sort causing inconsistent pagination
- Case-sensitive sorting when case-insensitive expected

**6. Follow-up questions:**

- How do you ensure stable sorting for pagination?
- How do you sort by nested properties?

**7. Senior summary:** "Sort via Pageable URL parameters or explicit Sort objects. Whitelist allowed sort fields for security. Add a secondary sort on unique field (ID) for stable pagination. Ensure sorted columns are indexed for performance."

---

# Question 61: How to handle CORS in Spring Boot?

**1. Definition:** CORS (Cross-Origin Resource Sharing) is configured via `@CrossOrigin`, `WebMvcConfigurer`, or `CorsFilter` to allow browsers to make cross-origin requests to your API.

**2. Why it matters:** APIs consumed by browser-based clients need proper CORS configuration. Misconfiguration either blocks legitimate requests or creates security vulnerabilities.

**3. Internal working:**

```
// Method 1: @CrossOrigin on controller
@RestController
@CrossOrigin(origins = "https://example.com")
public class ApiController {
    @CrossOrigin(origins = "https://other.com")  // Override
    @GetMapping("/data")
    public Data getData() { }
}

// Method 2: Global configuration
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("https://example.com", "https://app.example.com")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true)
            .maxAge(3600);
    }
}

// Method 3: CorsFilter (for Spring Security)
@Bean
public CorsFilter corsFilter() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("https://example.com"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return new CorsFilter(source);
}
```

**4. Real-world example:** Production CORS with Spring Security:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()));
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();

        // Environment-specific origins
        config.setAllowedOrigins(List.of(
            "https://app.example.com",
            "https://admin.example.com"
        ));

        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Request-Id"));
        config.setExposedHeaders(List.of("X-Total-Count", "X-Request-Id"));
        config.setAllowCredentials(true);
        config.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return source;
    }
}
```

**5. Pitfalls:**

- Using `allowedOrigins("*")` with `allowCredentials(true)` (forbidden by browsers)
- CORS config not applied due to Spring Security filter order
- Forgetting OPTIONS method for preflight requests
- Not exposing custom headers the client needs

**6. Follow-up questions:**

- What's a preflight request and when does it occur?
- How do you handle CORS with Spring Security?

**7. Senior summary:** ""Configure CORS via WebMvcConfigurer or CorsFilter (for Spring Security). Never use wildcard origins with credentials. Explicitly list allowed origins, methods, and headers. Configure with Spring Security using cors() DSL to ensure proper filter ordering."

---

# DATA + JPA QUESTIONS (62-81) - VERY IMPORTANT

---

## Question 62: Difference between JPA and Hibernate?

**1. Definition:** JPA (Jakarta Persistence API) is a specification defining ORM standards. Hibernate is an implementation of JPA that provides additional features beyond the spec.

**2. Why it matters:** Understanding this distinction enables portable code (using JPA APIs) while leveraging Hibernate-specific features when needed.

**3. Internal working:**

```
JPA Specification:
- EntityManager
- @Entity, @Table, @Column
- JPQL query language
- Criteria API
- Lifecycle annotations

Hibernate Implementation:
- Session (extends EntityManager)
- SessionFactory
- HQL (superset of JPQL)
- Hibernate-specific annotations (@BatchSize, @Where)
- Second-level caching
- Schema generation
```

Spring Data JPA:

```
// Uses JPA API through Hibernate
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    private String name;
}

// Spring Data repository (uses JPA EntityManager internally)
public interface UserRepository extends JpaRepository<User, Long> { }
```

**4. Real-world example:** Portable code with JPA, Hibernate extensions when needed:

```
// Portable JPA
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    private Customer customer;
}

// Hibernate-specific optimization
@Entity
@BatchSize(size = 25)  // Hibernate-specific
@Where(clause = "deleted = false")  // Hibernate-specific
public class Product {
    // Batch fetching and soft delete filter
}
```

**5. Pitfalls:**

- Coupling to Hibernate APIs unnecessarily
- Assuming JPA annotations work the same in other providers
- Not understanding when Hibernate extensions are beneficial

**6. Follow-up questions:**

- What Hibernate features aren't in JPA spec?
- How would you switch from Hibernate to EclipseLink?

**7. Senior summary:** "JPA is the specification, Hibernate is the implementation. Use JPA APIs for portability. Use Hibernate-specific features (@BatchSize, @Fetch, @Where) when needed for optimization. Spring Data JPA abstracts both, using EntityManager under the hood."

---

# Question 63: What is Spring Data JPA?

**1. Definition:** Spring Data JPA is a Spring module that simplifies JPA-based data access by providing repository abstractions, derived queries, and query methods that eliminate boilerplate code.

**2. Why it matters:** It dramatically reduces data access code—no DAO implementations needed for standard operations. Focus on domain logic, not CRUD boilerplate.

**3. Internal working:**

```
// Interface only - Spring generates implementation
public interface UserRepository extends JpaRepository<User, Long> {

    // Derived query - parsed from method name
    List<User> findByEmailAndStatus(String email, Status status);

    // Custom JPQL
    @Query("SELECT u FROM User u WHERE u.createdAt > :date")
    List<User> findRecentUsers(@Param("date") LocalDateTime date);

    // Native SQL
    @Query(value = "SELECT * FROM users WHERE email LIKE %?1%", nativeQuery = true)
    List<User> searchByEmail(String pattern);
}
```

Spring Data creates proxy at runtime:

1. `@EnableJpaRepositories` triggers scanning
2. `JpaRepositoryFactoryBean` creates repository implementations
3. `SimpleJpaRepository` provides base CRUD operations
4. Query methods parsed and converted to JPQL/Criteria

**4. Real-world example:** Complex repository with specifications:

```
public interface OrderRepository extends JpaRepository<Order, Long>,
        JpaSpecificationExecutor<Order> {

    // Derived query with pagination
    Page<Order> findByCustomerIdAndStatusIn(
        Long customerId, List<Status> statuses, Pageable pageable);

    // Modifying query
    @Modifying
    @Query("UPDATE Order o SET o.status = :status WHERE o.id IN :ids")
    int updateStatusBatch(@Param("ids") List<Long> ids, @Param("status") Status status);

    // Entity graph for fetch optimization
    @EntityGraph(attributePaths = {"customer", "items"})
    Optional<Order> findWithDetailsById(Long id);
}
```

**5. Pitfalls:**

- Over-relying on derived queries for complex logic
- Not understanding the generated SQL
- Missing @Modifying on update/delete queries
- Forgetting @Transactional for modifying operations

**6. Follow-up questions:**

- How does query derivation work?
- What's the difference between findBy and getBy?

**7. Senior summary:** "Spring Data JPA generates repository implementations at runtime, eliminating boilerplate. Use derived queries for simple cases, @Query for complex ones. JpaSpecificationExecutor enables dynamic queries. Always verify generated SQL for performance."

---

# Question 64: CrudRepository vs JpaRepository?

**1. Definition:** `CrudRepository` provides basic CRUD operations. `JpaRepository` extends it with JPA-specific features: batching, flushing, pagination, and query by example.

**2. Why it matters:** Choosing the right interface affects available operations. JpaRepository is typically preferred for JPA applications; CrudRepository for generic Spring Data usage.

**3. Internal working:**

```
Repository (marker)
    └── CrudRepository (basic CRUD)
            └── PagingAndSortingRepository (pagination)
                    └── JpaRepository (JPA-specific)

CrudRepository<T, ID>:
- save(entity)
- saveAll(entities)
- findById(id)
- existsById(id)
- findAll()
- count()
- delete(entity)
- deleteById(id)

JpaRepository<T, ID> adds:
- flush()
- saveAndFlush(entity)
- deleteAllInBatch()
- deleteInBatch(entities)
- findAll(Sort)
- findAll(Pageable)
- findAll(Example)
- getOne(id) / getById(id) / getReferenceById(id)
```

**4. Real-world example:**

```
// For batch operations, use JpaRepository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // saveAndFlush - immediate persist (needed before ID generation)
    default Product createAndGetId(Product product) {
        return saveAndFlush(product);  // ID available immediately
    }

    // deleteAllInBatch - single DELETE query
    default void bulkDelete(List<Long> ids) {
        deleteAllByIdInBatch(ids);  // Much faster than individual deletes
    }

    // getReferenceById - proxy without SELECT
    default void assignToCategory(Long productId, Long categoryId) {
        Product product = getReferenceById(productId);  // No SELECT
        Category category = categoryRepository.getReferenceById(categoryId);
        product.setCategory(category);
        // Single UPDATE, no SELECT queries
    }
}
```

**5. Pitfalls:**

- Using CrudRepository when batch operations are needed
- `getById/getReferenceById` throws on access, not on call
- Forgetting that deleteInBatch bypasses cascade/lifecycle callbacks

**6. Follow-up questions:**

- When would getReferenceById cause LazyInitializationException?
- What's the difference between delete() and deleteInBatch()?

**7. Senior summary:** "JpaRepository extends CrudRepository with JPA features: batching, flushing, pagination. Use deleteAllInBatch for bulk deletes (single query), getReferenceById for FK assignments without loading. Most JPA applications should use JpaRepository."

---

# Question 65: How transactions work internally?

**1. Definition:** Spring transactions use AOP proxies to intercept method calls, start transactions via `PlatformTransactionManager`, and commit/rollback based on method outcome.

**2. Why it matters:** Understanding transaction internals explains proxy limitations, propagation behavior, and isolation level effects—critical for avoiding data inconsistencies.

**3. Internal working:**

```
Method call → Proxy intercepts → TransactionInterceptor →
    → PlatformTransactionManager.getTransaction() →
    → (gets Connection from DataSource) →
    → setAutoCommit(false) →
    → Execute method →
    → commit() or rollback() →
    → Return connection to pool

Key components:
- @Transactional annotation
- TransactionInterceptor (AOP advice)
- PlatformTransactionManager (abstraction)
- DataSourceTransactionManager (JDBC)
- JpaTransactionManager (JPA)
```

Connection handling:

```
@Transactional
public void processOrder(Order order) {
    // Connection obtained, autocommit=false
    orderRepository.save(order);
    // Same connection reused
    inventoryService.reserve(order);
    // Same connection (propagation REQUIRED)
    // Method ends → commit
}
```

**4. Real-world example:** Transaction flow debugging:

```
@Service
@Slf4j
public class OrderService {

    @Transactional
    public void createOrder(OrderRequest request) {
        log.debug("TX active: {}", TransactionSynchronizationManager.isActualTransactionActive());
        log.debug("TX name: {}", TransactionSynchronizationManager.getCurrentTransactionName());

        // Both operations share the same transaction
        Order order = orderRepository.save(new Order(request));
        paymentService.charge(order);  // Same TX if REQUIRED propagation

        // If paymentService throws, both are rolled back
    }
}
```

**5. Pitfalls:**

- Calling @Transactional method from same class (proxy bypass)
- Catching exceptions, preventing rollback
- Long transactions holding connections
- Not understanding connection pooling impact

**6. Follow-up questions:**

- What happens when a @Transactional method calls another @Transactional method?
- How does transaction synchronization work with multiple resources?

**7. Senior summary:** "Transactions work via AOP proxies—TransactionInterceptor manages lifecycle through PlatformTransactionManager. Connection borrowed from pool, autocommit disabled, same connection used for all operations in transaction. Self-invocation bypasses proxy, breaking transaction handling."

---

# Question 66: What is @Transactional?

**1. Definition:** `@Transactional` is a declarative transaction management annotation that defines transaction boundaries, propagation, isolation, timeout, and rollback rules.

**2. Why it matters:** Proper transaction configuration ensures data consistency, proper isolation, and resource management. Misconfiguration leads to data corruption or performance issues.

**3. Internal working:**

```
@Transactional(
    propagation = Propagation.REQUIRED,  // Default
    isolation = Isolation.READ_COMMITTED, // Default varies by DB
    timeout = 30,                         // Seconds
    readOnly = false,                     // Default
    rollbackFor = Exception.class,        // Rollback on checked exceptions
    noRollbackFor = BusinessException.class
)
public void processPayment(Payment payment) {
    // All operations in single transaction
}
```

Placement rules:

```
@Service
@Transactional(readOnly = true)  // Class-level default
public class OrderService {

    public Order findById(Long id) {
        // Uses class-level: readOnly = true
        return orderRepository.findById(id).orElseThrow();
    }

    @Transactional  // Override: readOnly = false
    public Order create(OrderRequest request) {
        return orderRepository.save(new Order(request));
    }
}
```

**4. Real-world example:** Service layer transaction patterns:

```
@Service
@Transactional(readOnly = true)  // Safe default
public class AccountService {

    @Transactional(isolation = Isolation.SERIALIZABLE)
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findByIdWithLock(fromId);  // Pessimistic lock
        Account to = accountRepository.findByIdWithLock(toId);

        from.debit(amount);
        to.credit(amount);
        // Both saved on commit, or neither (atomicity)
    }

    @Transactional(timeout = 5, rollbackFor = Exception.class)
    public void processWithExternalCall(Long id) {
        // Timeout prevents long transactions
        accountRepository.process(id);
        externalService.notify();  // If this fails, rollback
    }
}
```

**5. Pitfalls:**

- Using on private methods (AOP doesn't work)
- Forgetting rollbackFor for checked exceptions
- readOnly=true with write operations (may silently fail)
- Self-invocation within the same class

**6. Follow-up questions:**

- What's the difference between class-level and method-level @Transactional?
- How do you handle transactions in async methods?

**7. Senior summary:** "@Transactional defines transaction boundaries with configurable propagation, isolation, timeout, and rollback rules. Apply at service layer, not repository. Use readOnly=true as default for queries. Remember: only works on public methods via proxy calls."

---

# Question 67: Transaction propagation types?

**1. Definition:** Propagation defines how transactions relate when a transactional method calls another transactional method—join existing, create new, or execute without transaction.

**2. Why it matters:** Propagation choice affects atomicity scope. Wrong propagation can cause unexpected commits/rollbacks or nested transaction complexity.

**3. Internal working:**

| Propagation | Existing TX | No TX | Use Case |
|---|---|---|---|
| REQUIRED | Join | Create | Default, most common |
| REQUIRES_NEW | Suspend, create new | Create | Audit logs, must commit independently |
| NESTED | Create savepoint | Create | Partial rollback |
| MANDATORY | Join | Throw exception | Must be called from TX |
| SUPPORTS | Join | Run without TX | Read methods |
| NOT_SUPPORTED | Suspend | Run without TX | Long-running reads |
| NEVER | Throw exception | Run without TX | Must not be in TX |

```
@Transactional(propagation = Propagation.REQUIRED)  // Default
public void processOrder(Order order) {
    orderRepository.save(order);  // Same TX

    auditService.log(order);  // See below
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(Order order) {
        // New TX - commits even if caller rolls back
        auditRepository.save(new AuditLog(order));
    }
}
```

**4. Real-world example:** Audit logging that must persist regardless of outcome:

```
@Service
public class PaymentService {

    @Transactional
    public void processPayment(Payment payment) {
        try {
            // Main transaction
            paymentRepository.save(payment);
            gatewayService.charge(payment);
        } catch (GatewayException e) {
            // Log audit BEFORE rollback
            auditService.logFailure(payment, e);  // REQUIRES_NEW
            throw e;  // Main TX rolls back, audit persists
        }
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logFailure(Payment payment, Exception error) {
        // Independent transaction - always commits
        auditRepository.save(new FailureLog(payment, error));
    }
}
```

**5. Pitfalls:**

- REQUIRES_NEW with long transactions (connection exhaustion)
- NESTED not supported by all transaction managers
- Assuming REQUIRES_NEW uses same connection (it doesn't)
- Savepoint rollback with NESTED not working with JPA/Hibernate

**6. Follow-up questions:**

- What happens to nested transaction if outer fails?
- How does REQUIRES_NEW affect connection pool?

**7. Senior summary:** "REQUIRED is the default—join or create. Use REQUIRES_NEW for operations that must commit independently (audit logs). NESTED creates savepoints for partial rollback. REQUIRES_NEW suspends outer TX and needs another connection—mind the pool."

---

# Question 68: Transaction isolation levels?

**1. Definition:** Isolation levels define how transactions are isolated from concurrent transactions, trading consistency guarantees against concurrency performance.

**2. Why it matters:** Wrong isolation causes data anomalies (dirty reads, phantom reads) or excessive locking (deadlocks, performance issues). Choose based on consistency requirements.

**3. Internal working:**

| Isolation | Dirty Read | Non-Repeatable Read | Phantom Read |
|---|---|---|---|
| READ_UNCOMMITTED | Possible | Possible | Possible |
| READ_COMMITTED | Prevented | Possible | Possible |
| REPEATABLE_READ | Prevented | Prevented | Possible |
| SERIALIZABLE | Prevented | Prevented | Prevented |

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public BigDecimal getAccountBalance(Long accountId) {
    // Highest isolation - full consistency
}

@Transactional(isolation = Isolation.READ_COMMITTED)
public List<Order> getOrders() {
    // Default for most DBs - good for reads
}
```

Database defaults:

- PostgreSQL: READ_COMMITTED
- MySQL InnoDB: REPEATABLE_READ
- Oracle: READ_COMMITTED
- SQL Server: READ_COMMITTED

**4. Real-world example:** Financial transfer requiring serializable:

```
@Service
public class AccountService {

    @Transactional(isolation = Isolation.SERIALIZABLE)
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        // SERIALIZABLE prevents phantom reads
        // No concurrent transfer can see intermediate state

        Account from = accountRepository.findById(fromId).orElseThrow();
        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException();
        }

        Account to = accountRepository.findById(toId).orElseThrow();
        from.debit(amount);
        to.credit(amount);
        // Both accounts saved atomically
    }

    @Transactional(isolation = Isolation.READ_COMMITTED)
    public List<Transaction> getTransactionHistory(Long accountId) {
        // Lower isolation is fine for read-only reporting
        return transactionRepository.findByAccountId(accountId);
    }
}
```

**5. Pitfalls:**

- Using SERIALIZABLE everywhere (major performance hit)
- Not considering database defaults
- Mixing isolation levels incorrectly
- Assuming application-level locking replaces database isolation

**6. Follow-up questions:**

- How does isolation level interact with optimistic locking?
- What's the performance impact of SERIALIZABLE?

**7. Senior summary:** "Isolation levels control visibility of concurrent changes. READ_COMMITTED is default for most DBs—prevents dirty reads. Use SERIALIZABLE for financial operations. Higher isolation = more locking = lower throughput. Consider optimistic locking as alternative."

---

# Question 69: Lazy vs Eager loading?

**1. Definition:** Lazy loading defers relationship fetching until accessed. Eager loading fetches relationships immediately with the parent entity.

**2. Why it matters:** Loading strategy dramatically affects performance. Eager causes over-fetching; lazy causes N+1 queries. Neither is universally correct—choose per use case.

**3. Internal working:**

```
@Entity
public class Order {
    @Id
    private Long id;

    @ManyToOne(fetch = FetchType.EAGER)  // Loaded with order
    private Customer customer;

    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY)  // Default
    private List<OrderItem> items;  // Proxy until accessed
}
```

JPA defaults:

- `@OneToMany`, `@ManyToMany`: LAZY
- `@ManyToOne`, `@OneToOne`: EAGER

Lazy loading mechanics:

```
Order order = orderRepository.findById(1L).get();
// SQL: SELECT * FROM orders WHERE id = 1

order.getItems().size();  // Triggers lazy load
// SQL: SELECT * FROM order_items WHERE order_id = 1
```

**4. Real-world example:** Strategic fetch configuration:

```
@Entity
public class Product {
    @Id
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)  // Override default EAGER
    private Category category;  // Don't always need this

    @OneToMany(mappedBy = "product", fetch = FetchType.LAZY)
    private List<Review> reviews;  // Definitely don't load by default
}

// When you need related data, use JOIN FETCH
public interface ProductRepository extends JpaRepository<Product, Long> {

    @Query("SELECT p FROM Product p JOIN FETCH p.category WHERE p.id = :id")
    Optional<Product> findByIdWithCategory(@Param("id") Long id);

    @EntityGraph(attributePaths = {"category", "reviews"})
    Optional<Product> findWithDetailsById(Long id);
}
```

**5. Pitfalls:**

- LazyInitializationException after session closes
- Eager loading causing Cartesian product
- N+1 queries with lazy loading in loops
- Using EAGER to "fix" LazyInitializationException (wrong solution)

**6. Follow-up questions:**

- How do you avoid LazyInitializationException?
- What's the Cartesian product problem with multiple eager collections?

**7. Senior summary:** "Always use LAZY as default—fetch when needed with JOIN FETCH or @EntityGraph. EAGER causes over-fetching and Cartesian products. Handle LazyInitializationException by fetching within transaction, not by switching to EAGER. Design queries around use cases."

---

# Question 70: What is N+1 problem?

**1. Definition:** N+1 is a performance anti-pattern where fetching N entities results in 1 query for parents plus N queries for their relationships, instead of a single join query.

**2. Why it matters:** N+1 is the most common JPA performance issue. With 100 entities, you get 101 queries instead of 1—devastating for performance.

**3. Internal working:**

```
// N+1 problem example
List<Order> orders = orderRepository.findAll();  // 1 query
for (Order order : orders) {
    // Each access triggers a new query = N queries
    log.info("Customer: {}", order.getCustomer().getName());
}
// Total: 1 + N queries!

// SQL:
// SELECT * FROM orders  -- 1 query
// SELECT * FROM customers WHERE id = 1  -- N queries
// SELECT * FROM customers WHERE id = 2
// SELECT * FROM customers WHERE id = 3
// ...
```

**4. Real-world example:** Solutions to N+1:

```
// Solution 1: JOIN FETCH in JPQL
@Query("SELECT o FROM Order o JOIN FETCH o.customer")
List<Order> findAllWithCustomers();
// Single query: SELECT o.*, c.* FROM orders o JOIN customers c

// Solution 2: @EntityGraph
@EntityGraph(attributePaths = {"customer", "items"})
List<Order> findAll();

// Solution 3: @BatchSize on entity
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)
    @BatchSize(size = 25)  // Hibernate fetches 25 at a time
    private Customer customer;
}

// Solution 4: Default batch fetch size
spring:
  jpa:
    properties:
      hibernate:
        default_batch_fetch_size: 25

// SQL with batch size:
// SELECT * FROM orders
// SELECT * FROM customers WHERE id IN (1, 2, 3, ... 25)
// Instead of 25 separate queries
```

**5. Pitfalls:**

- JOIN FETCH with pagination (HHH000104 warning)
- Multiple JOIN FETCHes causing Cartesian product
- Not monitoring actual SQL in development
- Using @BatchSize as only solution (JOIN FETCH is still better)

**6. Follow-up questions:**

- How do you detect N+1 queries in your application?
- What's the Cartesian product problem with multiple JOIN FETCHes?

**7. Senior summary:** "N+1 is 1 query for parents + N queries for relationships. Fix with JOIN FETCH for specific queries, @EntityGraph for repository methods, or global batch-fetch-size. Monitor SQL in development using spring.jpa.show-sql or P6Spy. JOIN FETCH is most efficient for known use cases."

---

# Question 71: How to solve N+1 problem?

**1. Definition:** Solutions include JOIN FETCH in JPQL, @EntityGraph annotations, Hibernate @BatchSize, or global batch fetch configuration—each with different trade-offs.

**2. Why it matters:** Choosing the right solution depends on use case—ad-hoc queries, consistent patterns, or global optimization. Wrong choice leads to over-fetching or complexity.

**3. Internal working:**

```
// 1. JOIN FETCH - explicit, optimal for specific query
@Query("SELECT o FROM Order o " +
       "JOIN FETCH o.customer " +
       "JOIN FETCH o.items " +
       "WHERE o.status = :status")
List<Order> findByStatusWithDetails(@Param("status") Status status);

// 2. @EntityGraph - reusable, declarative
@NamedEntityGraph(
    name = "Order.withDetails",
    attributeNodes = {
        @NamedAttributeNode("customer"),
        @NamedAttributeNode(value = "items", subgraph = "items-subgraph")
    },
    subgraphs = @NamedSubgraph(
        name = "items-subgraph",
        attributeNodes = @NamedAttributeNode("product")
    )
)
@Entity
public class Order { }

@EntityGraph("Order.withDetails")
List<Order> findByStatus(Status status);

// 3. @BatchSize - Hibernate specific, good default
@Entity
@BatchSize(size = 25)
public class Customer { }

// 4. Global batch fetch
spring.jpa.properties.hibernate.default_batch_fetch_size=25
```

**4. Real-world example:** Different solutions for different scenarios:

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // List view - only need customer name
    @Query("SELECT o FROM Order o JOIN FETCH o.customer")
    List<Order> findAllForListView();

    // Detail view - need everything
    @EntityGraph(attributePaths = {"customer", "items", "items.product", "payments"})
    Optional<Order> findDetailedById(Long id);

    // Report - specific fields only (DTO projection, no entities)
    @Query("SELECT new com.app.OrderSummary(o.id, o.total, c.name) " +
           "FROM Order o JOIN o.customer c")
    List<OrderSummary> findSummaries();  // No lazy loading issues!
}
```

**5. Pitfalls:**

- JOIN FETCH on collections with pagination (loads all, paginates in memory)
- Multiple collection JOIN FETCHes (Cartesian product)
- @BatchSize requiring hibernate-specific dependency
- DTO projection requiring constructor matching

**6. Follow-up questions:**

- How do you handle JOIN FETCH with pagination?
- When would you use DTO projection instead of entity fetching?

**7. Senior summary:** "Use JOIN FETCH for specific queries, @EntityGraph for repository methods, batch-fetch-size for global optimization, DTO projection for read-only views. Avoid JOIN FETCH with pagination on collections. Multiple collections require separate queries or batch fetching."

---

# Question 72: JPQL vs Native Query?

**1. Definition:** JPQL (Jakarta Persistence Query Language) is object-oriented, portable, works with entity mappings. Native queries are raw SQL, database-specific, needed for DB-specific features.

**2. Why it matters:** JPQL provides type safety and portability. Native queries offer full SQL power. Choosing correctly affects maintainability and performance.

**3. Internal working:**

```
// JPQL - entity-oriented
@Query("SELECT o FROM Order o WHERE o.customer.name = :name")
List<Order> findByCustomerName(@Param("name") String name);

// Native - SQL
@Query(value = "SELECT * FROM orders WHERE customer_id IN " +
       "(SELECT id FROM customers WHERE name = ?1)",
       nativeQuery = true)
List<Order> findByCustomerNameNative(String name);

// Native for DB-specific features
@Query(value = "SELECT * FROM orders WHERE data @> :jsonPath::jsonb",
       nativeQuery = true)
List<Order> findByJsonPath(@Param("jsonPath") String jsonPath);  // PostgreSQL jsonb
```

| Aspect | JPQL | Native Query |
|---|---|---|
| Syntax | Entity/property names | Table/column names |
| Portability | Yes | No |
| Type safety | Higher | Lower |
| DB features | Limited | Full |
| Performance | Good | Can be better |

**4. Real-world example:** When to use each:

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    // JPQL - standard queries, type-safe
    @Query("SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max")
    List<Product> findByPriceRange(
        @Param("min") BigDecimal min,
        @Param("max") BigDecimal max);

    // JPQL - JOIN FETCH works with entity model
    @Query("SELECT DISTINCT p FROM Product p LEFT JOIN FETCH p.reviews " +
           "WHERE p.category.name = :category")
    List<Product> findByCategoryWithReviews(@Param("category") String category);

    // Native - complex SQL, window functions
    @Query(value = """
        SELECT * FROM (
            SELECT *, RANK() OVER (PARTITION BY category_id ORDER BY sales DESC) as rank
            FROM products
        ) ranked WHERE rank <= 10
        """, nativeQuery = true)
    List<Product> findTop10ByCategory();

    // Native - full-text search (PostgreSQL)
    @Query(value = "SELECT * FROM products " +
           "WHERE to_tsvector('english', name || ' ' || description) " +
           "@@ plainto_tsquery('english', :search)",
           nativeQuery = true)
    List<Product> fullTextSearch(@Param("search") String search);
}
```

**5. Pitfalls:**

- Native queries not mapping to entity correctly
- Forgetting nativeQuery = true
- Using native for simple queries (unnecessary complexity)
- JPQL not supporting all SQL features (window functions)

**6. Follow-up questions:**

- How do you map native query results to DTOs?
- Can you use native queries with pagination?

**7. Senior summary:** "JPQL for entity-oriented, portable queries—use for most cases. Native SQL for DB-specific features (JSON, full-text, window functions). Native queries still map to entities but using column names. Prefer JPQL unless you need SQL-specific capabilities."

---

# Question 73: How @Query works?

**1. Definition:** `@Query` defines custom JPQL or native SQL queries on repository methods, overriding the default query derivation with explicit query logic.

**2. Why it matters:** Complex queries can't be expressed as derived query methods. @Query provides explicit control while maintaining repository abstraction.

**3. Internal working:**

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Positional parameters
    @Query("SELECT o FROM Order o WHERE o.status = ?1 AND o.total > ?2")
    List<Order> findByStatusAndMinTotal(Status status, BigDecimal minTotal);

    // Named parameters
    @Query("SELECT o FROM Order o WHERE o.status = :status AND o.createdAt > :date")
    List<Order> findRecentByStatus(
        @Param("status") Status status,
        @Param("date") LocalDateTime date);

    // Modifying queries (UPDATE, DELETE)
    @Modifying
    @Query("UPDATE Order o SET o.status = :newStatus WHERE o.status = :oldStatus")
    int updateStatus(@Param("oldStatus") Status old, @Param("newStatus") Status newStatus);

    // With pagination
    @Query("SELECT o FROM Order o WHERE o.customer.id = :customerId")
    Page<Order> findByCustomerId(@Param("customerId") Long customerId, Pageable pageable);

    // DTO projection
    @Query("SELECT new com.app.OrderSummary(o.id, o.total, o.status) FROM Order o")
    List<OrderSummary> findAllSummaries();
}
```

**4. Real-world example:** Advanced query patterns:

```
public interface UserRepository extends JpaRepository<User, Long> {

    // SpEL expression
    @Query("SELECT u FROM #{#entityName} u WHERE u.email = :email")
    Optional<User> findByEmail(@Param("email") String email);

    // Collection parameter
    @Query("SELECT u FROM User u WHERE u.id IN :ids")
    List<User> findByIds(@Param("ids") Collection<Long> ids);

    // Complex with subquery
    @Query("SELECT u FROM User u WHERE u.orderCount > " +
           "(SELECT AVG(u2.orderCount) FROM User u2)")
    List<User> findAboveAverageOrderCount();

    // Native with result set mapping
    @Query(value = "SELECT u.id, u.name, COUNT(o.id) as order_count " +
                   "FROM users u LEFT JOIN orders o ON u.id = o.user_id " +
                   "GROUP BY u.id, u.name",
           nativeQuery = true)
    List<Object[]> findUsersWithOrderCounts();  // Manual mapping needed
}
```

**5. Pitfalls:**

- Forgetting @Modifying on UPDATE/DELETE queries
- Missing @Transactional for modifying queries
- SpEL expressions not working without proper setup
- Native query result mapping complexity

**6. Follow-up questions:**

- How do you use dynamic queries with @Query?
- What's the difference between positional and named parameters?

**7. Senior summary:** "@Query enables explicit JPQL/SQL on repository methods. Use named parameters for readability. @Modifying required for UPDATE/DELETE. Combine with Pageable for pagination. For dynamic queries, consider Specifications or Querydsl instead."

---

# Question 74: Pagination with JPA?

**1. Definition:** JPA pagination uses `Pageable` parameter and `Page<T>` return type, generating LIMIT/OFFSET SQL and count queries for total elements.

**2. Why it matters:** Efficient pagination is crucial for performance. Understanding the underlying queries helps optimize for large datasets.

**3. Internal working:**

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Automatic pagination
    Page<Product> findByCategory(String category, Pageable pageable);

    // Slice - no count query (more efficient for "load more")
    Slice<Product> findByBrand(String brand, Pageable pageable);

    // With @Query
    @Query("SELECT p FROM Product p WHERE p.price > :minPrice")
    Page<Product> findExpensiveProducts(@Param("minPrice") BigDecimal min, Pageable pageable);
}

// Usage
Pageable pageable = PageRequest.of(0, 20, Sort.by("price").descending());
Page<Product> page = productRepository.findByCategory("Electronics", pageable);

page.getContent();       // List<Product>
page.getTotalElements(); // Total count
page.getTotalPages();    // Calculated pages
page.hasNext();          // More pages?
```

Generated SQL:

```
-- Main query
SELECT * FROM products WHERE category = ? ORDER BY price DESC LIMIT 20 OFFSET 0

-- Count query (for Page, not Slice)
SELECT COUNT(*) FROM products WHERE category = ?
```

**4. Real-world example:** Optimized pagination for large tables:

```
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Custom count query (optimized)
    @Query(
        value = "SELECT o FROM Order o WHERE o.status = :status",
        countQuery = "SELECT COUNT(o.id) FROM Order o WHERE o.status = :status"
    )
    Page<Order> findByStatus(@Param("status") Status status, Pageable pageable);

    // Keyset pagination for very large datasets
    @Query("SELECT o FROM Order o WHERE o.id > :lastId ORDER BY o.id")
    Slice<Order> findNextPage(@Param("lastId") Long lastId, Pageable pageable);

    // Projection to avoid loading full entities
    @Query("SELECT new com.app.OrderSummary(o.id, o.status, o.total) " +
           "FROM Order o WHERE o.customerId = :customerId")
    Page<OrderSummary> findSummariesByCustomer(
        @Param("customerId") Long customerId, Pageable pageable);
}
```

**5. Pitfalls:**

- Count queries on large tables (slow)
- OFFSET pagination performance degradation on high pages

- JOIN FETCH with pagination (in-memory pagination warning)
- Not using Slice when count isn't needed

**6. Follow-up questions:**

- What's the difference between Page and Slice?
- How do you implement keyset pagination?

**7. Senior summary:** "Page includes count query, Slice doesn't—use Slice for 'load more' UX. Custom countQuery optimizes count performance. For very large datasets, use keyset pagination (WHERE id > lastId) instead of OFFSET. Projection reduces data transfer."

---

# Question 75: Optimistic vs Pessimistic locking?

**1. Definition:** Optimistic locking uses version checking at commit time, assuming conflicts are rare. Pessimistic locking uses database locks, preventing concurrent access.

**2. Why it matters:** Wrong locking strategy either causes lost updates (optimistic without retry) or deadlocks/throughput issues (pessimistic overuse).

**3. Internal working:**

```
// Optimistic locking
@Entity
public class Product {
    @Id
    private Long id;

    @Version  // Automatic version check on update
    private Long version;

    private BigDecimal price;
}

// Pessimistic locking
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Product p WHERE p.id = :id")
    Optional<Product> findByIdWithLock(@Param("id") Long id);
}
```

| Aspect | Optimistic | Pessimistic |
|---|---|---|
| Lock time | At commit | At read |
| Conflicts | Detected at commit | Prevented |
| Performance | Better for low contention | Better for high contention |
| Deadlocks | Not possible | Possible |
| Retry needed | Yes | No |

**4. Real-world example:** Implementing both patterns:

```
@Service
public class InventoryService {

    // Optimistic - most operations
    @Transactional
    @Retryable(value = OptimisticLockException.class, maxAttempts = 3)
    public void updateStock(Long productId, int quantity) {
        Product product = productRepository.findById(productId).orElseThrow();
        product.adjustStock(quantity);
        // If version changed, OptimisticLockException thrown
    }

    // Pessimistic - critical financial operations
    @Transactional
    public void reserveStock(Long productId, int quantity) {
        // SELECT ... FOR UPDATE
        Product product = productRepository.findByIdWithLock(productId);
        if (product.getStock() < quantity) {
            throw new InsufficientStockException();
        }
        product.setStock(product.getStock() - quantity);
        // Lock released on commit
    }
}

// Lock modes
@Lock(LockModeType.PESSIMISTIC_READ)   // Shared lock
@Lock(LockModeType.PESSIMISTIC_WRITE)  // Exclusive lock
@Lock(LockModeType.PESSIMISTIC_FORCE_INCREMENT)  // Exclusive + version increment
```

**5. Pitfalls:**

- OptimisticLockException without retry logic
- Pessimistic locks held too long (deadlocks)
- Mixing lock types incorrectly
- Forgetting @Version is checked at flush, not at read

**6. Follow-up questions:**

- How do you implement retry for optimistic locking?
- When would you use PESSIMISTIC_FORCE_INCREMENT?

**7. Senior summary:** "Optimistic uses @Version for conflict detection at commit—needs retry logic. Pessimistic uses SELECT FOR UPDATE—prevents conflicts but risks deadlocks. Use optimistic by default, pessimistic for high-contention critical operations like inventory or account balance."

---

# Question 76: What is @Version?

**1. Definition:** `@Version` marks a field for optimistic locking—JPA automatically checks this value on update and throws `OptimisticLockException` if it changed since reading.

**2. Why it matters:** @Version is the simplest way to prevent lost updates in concurrent scenarios. Understanding its behavior is essential for data integrity.

**3. Internal working:**

```
@Entity
public class Order {
    @Id
    private Long id;

    @Version
    private Long version;  // Or Integer, Timestamp

    private Status status;
}
```

Update flow:

```
-- Read
SELECT id, version, status FROM orders WHERE id = 1
-- Returns: version = 5

-- Update (includes version check)
UPDATE orders SET status = 'COMPLETED', version = 6
WHERE id = 1 AND version = 5

-- If 0 rows updated → OptimisticLockException
```

Supported types:

- `int`/`Integer`
- `long`/`Long`
- `short`/`Short`
- `Timestamp` (less reliable due to precision)

**4. Real-world example:** Handling version conflicts:

```
@RestController
public class OrderController {

    @PutMapping("/orders/{id}")
    public ResponseEntity<?> updateOrder(
            @PathVariable Long id,
            @RequestBody OrderUpdateRequest request) {
        try {
            Order updated = orderService.update(id, request);
            return ResponseEntity.ok(updated);
        } catch (OptimisticLockException e) {
            return ResponseEntity.status(HttpStatus.CONFLICT)
                .body(new ErrorResponse("CONCURRENT_UPDATE",
                    "Order was modified by another user. Please refresh and retry."));
        }
    }
}

// Client can send version to detect conflicts early
public record OrderUpdateRequest(
    Long version,  // Client's known version
    Status status,
    String notes
) {}

@Service
public class OrderService {
    @Transactional
    public Order update(Long id, OrderUpdateRequest request) {
        Order order = orderRepository.findById(id).orElseThrow();

        // Early detection
        if (!order.getVersion().equals(request.version())) {
            throw new StaleDataException("Order has been modified");
        }

        order.setStatus(request.status());
        return orderRepository.save(order);
    }
}
```

**5. Pitfalls:**

- Not handling OptimisticLockException
- Using Timestamp version (clock skew issues)
- Expecting version check on read (it's at flush/commit)
- Detached entities losing version info

**6. Follow-up questions:**

- What happens if you manually modify the version field?
- How does @Version work with merge() vs save()?

**7. Senior summary:** "@Version enables automatic optimistic locking—JPA includes version in WHERE clause on updates. Throws OptimisticLockException on conflict. Use Long for simplicity. Always handle the exception with user-friendly message and retry option."

---

# Question 77: Entity lifecycle states?

**1. Definition:** JPA entities exist in four states: New (transient), Managed (persistent), Detached, and Removed—determining how the persistence context tracks changes.

**2. Why it matters:** Understanding state transitions explains when changes are persisted, why LazyInitializationException occurs, and how merge() vs persist() differ.

**3. Internal working:**

```
                  persist()
   NEW (Transient) ──────────> MANAGED (Persistent)
          │                       │  ▲
          │                 flush │  │ find/query
          │                       ▼  │
          │             [Database Sync]
          │                       │
          │           detach/clear/close
          │                       │
          ▼                       ▼
       [GC]                   DETACHED
          ▲                       │
          │             merge()   │
          │                       ▼
          │                   MANAGED
          │                       │
          │                remove()
          │                       │
          │                       ▼
          └──────────────────  REMOVED
                   flush()        │
                                  ▼
                          [Deleted from DB]
```

```
// NEW - not associated with persistence context
Order order = new Order();  // NEW

// MANAGED - tracked for changes
Order saved = entityManager.persist(order);  // MANAGED
Order found = entityManager.find(Order.class, 1L);  // MANAGED

// DETACHED - no longer tracked
entityManager.detach(order);  // DETACHED
entityManager.clear();  // All entities DETACHED

// REMOVED - scheduled for deletion
entityManager.remove(order);  // REMOVED
```

**4. Real-world example:** State transitions in practice:

```
@Service
public class OrderService {

    @Transactional
    public Order createOrder(OrderRequest request) {
        Order order = new Order(request);  // NEW

        orderRepository.save(order);  // MANAGED
        // Changes are tracked - dirty checking

        order.setStatus(Status.PROCESSING);  // Change tracked
        // No explicit save needed!

        return order;  // Still MANAGED until TX ends
    }  // TX commits → flush → order saved with PROCESSING status

    public Order getAndModify(Long id) {
        Order order = orderRepository.findById(id).get();  // MANAGED
        // TX ends here (no @Transactional)

        order.setStatus(Status.SHIPPED);  // DETACHED - change NOT tracked!
        orderRepository.save(order);  // merge() called internally
        // Now changes are persisted
    }
}
```

**5. Pitfalls:**

- Modifying detached entities expecting auto-save
- LazyInitializationException on detached entity relationships
- Calling persist() on detached entity (exception)
- Not understanding dirty checking only works on managed entities

**6. Follow-up questions:**

- What's the difference between persist() and merge()?
- How does dirty checking work?

**7. Senior summary:** "Entities are New (transient), Managed (persistent/tracked), Detached, or Removed. Dirty checking only works on Managed entities. After transaction ends, entities become Detached. Use merge() for detached entities, persist() for new ones."

---

# Question 78: What is dirty checking?

**1. Definition:** Dirty checking is JPA's mechanism to automatically detect changes to managed entities and generate UPDATE statements at flush time without explicit save calls.

**2. Why it matters:** It eliminates explicit update calls for managed entities but can cause unexpected updates if not understood. Understanding it prevents both missed updates and unexpected ones.

**3. Internal working:**

```
@Transactional
public void updateOrder(Long orderId, Status newStatus) {
    Order order = orderRepository.findById(orderId).get();  // Snapshot saved

    order.setStatus(newStatus);  // Entity modified

    // No save() call needed!
}  // TX commits → flush → diff detected → UPDATE generated
```

How it works:

1. When entity loaded, Hibernate stores a snapshot of original values
2. At flush time, compares current state with snapshot
3. If different, generates UPDATE for changed columns
4. `@DynamicUpdate` limits UPDATE to only changed columns

```
@Entity
@DynamicUpdate  // Only update changed columns
public class Order {
    // Without: UPDATE orders SET status=?, total=?, ... WHERE id=?
    // With: UPDATE orders SET status=? WHERE id=?
}
```

**4. Real-world example:** Intentional vs unintentional dirty checking:

```
@Service
public class ProductService {

    // Intentional - modify and auto-save
    @Transactional
    public void updatePrice(Long productId, BigDecimal newPrice) {
        Product product = productRepository.findById(productId).get();
        product.setPrice(newPrice);  // Will be saved automatically
    }

    // Unintentional - be careful!
    @Transactional
    public ProductDto getProductWithCalculation(Long id) {
        Product product = productRepository.findById(id).get();

        // Oops! This modifies the entity
        product.setViewCount(product.getViewCount() + 1);

        // Even though we only wanted to return data,
        // an UPDATE will be issued!

        return ProductDto.from(product);
    }

    // Solution: Read-only transaction
    @Transactional(readOnly = true)
    public ProductDto getProductReadOnly(Long id) {
        Product product = productRepository.findById(id).get();
        product.setViewCount(product.getViewCount() + 1);  // Ignored!
        return ProductDto.from(product);
    }
}
```

**5. Pitfalls:**

- Unintentional modifications in read operations
- Performance impact of snapshot comparison on large entities
- Assuming all changes are tracked (only managed entities)
- Not using readOnly=true for pure read operations

**6. Follow-up questions:**

- How does @DynamicUpdate affect dirty checking?
- When does flush happen in a transaction?

**7. Senior summary:** "Dirty checking compares entity state at flush with original snapshot, auto-generating UPDATEs. Use @Transactional(readOnly=true) for reads to disable it. @DynamicUpdate limits UPDATE to changed columns only. Be careful—any modification to managed entity persists!"

---

# Question 79: Difference between save() and saveAndFlush()?

**1. Definition:** `save()` persists the entity but defers SQL execution until flush. `saveAndFlush()` immediately synchronizes with the database, executing SQL right away.

**2. Why it matters:** Understanding the difference is critical for operations requiring immediate ID generation, sequence access, or when you need to ensure data is in the database before continuing.

**3. Internal working:**

```
// save() - deferred write
Order order = new Order();
orderRepository.save(order);
// Entity is managed, but SQL may not be executed yet
// SQL executed at: flush, commit, or before query

// saveAndFlush() - immediate write
Order order = new Order();
Order saved = orderRepository.saveAndFlush(order);
// SQL executed immediately
// saved.getId() is guaranteed to be populated
```

When flush happens automatically:

1. Transaction commit
2. Before JPQL/native query execution
3. Manual `entityManager.flush()` call
4. `saveAndFlush()` / `flush()` repository methods

**4. Real-world example:** When saveAndFlush is needed:

```
@Service
public class OrderService {

    // Need ID immediately for external service
    @Transactional
    public void createAndNotify(OrderRequest request) {
        Order order = new Order(request);
        Order saved = orderRepository.saveAndFlush(order);

        // ID is guaranteed to exist now
        notificationService.sendOrderCreated(saved.getId());
    }

    // Batch processing - flush periodically
    @Transactional
    public void importProducts(List<ProductData> products) {
        for (int i = 0; i < products.size(); i++) {
            productRepository.save(new Product(products.get(i)));

            if (i % 50 == 0) {
                entityManager.flush();
                entityManager.clear();  // Prevent memory issues
            }
        }
    }

    // Trigger database constraint check immediately
    @Transactional
    public void createUnique(String email) {
        User user = new User(email);
        try {
            userRepository.saveAndFlush(user);  // Unique constraint checked NOW
        } catch (DataIntegrityViolationException e) {
            throw new EmailExistsException(email);
        }
    }
}
```

**5. Pitfalls:**

- Using saveAndFlush everywhere (unnecessary overhead)
- Expecting save() to fail immediately on constraint violation
- Forgetting that flush doesn't commit
- Memory issues with large batches without periodic flush/clear

**6. Follow-up questions:**

- When is save() sufficient vs saveAndFlush() required?
- What's the relationship between flush and commit?

**7. Senior summary:** "save() defers SQL until flush/commit; saveAndFlush() executes immediately. Use saveAndFlush when you need the generated ID immediately, want constraint checking before continuing, or need to synchronize with external systems. flush != commit—transaction can still rollback."

---

# Question 80: How auditing works in JPA?

**1. Definition:** JPA auditing automatically populates created/modified timestamps and user information using `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy` annotations.

**2. Why it matters:** Audit fields are required for almost every production entity—knowing who changed what and when. Automatic auditing eliminates manual code and ensures consistency.

**3. Internal working:**

```
// Enable auditing
@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
public class JpaConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        return () -> Optional.ofNullable(SecurityContextHolder.getContext())
            .map(SecurityContext::getAuthentication)
            .filter(Authentication::isAuthenticated)
            .map(Authentication::getName);
    }
}

// Base auditable entity
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime modifiedAt;

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;

    @LastModifiedBy
    private String modifiedBy;
}

// Entities extend base
@Entity
public class Order extends BaseEntity {
    @Id @GeneratedValue
    private Long id;
    private Status status;
}
```

**4. Real-world example:** Advanced auditing with custom resolver:

```
@Configuration
@EnableJpaAuditing(dateTimeProviderRef = "dateTimeProvider")
public class AuditConfig {

    @Bean
    public DateTimeProvider dateTimeProvider(Clock clock) {
        return () -> Optional.of(LocalDateTime.now(clock));
    }

    @Bean
    public AuditorAware<String> auditorAware() {
        return () -> {
            // System operations
            if (isSystemContext()) {
                return Optional.of("SYSTEM");
            }

            // Batch jobs
            if (isBatchJob()) {
                return Optional.of("BATCH:" + getBatchJobName());
            }

            // Regular user
            return getCurrentUser();
        };
    }
}

// Audit history table (Hibernate Envers)
@Entity
@Audited  // Enables revision history
public class Product extends BaseEntity {
    private String name;
    private BigDecimal price;
}

// Query audit history
AuditReader reader = AuditReaderFactory.get(entityManager);
List<Number> revisions = reader.getRevisions(Product.class, productId);
Product oldVersion = reader.find(Product.class, productId, revisionNumber);
```

**5. Pitfalls:**

- Forgetting @EntityListeners on entity
- AuditorAware returning empty for system/batch operations
- Not making createdAt/createdBy updatable=false
- Auditing not working without @EnableJpaAuditing

**6. Follow-up questions:**

- How do you audit changes to entity content, not just metadata?
- How does Hibernate Envers differ from Spring Data auditing?

**7. Senior summary:** "Spring Data auditing auto-populates audit fields via @EntityListeners. @CreatedDate/@LastModifiedDate for timestamps, @CreatedBy/@LastModifiedBy for users (via AuditorAware). Extend a @MappedSuperclass for consistency. For full change history, use Hibernate Envers."

---

# Question 81: How to handle large data efficiently?

**1. Definition:** Large data handling uses techniques like pagination, streaming, batch processing, stateless sessions, and native queries to prevent memory exhaustion and optimize performance.

**2. Why it matters:** Loading 1 million records into memory crashes applications. Proper techniques enable processing any data volume while maintaining reasonable memory and performance.

**3. Internal working:**

```
// WRONG - loads all into memory
List<Order> allOrders = orderRepository.findAll();  // OOM!

// RIGHT - pagination
Page<Order> page = orderRepository.findAll(PageRequest.of(0, 1000));
while (page.hasNext()) {
    process(page.getContent());
    page = orderRepository.findAll(page.nextPageable());
}

// RIGHT - streaming (requires transaction)
@Transactional(readOnly = true)
public void processLargeData() {
    try (Stream<Order> stream = orderRepository.streamAll()) {
        stream.forEach(this::process);
    }
}

// RIGHT - batch insert with clear
@Transactional
public void bulkInsert(List<ProductData> data) {
    int batchSize = 50;
    for (int i = 0; i < data.size(); i++) {
        entityManager.persist(new Product(data.get(i)));

        if (i > 0 && i % batchSize == 0) {
            entityManager.flush();
            entityManager.clear();  // Release memory
        }
    }
}
```

**4. Real-world example:** Production data export:

```java
@Service
public class DataExportService {

    // Stream to file without loading all in memory
    @Transactional(readOnly = true)
    public void exportToFile(Path outputPath) {
        try (Stream<Order> stream = orderRepository.streamAllOrders();
             BufferedWriter writer = Files.newBufferedWriter(outputPath)) {

            stream
                .map(this::toCsv)
                .forEach(line -> writeLine(writer, line));
        }
    }

    // Batch processing with stateless session (Hibernate)
    public void processBatch() {
        StatelessSession session = sessionFactory.openStatelessSession();
        Transaction tx = session.beginTransaction();

        ScrollableResults results = session
            .createQuery("FROM Order WHERE status = :status", Order.class)
            .setParameter("status", Status.PENDING)
            .setFetchSize(1000)
            .scroll(ScrollMode.FORWARD_ONLY);

        int count = 0;
        while (results.next()) {
            Order order = (Order) results.get(0);
            process(order);
            session.update(order);

            if (++count % 1000 == 0) {
                tx.commit();
                tx = session.beginTransaction();
            }
        }

        tx.commit();
        session.close();
    }
}

// Repository with stream support
public interface OrderRepository extends JpaRepository<Order, Long> {

    @Query("SELECT o FROM Order o")
    @QueryHints(@QueryHint(name = HINT_FETCH_SIZE, value = "1000"))
    Stream<Order> streamAllOrders();
}
```

**5. Pitfalls:**

- Streaming without transaction (connection closed)
- Not closing Stream (resource leak)
- Batch inserts without hibernate.jdbc.batch_size config
- forgetting to clear EntityManager in batch operations

**6. Follow-up questions:**

- What's the difference between Stream and ScrollableResults?
- How do you configure Hibernate batch insert size?

**7. Senior summary:** ""For large data: paginate for UI, stream for processing, batch with flush/clear for inserts. Enable JDBC batching via hibernate.jdbc.batch_size. StatelessSession bypasses persistence context for true streaming. Always close streams and monitor memory."

---

# SECURITY QUESTIONS (82-101) - 5+ YOE KILLER QUESTIONS

## Question 82: What is Spring Security?

**1. Definition:** Spring Security is a comprehensive security framework providing authentication, authorization, protection against common exploits (CSRF, session fixation), and integration with various authentication mechanisms.

**2. Why it matters:** Security is non-negotiable in production. Spring Security provides battle-tested implementations of complex security patterns that would be error-prone to implement manually.

**3. Internal working:**

```
Request → FilterChain → SecurityFilterChain →
    [CsrfFilter → AuthenticationFilter → AuthorizationFilter → ...]
    → Controller

Key components:
- SecurityFilterChain: Configures filter chain
- AuthenticationManager: Verifies credentials
- UserDetailsService: Loads user data
- SecurityContext: Holds authentication
- AccessDecisionManager: Authorization decisions
```

Spring Boot auto-configuration:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/")
            );
        return http.build();
    }
}
```

**4. Real-world example:** Multi-layer security configuration:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .csrf(csrf -> csrf.disable())  // Stateless API
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 ->
                oauth2.jwt(jwt -> jwt.decoder(jwtDecoder())));
        return http.build();
    }

    @Bean
    @Order(2)
    public SecurityFilterChain webSecurity(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/css/**", "/js/**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults());
        return http.build();
    }
}
```

**5. Pitfalls:**

- Disabling CSRF without understanding implications
- Over-permissive security rules
- Not securing actuator endpoints
- Mixing authentication methods incorrectly

**6. Follow-up questions:**

- How do you configure multiple SecurityFilterChains?
- What's the order of security filters?

**7. Senior summary:** "Spring Security provides comprehensive authentication/authorization via filter chain architecture. Configure with SecurityFilterChain bean, use @EnableMethodSecurity for method-level security. Separate API (stateless/JWT) and web (session/form) security configs."

---

# Question 83: How authentication works?

**1. Definition:** Authentication verifies user identity by extracting credentials from the request, validating them against a UserDetailsService, and storing the result in SecurityContext.

**2. Why it matters:** Understanding the flow enables custom authentication schemes, proper error handling, and debugging authentication issues.

**3. Internal working:**

```
Request with credentials
    ↓
AuthenticationFilter (extracts credentials)
    ↓
AuthenticationManager.authenticate()
    ↓
AuthenticationProvider.authenticate()
    ↓
UserDetailsService.loadUserByUsername()
    ↓
Password verification (PasswordEncoder)
    ↓
Authentication object created
    ↓
SecurityContextHolder.setAuthentication()
    ↓
Request proceeds (authenticated)
```

```java
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getEmail())
            .password(user.getPassword())  // Already encoded
            .authorities(user.getRoles().stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .toList())
            .accountLocked(!user.isActive())
            .build();
    }
}
```

**4. Real-world example:** Custom authentication with additional validation:

```
@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

    @Override
    public Authentication authenticate(Authentication auth) {
        String username = auth.getName();
        String password = auth.getCredentials().toString();

        User user = userService.findByEmail(username);

        // Custom validations
        if (user.isLocked()) {
            throw new LockedException("Account locked due to too many failed attempts");
        }
        if (user.requiresMfaVerification()) {
            throw new MfaRequiredException(user.getMfaToken());
        }
        if (!passwordEncoder.matches(password, user.getPassword())) {
            userService.recordFailedAttempt(user);
            throw new BadCredentialsException("Invalid credentials");
        }

        userService.resetFailedAttempts(user);

        return new UsernamePasswordAuthenticationToken(
            user, null, user.getAuthorities());
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return UsernamePasswordAuthenticationToken.class
            .isAssignableFrom(authentication);
    }
}
```

**5. Pitfalls:**

- Exposing whether username exists (security info leak)
- Not locking accounts after failed attempts
- Storing passwords in plain text
- Session fixation after authentication

**6. Follow-up questions:**

- How do you implement multi-factor authentication?
- What's the difference between AuthenticationProvider and UserDetailsService?

**7. Senior summary:** "Authentication flows through filters → AuthenticationManager → AuthenticationProvider → UserDetailsService. Passwords verified via PasswordEncoder. Result stored in SecurityContext. Custom providers enable MFA, account locking, and custom validation logic."

---

# Question 84: How authorization works?

**1. Definition:** Authorization determines if an authenticated user has permission to access a resource, using URL-based rules, method-level annotations, or custom access decisions.

**2. Why it matters:** Authentication proves identity; authorization enforces access control. Proper authorization prevents privilege escalation and unauthorized data access.

**3. Internal working:**

```
// URL-based authorization
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/api/**").hasAuthority("DELETE_PRIVILEGE")
    .anyRequest().authenticated()
);

// Method-level authorization
@EnableMethodSecurity
public class SecurityConfig { }

@Service
public class OrderService {

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteOrder(Long id) { }

    @PreAuthorize("#userId == authentication.principal.id")
    public List<Order> getOrdersForUser(Long userId) { }

    @PostAuthorize("returnObject.owner == authentication.name")
    public Order getOrder(Long id) { }

    @PreAuthorize("@accessChecker.canAccess(#orderId)")
    public void processOrder(Long orderId) { }
}
```

Authorization flow:

```
AuthorizationFilter intercepts request
    ↓
AuthorizationManager.check()
    ↓
Check against configured rules
    ↓
AccessDeniedException if unauthorized
```

**4. Real-world example:** Complex authorization with custom logic:

```
@Component
public class OrderAccessChecker {

    public boolean canAccess(Long orderId, Authentication auth) {
        Order order = orderRepository.findById(orderId).orElse(null);
        if (order == null) return false;

        UserPrincipal user = (UserPrincipal) auth.getPrincipal();

        // Admin can access all
        if (user.hasRole("ADMIN")) return true;

        // Owner can access their orders
        if (order.getCustomerId().equals(user.getId())) return true;

        // Support staff can access orders in their region
        if (user.hasRole("SUPPORT") &&
            order.getRegion().equals(user.getRegion())) return true;

        return false;
    }
}

@RestController
public class OrderController {

    @GetMapping("/orders/{id}")
    @PreAuthorize("@orderAccessChecker.canAccess(#id, authentication)")
    public Order getOrder(@PathVariable Long id) {
        return orderService.findById(id);
    }

    @DeleteMapping("/orders/{id}")
    @PreAuthorize("hasRole('ADMIN') or " +
                  "(hasRole('USER') and @orderAccessChecker.isOwner(#id, authentication))")
    public void deleteOrder(@PathVariable Long id) {
        orderService.delete(id);
    }
}
```

**5. Pitfalls:**

- Over-relying on URL patterns (methods can be missed)
- Not using method security for service layer
- Forgetting to enable @EnableMethodSecurity
- SPEL injection in dynamic authorization expressions

**6. Follow-up questions:**

- What's the difference between @Secured, @PreAuthorize, and @RolesAllowed?
- How do you test authorization rules?

**7. Senior summary:** "Authorization uses URL patterns for coarse control, @PreAuthorize/@PostAuthorize for method-level. Custom beans in SpEL enable complex logic. Layer URL security (broad) with method security (specific). Test authorization rules explicitly."

---

# Question 85: JWT authentication flow?

**1. Definition:** JWT (JSON Web Token) authentication is a stateless mechanism where the server issues a signed token on login, and the client includes it in subsequent requests for authentication.

**2. Why it matters:** JWTs enable stateless, scalable APIs—no server-side session storage. Critical for microservices where requests may hit different instances.

**3. Internal working:**

```
1. Client sends credentials to /auth/login
2. Server validates credentials
3. Server generates JWT with claims (user ID, roles, expiry)
4. Server signs JWT with secret key
5. Client stores JWT (localStorage/cookie)
6. Client sends JWT in Authorization header
7. Server validates signature and extracts claims
8. Server authenticates based on claims

JWT Structure:
Header.Payload.Signature

{
  "alg": "HS256"
}
.
{
  "sub": "user@example.com",
  "roles": ["USER", "ADMIN"],
  "exp": 1735689600,
  "iat": 1735686000
}
.
HMACSHA256(header + "." + payload, secret)
```

**4. Real-world example:** Complete JWT implementation:

```java
@Configuration
@EnableWebSecurity
public class JwtSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws IOException, ServletException {
        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String jwt = authHeader.substring(7);
            try {
                Claims claims = jwtService.validateToken(jwt);
                UserDetails user = userDetailsService.loadUserByUsername(claims.getSubject());

                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                // Invalid token - continue as unauthenticated
            }
        }

        chain.doFilter(request, response);
    }
}

@Service
public class JwtService {
    private final SecretKey key = Keys.hmacShaKeyFor(secretBytes);

    public String generateToken(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .claim("roles", user.getAuthorities())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + 3600000)) // 1 hour
            .signWith(key)
            .compact();
    }

    public Claims validateToken(String token) {
        return Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
```

```
            .getPayload();
    }
}
```

**5. Pitfalls:**

- Storing sensitive data in JWT payload (it's base64, not encrypted)
- Short secret keys (use at least 256 bits)
- No token revocation strategy
- Storing JWT in localStorage (XSS vulnerable)

**6. Follow-up questions:**

- How do you revoke a JWT before expiry?
- What's the difference between symmetric and asymmetric JWT signing?

**7. Senior summary:** "JWT enables stateless authentication—server issues signed token, client presents it. Validate signature and expiry on each request. Use strong secrets, short expiry times with refresh tokens. Store in httpOnly cookie to prevent XSS theft."

---

# Question 86: Stateless vs Stateful authentication?

**1. Definition:** Stateful authentication stores session data on the server (e.g., HttpSession). Stateless authentication includes all necessary data in each request (e.g., JWT).

**2. Why it matters:** Choice affects scalability, complexity, and security characteristics. Microservices favor stateless; traditional web apps often use stateful.

**3. Internal working:**

| Aspect | Stateful (Session) | Stateless (JWT) |
|---|---|---|
| Server storage | Yes (session store) | No |
| Scalability | Requires sticky sessions or shared store | Unlimited horizontal scale |
| Revocation | Easy (invalidate session) | Difficult (need blacklist) |
| Data freshness | Always current | Stale until token expires |
| Security | Session ID theft | Token theft |

```
// Stateful configuration
http.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
    .maximumSessions(1)
    .sessionRegistry(sessionRegistry())
);

// Stateless configuration
http.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
);
```

**4. Real-world example:** Hybrid approach for different endpoints:

```
@Configuration
public class SecurityConfig {

    // API - stateless JWT
    @Bean
    @Order(1)
    public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    // Web UI - stateful session
    @Bean
    @Order(2)
    public SecurityFilterChain webSecurity(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/**")
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                .maximumSessions(1)
                .expiredUrl("/login?expired"))
            .formLogin(Customizer.withDefaults());
        return http.build();
    }
}
```

**5. Pitfalls:**

- Using stateless without considering revocation needs
- Stateful without distributed session store in clustered env
- Mixing session and JWT in confusing ways
- JWT expiry too long (stale permissions)

**6. Follow-up questions:**

- How do you handle logout with stateless authentication?
- What's the trade-off between short and long JWT expiry?

**7. Senior summary:** "Stateful uses server-side sessions—easy revocation but requires session store for scaling. Stateless uses tokens—infinitely scalable but revocation requires additional mechanisms. Choose based on architecture: JWT for APIs/microservices, sessions for traditional web apps."

---

# Question 87: OAuth2 basics?

**1. Definition:** OAuth2 is an authorization framework that enables third-party applications to access resources on behalf of users without sharing credentials, using access tokens.

**2. Why it matters:** OAuth2 is the standard for API authorization and social login. Understanding flows enables proper integration with identity providers.

**3. Internal working:** OAuth2 roles:

- **Resource Owner**: User who owns the data
- **Client**: Application requesting access
- **Authorization Server**: Issues tokens (Keycloak, Auth0)
- **Resource Server**: API with protected resources

Common flows:

```
Authorization Code (web apps):
1. User clicks "Login with Google"
2. Redirect to Google authorization endpoint
3. User approves
4. Google redirects back with authorization code
5. Server exchanges code for access token
6. Server uses token to access resources

Client Credentials (server-to-server):
1. Service sends client_id + client_secret
2. Authorization server returns access token
3. Service uses token to call other services
```

**4. Real-world example:** Spring Security OAuth2 configuration:

```
// OAuth2 Login (social login)
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .oauth2Login(oauth2 -> oauth2
            .userInfoEndpoint(userInfo -> userInfo
                .userService(customOAuth2UserService))
            .successHandler(oAuth2SuccessHandler)
        );
    return http.build();
}

// OAuth2 Resource Server (API protection)
@Bean
public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    http
        .oauth2ResourceServer(oauth2 -> oauth2
            .jwt(jwt -> jwt
                .jwtAuthenticationConverter(jwtAuthConverter())
            )
        );
    return http.build();
}

// application.yml
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: email, profile
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com/realms/myapp
```

**5. Pitfalls:**

- Using implicit flow (deprecated, insecure)
- Not validating token issuer
- Storing tokens insecurely
- Confusion between authentication (OpenID Connect) and authorization (OAuth2)

**6. Follow-up questions:**

- What's the difference between OAuth2 and OpenID Connect?
- When would you use Client Credentials vs Authorization Code flow?

**7. Senior summary:** "OAuth2 is authorization framework using access tokens. Authorization Code flow for user-facing apps, Client Credentials for service-to-service. Spring Security provides both client (oauth2Login) and resource server (oauth2ResourceServer) support. OpenID Connect adds authentication layer."

---

# Question 88: How to secure REST APIs?

**1. Definition:** REST API security combines authentication (JWT/OAuth2), authorization (role/scope-based), transport security (HTTPS), input validation, and protection against common attacks.

**2. Why it matters:** APIs are prime attack targets. Layered security defends against various attack vectors—authentication bypass, injection, data exposure.

**3. Internal working:** Security layers:

```
1. Transport: HTTPS/TLS
2. Authentication: JWT, OAuth2, API Keys
3. Authorization: Roles, Scopes, Resource-based
4. Input Validation: Bean Validation, sanitization
5. Rate Limiting: Prevent abuse
6. Output: No sensitive data exposure
```

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class ApiSecurityConfig {

    @Bean
    public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .csrf(csrf -> csrf.disable())  // Stateless API
            .cors(cors -> cors.configurationSource(corsConfig()))
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()))
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(customEntryPoint)
                .accessDeniedHandler(customAccessDenied)
            );
        return http.build();
    }
}
```

**4. Real-world example:** Comprehensive API security:

```
@RestController
@RequestMapping("/api/v1")
public class SecureApiController {

    @GetMapping("/users/{id}")
    @PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
    public UserDto getUser(@PathVariable Long id) {
        // Authorization checked
        return userService.findById(id);
    }

    @PostMapping("/orders")
    @RateLimited(requests = 100, period = "1m")  // Custom annotation
    public Order createOrder(@RequestBody @Valid OrderRequest request) {
        // Input validated
        return orderService.create(request);
    }
}

// Security headers
@Component
public class SecurityHeadersFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {
        HttpServletResponse response = (HttpServletResponse) res;
        response.setHeader("X-Content-Type-Options", "nosniff");
        response.setHeader("X-Frame-Options", "DENY");
        response.setHeader("X-XSS-Protection", "1; mode=block");
        response.setHeader("Cache-Control", "no-store");
        chain.doFilter(req, res);
    }
}
```

**5. Pitfalls:**

- HTTPS in production but HTTP in dev (behavior differences)
- Verbose error messages exposing system info
- Not validating Content-Type header
- Missing rate limiting

**6. Follow-up questions:**

- How do you prevent enumeration attacks?
- What security headers should every API return?

**7. Senior summary:** "Secure APIs with layers: HTTPS transport, JWT/OAuth2 authentication, role-based authorization, input validation, rate limiting, security headers. Disable CSRF for stateless APIs but keep it for session-based. Return consistent error messages without exposing internals."

---

# Question 89: Role-based access control?

**1. Definition:** RBAC assigns permissions to roles (ADMIN, USER) rather than individuals. Users are assigned roles, and access decisions are made based on role membership.

**2. Why it matters:** RBAC simplifies permission management—manage roles, not individual user permissions. Scales well and is easier to audit.

**3. Internal working:**

```
// Role definition in UserDetails
public class CustomUserDetails implements UserDetails {
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return roles.stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
            .collect(Collectors.toList());
    }
}

// URL-based RBAC
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers("/api/**").hasAuthority("API_ACCESS")
);

// Method-level RBAC
@PreAuthorize("hasRole('ADMIN')")
public void adminOperation() { }

@PreAuthorize("hasAnyRole('USER', 'ADMIN')")
public void userOperation() { }
```

Note: Spring Security automatically adds "ROLE_" prefix for hasRole() checks.

**4. Real-world example:** Hierarchical roles with permissions:

```
@Entity
public class Role {
    @Id
    private Long id;
    private String name;

    @ManyToMany(fetch = FetchType.EAGER)
    private Set<Permission> permissions;
}

@Entity
public class Permission {
    @Id
    private Long id;
    private String name;  // READ_USERS, WRITE_USERS, DELETE_USERS
}

// Custom authority mapping
@Component
public class CustomUserDetailsService implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByEmailWithRoles(username);

        List<GrantedAuthority> authorities = new ArrayList<>();

        for (Role role : user.getRoles()) {
            authorities.add(new SimpleGrantedAuthority("ROLE_" + role.getName()));
            for (Permission perm : role.getPermissions()) {
                authorities.add(new SimpleGrantedAuthority(perm.getName()));
            }
        }

        return new CustomUserPrincipal(user, authorities);
    }
}

// Usage with permissions
@PreAuthorize("hasAuthority('DELETE_USERS')")
public void deleteUser(Long id) { }

@PreAuthorize("hasRole('ADMIN') or hasAuthority('MANAGE_ORDERS')")
public void cancelOrder(Long id) { }
```

**5. Pitfalls:**

- Role explosion (too many fine-grained roles)
- Hardcoding roles in code (hard to change)
- Not considering role hierarchy
- Mixing ROLE_ prefix inconsistently

**6. Follow-up questions:**

- How do you implement role hierarchy?
- When would you use permissions instead of roles?

**7. Senior summary:** "RBAC assigns permissions via roles, not individuals. Spring Security uses GrantedAuthority with ROLE_ prefix. Combine roles (broad) with permissions (specific) for flexibility. Use role hierarchy for inheritance. Store in database for dynamic management."

---

# Question 90: Filter vs Interceptor?

**1. Definition:** Filters are Servlet API components operating on raw requests/responses. Interceptors are Spring MVC components with access to handler information and model.

**2. Why it matters:** Choosing correctly affects what information is available and when in the request lifecycle. Security typically uses filters; cross-cutting MVC concerns use interceptors.

**3. Internal working:**

```
Request → Filter₁ → Filter₂ → ... → DispatcherServlet →
         → Interceptor.preHandle() →
         → Controller →
         → Interceptor.postHandle() →
         → View rendering →
         → Interceptor.afterCompletion() →
         ← Filter₂ ← Filter₁ ← Response
```

| Aspect | Filter | Interceptor |
|---|---|---|
| API | Servlet | Spring MVC |
| Access | Request/Response | Handler, ModelAndView |
| Scope | All requests | Spring MVC requests only |
| Order | Before DispatcherServlet | After DispatcherServlet |
| Exception handling | Limited | Full Spring exception handling |

```
// Filter
@Component
public class LoggingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {
        // Before - raw servlet objects
        chain.doFilter(req, res);
        // After
    }
}

// Interceptor
@Component
public class LoggingInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) {
        // Handler info available
        if (handler instanceof HandlerMethod) {
            HandlerMethod method = (HandlerMethod) handler;
            log.info("Calling: {}", method.getMethod().getName());
        }
        return true;  // Continue chain
    }

    @Override
    public void postHandle(HttpServletRequest request,
                           HttpServletResponse response,
                           Object handler,
                           ModelAndView modelAndView) {
        // After controller, before view
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
                                HttpServletResponse response,
                                Object handler,
                                Exception ex) {
        // After complete request (including view)
    }
}
```

**4. Real-world example:** Using both appropriately:

```
// Security - Filter (before Spring MVC)
@Component
public class ApiKeyFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) {
        String apiKey = request.getHeader("X-API-Key");
        if (!isValidApiKey(apiKey)) {
            response.setStatus(401);
            return;
        }
        chain.doFilter(request, response);
    }
}

// Audit logging - Interceptor (handler info available)
@Component
public class AuditInterceptor implements HandlerInterceptor {
    @Override
    public void afterCompletion(HttpServletRequest request,
                                HttpServletResponse response,
                                Object handler,
                                Exception ex) {
        if (handler instanceof HandlerMethod) {
            HandlerMethod method = (HandlerMethod) handler;
            if (method.hasMethodAnnotation(Audited.class)) {
                auditService.log(request, response, method);
            }
        }
    }
}
```

**5. Pitfalls:**

- Using interceptor for security (too late in chain)
- Filter for logic requiring Spring context (not available early)
- Not registering interceptor with WebMvcConfigurer
- Filter ordering issues

**6. Follow-up questions:**

- Can an interceptor prevent a request from reaching the controller?
- How do you order multiple filters?

**7. Senior summary:** "Filters for security and low-level concerns (before Spring MVC). Interceptors for MVC-aware cross-cutting concerns (audit, logging). Filters see raw servlet; interceptors see handler info. Security filters must run before DispatcherServlet."

---

# Question 91: What is CSRF?

**1. Definition:** CSRF (Cross-Site Request Forgery) is an attack where a malicious site tricks a user's browser into making unwanted requests to a site where they're authenticated, using their existing session.

**2. Why it matters:** CSRF can lead to unauthorized state changes—fund transfers, password changes, data deletion. Protection is essential for session-based applications.

**3. Internal working:** Attack scenario:

```
1. User logs into bank.com, session cookie set
2. User visits evil.com
3. evil.com has: <img src="https://bank.com/transfer?to=attacker&amount=1000">
4. Browser sends request WITH session cookie
5. Bank.com sees valid session, executes transfer
```

Protection (Synchronizer Token Pattern):

```
// Spring Security default for session-based apps
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
);

// In HTML forms
<form method="post" action="/transfer">
    <input type="hidden" name="${_csrf.parameterName}" value="${_csrf.token}"/>
    ...
</form>

// In AJAX (from meta tag or cookie)
fetch('/api/transfer', {
    method: 'POST',
    headers: {
        'X-CSRF-TOKEN': csrfToken
    }
});
```

**4. Real-world example:** Configuring CSRF properly:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain webSecurity(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf
                // Cookie-based token for SPAs
                .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
                // Custom handler
                .csrfTokenRequestHandler(new CsrfTokenRequestAttributeHandler())
                // Ignore specific paths (webhooks)
                .ignoringRequestMatchers("/webhook/**")
            );
        return http.build();
    }

    // For stateless APIs - disable CSRF
    @Bean
    @Order(1)
    public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")
            .csrf(csrf -> csrf.disable())  // Safe for stateless
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
        return http.build();
    }
}
```

**5. Pitfalls:**

- Disabling CSRF for session-based apps
- Not including token in AJAX requests
- Storing CSRF token in localStorage (defeats purpose)
- Thinking CSRF protects against XSS (it doesn't)

**6. Follow-up questions:**

- Why is CSRF not needed for stateless APIs?
- What's the difference between CSRF token in cookie vs session?

**7. Senior summary:** "CSRF exploits session cookies to forge requests. Spring Security uses synchronizer tokens—server verifies token matches session. Safe to disable for stateless JWT APIs (no session to exploit). Always enable for session-based apps."

# Question 92: CORS vs CSRF?

**1. Definition:** CORS controls which origins can make cross-origin requests to your server. CSRF prevents malicious sites from making authenticated requests on behalf of users.

**2. Why it matters:** Both involve cross-origin scenarios but solve different problems. Confusing them leads to security vulnerabilities or broken functionality.

**3. Internal working:**

| Aspect | CORS | CSRF |
|---|---|---|
| Purpose | Allow cross-origin access | Prevent forged requests |
| Direction | Who can call my API | Stop malicious callers |
| Mechanism | Browser preflight + headers | Token verification |
| Configuration | Server whitelist | Token generation/validation |
| When needed | APIs consumed by other domains | Session-based auth |

```
// CORS - "who can call me"
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("https://app.example.com"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", config);
    return source;
}

// CSRF - "verify it's really my user"
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
);
```

**4. Real-world example:** SPA with API backend:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // CORS - allow frontend domain
            .cors(cors -> cors.configurationSource(corsConfig()))

            // CSRF - frontend sends token in header
            .csrf(csrf -> csrf
                .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
                .csrfTokenRequestHandler(new SpaCsrfTokenRequestHandler())
            )

            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED))

            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            );

        return http.build();
    }
}

// Frontend (React) sends both:
// 1. Cookie with session ID (CORS allows it)
// 2. CSRF token in header (proves it's not forged)
axios.defaults.withCredentials = true;  // Send cookies
axios.defaults.headers.common['X-CSRF-TOKEN'] = getCsrfToken();
```

**5. Pitfalls:**

- Thinking CORS protects against CSRF (it doesn't)
- Disabling CSRF because CORS is configured (wrong)
- allowedOrigins("*") with allowCredentials(true) (forbidden)
- Not understanding both can be needed simultaneously

**6. Follow-up questions:**

- Can CORS prevent CSRF attacks?
- Why can't you use wildcard origins with credentials?

**7. Senior summary:** "CORS controls access—which domains can call your API. CSRF prevents forgery—verifies requests come from legitimate users. Both needed for session-based apps with cross-origin frontends. CORS allows the call; CSRF verifies authenticity."

---

# Question 93: How password encoding works?

**1. Definition:** Password encoding uses one-way hashing with salt to store passwords securely. Spring Security's `PasswordEncoder` handles hashing on storage and verification on login.

**2. Why it matters:** Plain text or weak hashing leads to mass credential theft on breach. Proper encoding limits damage—attackers get hashes, not passwords.

**3. Internal working:**

```
// Configure encoder
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);  // Work factor 12
}

// Registration
String rawPassword = "user123";
String encoded = passwordEncoder.encode(rawPassword);
// $2a$12$K7L1OJ45/N7IzVQT8vWeEO...

// Login verification
boolean matches = passwordEncoder.matches(rawPassword, storedHash);
```

BCrypt internals:

```
$2a$12$K7L1OJ45/N7IzVQT8vWeEOgKL3p.../...
 |  |  |                          |
 |  |  └─ Salt (22 chars)          └─ Hash
 |  └─ Work factor (2^12 iterations)
 └─ Algorithm identifier
```

**4. Real-world example:** Upgrading password encoding:

```
@Bean
public PasswordEncoder passwordEncoder() {
    // DelegatingPasswordEncoder for migration
    Map<String, PasswordEncoder> encoders = new HashMap<>();
    encoders.put("bcrypt", new BCryptPasswordEncoder(12));
    encoders.put("scrypt", SCryptPasswordEncoder.defaultsForSpringSecurity_v5_8());
    encoders.put("argon2", Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8());

    DelegatingPasswordEncoder encoder =
        new DelegatingPasswordEncoder("argon2", encoders);

    // Handle legacy passwords without prefix
    encoder.setDefaultPasswordEncoderForMatches(new BCryptPasswordEncoder(10));

    return encoder;
}

// Stored passwords:
// Old: $2a$10$... (bcrypt, no prefix - uses default)
// Migrated: {bcrypt}$2a$12$... (bcrypt with prefix)
// New: {argon2}... (argon2)

// Re-encode on login if using old encoder
@Component
public class PasswordUpgradeService {
    public void upgradePasswordIfNeeded(User user, String rawPassword) {
        if (passwordEncoder.upgradeEncoding(user.getPassword())) {
            user.setPassword(passwordEncoder.encode(rawPassword));
            userRepository.save(user);
        }
    }
}
```

**5. Pitfalls:**

- Using MD5 or SHA-1 (too fast, easily cracked)
- Low work factor (benchmark for ~1 second hash time)
- Not salting (enables rainbow tables)
- Comparing encoded values directly (use matches())

**6. Follow-up questions:**

- Why is BCrypt better than SHA-256 for passwords?
- How do you migrate from weak to strong password encoding?

**7. Senior summary:** "Use BCrypt, SCrypt, or Argon2—adaptive algorithms designed for passwords. Work factor should make hashing take ~1 second. Use DelegatingPasswordEncoder for algorithm migration. Never compare hashes directly—use matches() which handles timing attacks."

---

# Question 94: What is SecurityFilterChain?

**1. Definition:** `SecurityFilterChain` is a bean defining a set of security filters and their configuration for matching requests. It replaced the deprecated `WebSecurityConfigurerAdapter` in Spring Security 5.7+.

**2. Why it matters:** It's the modern way to configure security. Multiple SecurityFilterChains enable different security policies for different parts of your application.

**3. Internal working:**

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // First filter chain - API security
    @Bean
    @Order(1)
    public SecurityFilterChain apiSecurityFilterChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**")  // Only matches /api paths
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));

        return http.build();
    }

    // Second filter chain - Web security
    @Bean
    @Order(2)
    public SecurityFilterChain webSecurityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login", "/css/**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard")
            );

        return http.build();
    }
}
```

Filter chain selection:

1. Iterate through beans by @Order
2. First chain where securityMatcher matches handles request
3. If no matcher specified, matches all requests

**4. Real-world example:** Complex multi-tenant security:

```java
@Configuration
@EnableWebSecurity
public class MultiSecurityConfig {

    // Health endpoints - no security
    @Bean
    @Order(0)
    public SecurityFilterChain healthSecurityChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/actuator/health/**")
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
        return http.build();
    }

    // Admin API - strict security
    @Bean
    @Order(1)
    public SecurityFilterChain adminApiChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/admin/**")
            .authorizeHttpRequests(auth -> auth
                .anyRequest().hasRole("ADMIN"))
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt
                .jwtAuthenticationConverter(adminJwtConverter())));
        return http.build();
    }

    // Public API - rate-limited, basic auth
    @Bean
    @Order(2)
    public SecurityFilterChain publicApiChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/public/**")
            .addFilterBefore(rateLimitFilter, UsernamePasswordAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
        return http.build();
    }

    // Default - form login
    @Bean
    @Order(3)
    public SecurityFilterChain defaultChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated())
            .formLogin(Customizer.withDefaults());
        return http.build();
    }
}
```

**5. Pitfalls:**

- Order matters—first match wins
- Missing securityMatcher makes chain match everything
- Duplicate filter chains for same paths
- Not building the chain (forgetting .build())

**6. Follow-up questions:**

- How does securityMatcher differ from authorizeHttpRequests?
- Can a request match multiple SecurityFilterChains?

**7. Senior summary:** "SecurityFilterChain is the modern security configuration approach—one bean per security policy. Use @Order to control precedence, securityMatcher to scope paths. First matching chain handles the request. Enables clean separation of API vs web security."

---

# Question 95: How custom authentication works?

**1. Definition:** Custom authentication involves implementing `AuthenticationProvider` to handle non-standard authentication mechanisms like multi-factor, LDAP, or external identity systems.

**2. Why it matters:** Real applications often need custom authentication logic—MFA, account locking, external providers. Understanding the extension points enables these requirements.

**3. Internal working:**

```
@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

    @Override
    public Authentication authenticate(Authentication authentication)
            throws AuthenticationException {

        String username = authentication.getName();
        String password = authentication.getCredentials().toString();
        String mfaCode = ((CustomAuthenticationToken) authentication).getMfaCode();

        // Load user
        User user = userService.findByUsername(username);
        if (user == null) {
            throw new BadCredentialsException("User not found");
        }

        // Validate password
        if (!passwordEncoder.matches(password, user.getPassword())) {
            userService.recordFailedAttempt(user);
            throw new BadCredentialsException("Invalid password");
        }

        // Validate MFA
        if (user.isMfaEnabled()) {
            if (!mfaService.validate(user, mfaCode)) {
                throw new BadCredentialsException("Invalid MFA code");
            }
        }

        // Success
        userService.clearFailedAttempts(user);
        return new UsernamePasswordAuthenticationToken(
            user, null, user.getAuthorities());
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return CustomAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```

**4. Real-world example:** Complete MFA flow:

```
// Custom token with MFA code
public class MfaAuthenticationToken extends UsernamePasswordAuthenticationToken {
    private final String mfaCode;

    public MfaAuthenticationToken(String username, String password, String mfaCode) {
        super(username, password);
        this.mfaCode = mfaCode;
    }
}

// Custom filter to create token
public class MfaAuthenticationFilter extends UsernamePasswordAuthenticationFilter {

    @Override
    public Authentication attemptAuthentication(HttpServletRequest request,
                                                HttpServletResponse response) {
        String username = obtainUsername(request);
        String password = obtainPassword(request);
        String mfaCode = request.getParameter("mfaCode");

        MfaAuthenticationToken token =
            new MfaAuthenticationToken(username, password, mfaCode);

        return getAuthenticationManager().authenticate(token);
    }
}

// Configuration
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authenticationProvider(customAuthenticationProvider)
        .addFilterAt(mfaAuthenticationFilter(), UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

**5. Pitfalls:**

- Not handling all exception types (information leak)
- Timing attacks in credential validation
- Forgetting to register custom provider
- supports() method not matching token type

**6. Follow-up questions:**

- How do you chain multiple AuthenticationProviders?
- How do you handle partial authentication (MFA step)?

**7. Senior summary:** "Custom authentication needs AuthenticationProvider implementation. Override authenticate() for validation logic, supports() to declare token types handled. Register with AuthenticationManagerBuilder. For MFA, use custom token types and staged authentication."

---

# Question 96: How to secure Actuator endpoints?

**1. Definition:** Actuator endpoints are secured via SecurityFilterChain configuration, typically separating management security from main application security.

**2. Why it matters:** Actuator exposes sensitive data—environment variables, heap dumps, shutdown. Unsecured actuators have led to major breaches.

**3. Internal working:**

```
@Configuration
@EnableWebSecurity
public class ActuatorSecurityConfig {

    @Bean
    @Order(1)
    public SecurityFilterChain actuatorSecurityChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher(EndpointRequest.toAnyEndpoint())
            .authorizeHttpRequests(auth -> auth
                // Health and info - public (for load balancers)
                .requestMatchers(EndpointRequest.to("health", "info")).permitAll()
                // Prometheus metrics - internal network only
                .requestMatchers(EndpointRequest.to("prometheus"))
                    .access(hasIpAddress("10.0.0.0/8").or(hasIpAddress("192.168.0.0/16")))
                // Everything else - admin only
                .anyRequest().hasRole("ACTUATOR_ADMIN")
            )
            .httpBasic(Customizer.withDefaults());  // Simple auth for actuator

        return http.build();
    }
}
```

Configuration:

```
management:
  server:
    port: 8081  # Separate port
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
      base-path: /management
  endpoint:
    health:
      show-details: when_authorized
      probes:
        enabled: true
    env:
      show-values: never  # Never show sensitive values
```

**4. Real-world example:** Production actuator security:

```java
@Configuration
public class ActuatorConfig {

    @Bean
    @Order(0)
    public SecurityFilterChain actuatorChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/actuator/**")
            .authorizeHttpRequests(auth -> auth
                // Kubernetes probes
                .requestMatchers("/actuator/health/liveness").permitAll()
                .requestMatchers("/actuator/health/readiness").permitAll()
                // Prometheus scraping - service account
                .requestMatchers("/actuator/prometheus")
                    .hasRole("METRICS_SCRAPER")
                // Admin operations - require MFA
                .requestMatchers("/actuator/loggers/**", "/actuator/env/**")
                    .access(hasRole("ADMIN").and(hasMfaVerified()))
                // Thread dump, heap dump - super admin only
                .requestMatchers("/actuator/threaddump", "/actuator/heapdump")
                    .hasRole("SUPER_ADMIN")
                .anyRequest().hasRole("ADMIN")
            )
            .httpBasic(basic -> basic.realmName("Actuator"));

        return http.build();
    }
}

// Custom authorization for MFA
private static AuthorizationManager<RequestAuthorizationContext> hasMfaVerified() {
    return (auth, context) -> {
        Authentication authentication = auth.get();
        if (authentication instanceof CustomAuthentication custom) {
            return new AuthorizationDecision(custom.isMfaVerified());
        }
        return new AuthorizationDecision(false);
    };
}
```

**5. Pitfalls:**

- Exposing actuator on same port as application
- /env endpoint showing secrets
- /heapdump downloadable without auth
- Not using EndpointRequest matchers (path changes break security)

**6. Follow-up questions:**

- How do you expose actuator only to internal network?
- Which actuator endpoints are most dangerous?

**7. Senior summary:** "Secure actuator with separate SecurityFilterChain using EndpointRequest matchers. Health/info can be public for probes; everything else needs authentication. Use separate management port, hide sensitive endpoint values, and restrict heapdump/threaddump to super admins only."

---

# Question 97: What is OncePerRequestFilter?

**1. Definition:** `OncePerRequestFilter` is a base class ensuring filter logic executes exactly once per request, even if the request is forwarded or included internally.

**2. Why it matters:** Standard filters can execute multiple times on internal forwards/includes. OncePerRequestFilter prevents duplicate processing for authentication, logging, and other once-per-request logic.

**3. Internal working:**

```
public abstract class OncePerRequestFilter extends GenericFilterBean {

    @Override
    public final void doFilter(ServletRequest request, ServletResponse response,
                               FilterChain chain) {
        String alreadyFilteredAttribute = getAlreadyFilteredAttributeName();

        if (request.getAttribute(alreadyFilteredAttribute) != null) {
            // Already executed - skip
            chain.doFilter(request, response);
        } else {
            // Mark as executed
            request.setAttribute(alreadyFilteredAttribute, Boolean.TRUE);
            try {
                doFilterInternal((HttpServletRequest) request,
                                 (HttpServletResponse) response, chain);
            } finally {
                request.removeAttribute(alreadyFilteredAttribute);
            }
        }
    }

    protected abstract void doFilterInternal(HttpServletRequest request,
                                             HttpServletResponse response,
                                             FilterChain chain);
}
```

**4. Real-world example:** JWT authentication filter:

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);

            try {
                Claims claims = jwtService.validateToken(token);

                UserDetails userDetails = userDetailsService
                    .loadUserByUsername(claims.getSubject());

                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());

                auth.setDetails(new WebAuthenticationDetailsSource()
                    .buildDetails(request));

                SecurityContextHolder.getContext().setAuthentication(auth);

            } catch (JwtException e) {
                logger.debug("Invalid JWT token: {}", e.getMessage());
            }
        }

        chain.doFilter(request, response);
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        // Skip filter for certain paths
        return request.getServletPath().startsWith("/public/");
    }
}
```

**5. Pitfalls:**

- Extending Filter instead (runs multiple times on forwards)
- Not calling chain.doFilter() (breaks chain)
- Heavy processing without shouldNotFilter optimization
- Throwing exceptions instead of continuing chain for optional auth

**6. Follow-up questions:**

- When would doFilter be called multiple times for one request?
- How do you skip the filter for certain paths?

**7. Senior summary:** "OncePerRequestFilter guarantees exactly-once execution per request, handling forwards/includes automatically. Use for authentication, logging, and other once-per-request logic. Override shouldNotFilter() to skip paths. Always continue the chain—don't block for optional processing."

---

# Question 98: Session management in Spring Security?

**1. Definition:** Session management controls session creation, concurrency, fixation protection, and timeout—critical for session-based authentication security.

**2. Why it matters:** Poor session management leads to session hijacking, fixation attacks, and resource exhaustion from too many concurrent sessions.

**3. Internal working:**

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .sessionManagement(session -> session
            // Session creation policy
            .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)

            // Session fixation protection
            .sessionFixation(fixation -> fixation.migrateSession())

            // Concurrent sessions
            .maximumSessions(1)
            .maxSessionsPreventsLogin(false)  // Expire old session
            .expiredUrl("/login?expired")
            .sessionRegistry(sessionRegistry())
        )
        .logout(logout -> logout
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
        );

    return http.build();
}

@Bean
public SessionRegistry sessionRegistry() {
    return new SessionRegistryImpl();
}
```

Session creation policies:

- `ALWAYS`: Always create session
- `IF_REQUIRED`: Create if needed (default)
- `NEVER`: Never create, use existing
- `STATELESS`: Never create or use

**4. Real-world example:** Complete session management:

```java
@Configuration
@EnableWebSecurity
public class SessionSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                .sessionFixation().migrateSession()
                .maximumSessions(2)
                .maxSessionsPreventsLogin(true)  // Block new login
                .expiredSessionStrategy(event -> {
                    HttpServletResponse response = event.getResponse();
                    response.setStatus(401);
                    response.getWriter().write("Session expired");
                })
                .sessionRegistry(sessionRegistry())
            )
            .rememberMe(remember -> remember
                .tokenRepository(persistentTokenRepository())
                .tokenValiditySeconds(14 * 24 * 60 * 60)  // 14 days
            );

        return http.build();
    }

    // For listing active sessions
    @Bean
    public SessionRegistry sessionRegistry() {
        return new SessionRegistryImpl();
    }

    // Admin endpoint to view/kill sessions
    @GetMapping("/admin/sessions")
    @PreAuthorize("hasRole('ADMIN')")
    public List<SessionInfo> getActiveSessions() {
        return sessionRegistry.getAllPrincipals().stream()
            .flatMap(principal -> sessionRegistry
                .getAllSessions(principal, false).stream())
            .map(SessionInfo::from)
            .toList();
    }

    @DeleteMapping("/admin/sessions/{sessionId}")
    @PreAuthorize("hasRole('ADMIN')")
    public void expireSession(@PathVariable String sessionId) {
        sessionRegistry.getSessionInformation(sessionId).expireNow();
    }
}
```

**5. Pitfalls:**

- Session fixation not enabled (vulnerable to attack)
- Unlimited concurrent sessions (resource exhaustion)
- Remember-me without persistent token (less secure)
- Not invalidating session on logout

**6. Follow-up questions:**

- What is session fixation attack?
- How do you share sessions across multiple servers?

**7. Senior summary:** "Session management controls creation, concurrency, and fixation protection. migrateSession() prevents fixation attacks. Limit concurrent sessions with maximumSessions(). Use SessionRegistry for admin session management. For clusters, use Redis-backed session store."

---

# Question 99: Token expiration handling?

**1. Definition:** Token expiration handling involves setting appropriate expiry times, detecting expired tokens, and providing refresh mechanisms to maintain user sessions without re-authentication.

**2. Why it matters:** Short-lived tokens limit damage from theft. Proper expiration handling balances security with user experience—users shouldn't re-login frequently.

**3. Internal working:**

```
// Token with expiration
public String generateToken(UserDetails user) {
    Date now = new Date();
    Date expiry = new Date(now.getTime() + ACCESS_TOKEN_VALIDITY);

    return Jwts.builder()
        .subject(user.getUsername())
        .issuedAt(now)
        .expiration(expiry)
        .signWith(secretKey)
        .compact();
}

// Validation with expiry check
public Claims validateToken(String token) {
    try {
        return Jwts.parser()
            .verifyWith(secretKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    } catch (ExpiredJwtException e) {
        throw new TokenExpiredException("Token has expired");
    } catch (JwtException e) {
        throw new InvalidTokenException("Invalid token");
    }
}
```

**4. Real-world example:** Complete token lifecycle management:

```java
@Service
public class TokenService {

    private static final long ACCESS_TOKEN_VALIDITY = 15 * 60 * 1000;  // 15 mins
    private static final long REFRESH_TOKEN_VALIDITY = 7 * 24 * 60 * 60 * 1000;  // 7 days

    public TokenPair generateTokenPair(User user) {
        String accessToken = generateAccessToken(user);
        String refreshToken = generateRefreshToken(user);

        // Store refresh token for revocation capability
        refreshTokenRepository.save(new RefreshToken(
            user.getId(),
            hash(refreshToken),
            Instant.now().plusMillis(REFRESH_TOKEN_VALIDITY)
        ));

        return new TokenPair(accessToken, refreshToken);
    }

    public TokenPair refreshAccessToken(String refreshToken) {
        // Validate refresh token
        Claims claims = validateRefreshToken(refreshToken);

        // Check if token is revoked
        if (!refreshTokenRepository.existsByUserIdAndHash(
                claims.get("userId", Long.class), hash(refreshToken))) {
            throw new TokenRevokedException("Refresh token has been revoked");
        }

        // Generate new pair (refresh token rotation)
        User user = userService.findById(claims.get("userId", Long.class));
        refreshTokenRepository.deleteByHash(hash(refreshToken));  // Revoke old
        return generateTokenPair(user);
    }

    public void revokeAllTokens(Long userId) {
        refreshTokenRepository.deleteAllByUserId(userId);
    }
}

@RestController
public class AuthController {

    @PostMapping("/auth/refresh")
    public ResponseEntity<TokenResponse> refresh(
            @CookieValue("refresh_token") String refreshToken) {
        try {
            TokenPair tokens = tokenService.refreshAccessToken(refreshToken);
            return ResponseEntity.ok()
                .header(HttpHeaders.SET_COOKIE, createRefreshCookie(tokens.refreshToken()))
                .body(new TokenResponse(tokens.accessToken()));
        } catch (TokenExpiredException e) {
            return ResponseEntity.status(401)
                .body(new TokenResponse(null, "Refresh token expired"));
        }
    }
}
```

**5. Pitfalls:**

- Access tokens too long-lived (security risk)
- Refresh tokens without rotation (theft risk)
- No token revocation mechanism
- Storing refresh token in localStorage (XSS vulnerable)

**6. Follow-up questions:**

- What's refresh token rotation?
- How do you handle token expiration in SPAs?

**7. Senior summary:** "Short-lived access tokens (15 mins) with longer refresh tokens (days). Store refresh tokens server-side for revocation. Rotate refresh tokens on use. Handle expiration gracefully—silent refresh in SPA, redirect for server-rendered apps. Use httpOnly cookies for refresh tokens."

---

# Question 100: Refresh token flow?

**1. Definition:** Refresh token flow uses a long-lived refresh token to obtain new access tokens without re-authentication, maintaining user sessions while keeping access tokens short-lived.

**2. Why it matters:** It balances security (short access token lifetime) with UX (users don't re-authenticate frequently). Enables token revocation and session management.

**3. Internal working:**

```
1. User logs in with credentials
2. Server returns:
   - Short-lived access token (15 mins)
   - Long-lived refresh token (7 days)
3. Access token used for API calls
4. When access token expires (401):
   - Client sends refresh token to /auth/refresh
   - Server validates refresh token
   - Server issues new access token (and optionally new refresh token)
5. If refresh token expired → user must re-authenticate
```

```
// Token storage recommendations
// Access token: Memory (JS variable) - never localStorage
// Refresh token: httpOnly cookie - not accessible to JS
```

**4. Real-world example:** Secure refresh implementation:

```java
@RestController
@RequestMapping("/auth")
public class AuthController {

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> login(@RequestBody @Valid LoginRequest request) {
        Authentication auth = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.email(), request.password())
        );

        User user = (User) auth.getPrincipal();
        TokenPair tokens = tokenService.generateTokenPair(user);

        ResponseCookie refreshCookie = ResponseCookie
            .from("refresh_token", tokens.refreshToken())
            .httpOnly(true)
            .secure(true)
            .sameSite("Strict")
            .path("/auth/refresh")
            .maxAge(Duration.ofDays(7))
            .build();

        return ResponseEntity.ok()
            .header(HttpHeaders.SET_COOKIE, refreshCookie.toString())
            .body(new LoginResponse(tokens.accessToken(), user.toDto()));
    }

    @PostMapping("/refresh")
    public ResponseEntity<?> refresh(
            @CookieValue(name = "refresh_token", required = false) String refreshToken) {

        if (refreshToken == null) {
            return ResponseEntity.status(401).body("No refresh token");
        }

        try {
            TokenPair tokens = tokenService.refreshAccessToken(refreshToken);

            // Rotate refresh token
            ResponseCookie newRefreshCookie = ResponseCookie
                .from("refresh_token", tokens.refreshToken())
                .httpOnly(true)
                .secure(true)
                .sameSite("Strict")
                .path("/auth/refresh")
                .maxAge(Duration.ofDays(7))
                .build();

            return ResponseEntity.ok()
                .header(HttpHeaders.SET_COOKIE, newRefreshCookie.toString())
                .body(new RefreshResponse(tokens.accessToken()));

        } catch (TokenException e) {
            // Clear invalid cookie
            ResponseCookie clearCookie = ResponseCookie
                .from("refresh_token", "")
                .maxAge(0)
                .build();

            return ResponseEntity.status(401)
                .header(HttpHeaders.SET_COOKIE, clearCookie.toString())
                .body("Invalid refresh token");
        }
    }
```

```
    @PostMapping("/logout")
    public ResponseEntity<?> logout(
            @CookieValue(name = "refresh_token", required = false) String refreshToken) {

        if (refreshToken != null) {
            tokenService.revokeToken(refreshToken);
        }

        ResponseCookie clearCookie = ResponseCookie
            .from("refresh_token", "")
            .maxAge(0)
            .path("/auth/refresh")
            .build();

        return ResponseEntity.ok()
            .header(HttpHeaders.SET_COOKIE, clearCookie.toString())
            .body("Logged out");
    }
}
```

SPA client handling:

```
// Axios interceptor for automatic refresh
axios.interceptors.response.use(
    response => response,
    async error => {
        if (error.response?.status === 401 && !error.config._retry) {
            error.config._retry = true;

            try {
                const response = await axios.post('/auth/refresh');
                // New access token in response
                setAccessToken(response.data.accessToken);

                // Retry original request
                error.config.headers.Authorization =
                    `Bearer ${response.data.accessToken}`;
                return axios(error.config);
            } catch (refreshError) {
                // Refresh failed - redirect to login
                window.location.href = '/login';
            }
        }
        return Promise.reject(error);
    }
);
```

**5. Pitfalls:**

- Refresh token in localStorage (XSS theft)
- No token rotation (stolen token valid until expiry)
- Refresh endpoint not rate-limited
- Not invalidating refresh tokens on password change

**6. Follow-up questions:**

- What is refresh token rotation and why is it important?
- How do you handle refresh token theft?

**7. Senior summary:** "Refresh tokens extend sessions without re-authentication. Store in httpOnly cookies, rotate on use, store server-side for revocation. Implement silent refresh in SPA with axios interceptor. Invalidate all refresh tokens on password change or logout."

---

# Question 101: Spring Security best practices?

**1. Definition:** Security best practices include defense in depth, least privilege, secure defaults, proper secret management, and regular security audits.

**2. Why it matters:** Security is only as strong as the weakest link. Following best practices systematically reduces attack surface and limits breach impact.

**3. Internal working:**

**Authentication:**

```
// Use strong password encoding
@Bean
public PasswordEncoder passwordEncoder() {
    return Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8();
}

// Account lockout after failed attempts
// Rate limiting on authentication endpoints
// MFA for sensitive operations
```

**Authorization:**

```
// Default deny, explicit allow
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/public/**").permitAll()
    .anyRequest().authenticated()  // Default: deny
);

// Method-level security for sensitive operations
@PreAuthorize("hasRole('ADMIN') and @accessChecker.canModify(#id)")
public void modifyResource(Long id) { }
```

**4. Real-world example:** Production security configuration:

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class ProductionSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // Headers
            .headers(headers -> headers
                .contentSecurityPolicy(csp -> csp
                    .policyDirectives("default-src 'self'; script-src 'self'"))
                .frameOptions(frame -> frame.deny())
                .httpStrictTransportSecurity(hsts -> hsts
                    .maxAgeInSeconds(31536000)
                    .includeSubDomains(true))
            )

            // CSRF for session-based
            .csrf(csrf -> csrf
                .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()))

            // Session security
            .sessionManagement(session -> session
                .sessionFixation().migrateSession()
                .maximumSessions(3)
                .sessionRegistry(sessionRegistry()))

            // Authentication
            .formLogin(form -> form
                .loginPage("/login")
                .failureHandler(authenticationFailureHandler())
                .successHandler(authenticationSuccessHandler()))

            // Logout
            .logout(logout -> logout
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID", "remember-me")
                .clearAuthentication(true))

            // Exception handling
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(customEntryPoint())
                .accessDeniedHandler(customAccessDenied()));

        return http.build();
    }

    // Log failed attempts
    @Bean
    public AuthenticationFailureHandler authenticationFailureHandler() {
        return (request, response, exception) -> {
            String ip = request.getRemoteAddr();
            String username = request.getParameter("username");
            securityAuditService.logFailedLogin(ip, username, exception.getMessage());

            response.sendRedirect("/login?error");
        };
    }
}
```

**Checklist:**

- [ ] HTTPS everywhere (HSTS enabled)

- [ ] Strong password policy and encoding
- [ ] Account lockout and rate limiting
- [ ] CSRF protection for session-based apps
- [ ] Secure session management
- [ ] Proper CORS configuration
- [ ] Security headers (CSP, X-Frame-Options, etc.)
- [ ] Input validation and output encoding
- [ ] Secure secret management (not in code)
- [ ] Regular dependency updates
- [ ] Security logging and monitoring
- [ ] Actuator endpoints secured
- [ ] Error messages don't leak info

**5. Pitfalls:**

- Security through obscurity
- Overly permissive CORS
- Secrets in source control
- Not updating dependencies
- No security logging

**6. Follow-up questions:**

- How do you audit security configurations?
- What's your approach to security testing?

**7. Senior summary:** "Defense in depth: multiple security layers. Secure defaults with explicit exceptions. Store secrets in vault, not code. Enable security headers, enforce HTTPS. Log security events, update dependencies regularly. Regular security audits and penetration testing.""

# Berribot Assessment - Interview Preparation

## Profile: Senior Full Stack Developer (4.8+ Years)

### Tech Stack: Java 17, Spring Boot, Angular 16, MongoDB, MySQL, Microservices, AWS S3

---

# SECTION 1: VIDEO INTERVIEW QUESTIONS (MOST IMPORTANT)

---

## Q1: Tell Me About Yourself

**Best Answer (60-90 seconds):**

"I'm a Senior Full Stack Developer with nearly 5 years of experience building scalable enterprise applications using Java, Spring Boot, and Angular.

Currently at Techversant Infotech, I lead the development of RoboGebra.ai, an AI-powered educational platform. I architected the backend using Spring Boot 3.2 with microservices, integrated OpenAI APIs with intelligent caching that reduced API costs by 40%, and built a responsive Angular 16 frontend.

One of my key achievements was leading the migration from Spring Boot 2 to 3.2 and Angular 4 to 16 across multiple projects—handling breaking changes, updating deprecated APIs, and ensuring zero production downtime.

I'm passionate about writing clean, maintainable code and solving complex problems. I've worked extensively with MongoDB for flexible data models, MySQL for transactional systems, and AWS S3 for scalable file storage.

I'm now looking for opportunities where I can contribute to challenging projects while growing as a technical leader."

---

## Q2: Explain One Challenging Project (RoboGebra.ai)

**Best Answer:**

"RoboGebra.ai is an AI-powered educational platform that helps students solve mathematical problems with step-by-step explanations.

**The Challenge:** We needed to integrate OpenAI's API to generate solutions, but each API call was expensive and slow—around 2-3 seconds per request. With thousands of students using the platform, this created both cost and performance issues.

**My Contribution:**

1. **Architecture Design:** I designed the backend using Spring Boot 3.2 with a clean microservices approach—separate services for user management, problem processing, and AI integration.
2. **Smart Caching Solution:** I implemented a multi-layer caching strategy:
    - Redis cache for frequently asked questions
    - Hash-based similarity matching so similar problems hit the cache
    - Result: 40% reduction in API calls and response time dropped from 2.5s to 800ms for cached queries.
3. **AWS S3 Integration:** Students could upload problem images. I integrated AWS S3 with pre-signed URLs for secure uploads, automatic cleanup policies, and CDN integration for fast delivery.
4. **Database Optimization:** Used MongoDB for flexible problem storage with compound indexes on problem categories and difficulty levels, improving query performance by 60%.

**Result:** The platform now handles 10,000+ daily users with 99.9% uptime. API costs reduced by 40%, and average response time improved by 65%."

---

## Follow-up: Why Spring Boot?

"I chose Spring Boot for three reasons:

1. **Rapid Development:** Auto-configuration and starter dependencies let us focus on business logic rather than boilerplate setup.
2. **Production-Ready:** Built-in actuator endpoints for health checks, metrics, and monitoring—critical for a production system with thousands of users.
3. **Ecosystem:** Excellent integration with MongoDB, Redis, and AWS SDKs. Spring Security made it easy to implement JWT authentication.

We started with Spring Boot 2 and later migrated to 3.2 to leverage Java 17 features like records and pattern matching, which made our DTOs cleaner and more maintainable."

---

## Follow-up: How Did AI Caching Help Reduce Cost?

"OpenAI charges per token—roughly $0.002 per 1K tokens. With thousands of daily requests, costs were adding up quickly.

**My Solution:**

1. **Content-Based Hashing:** I created a normalized hash of each math problem. Before calling OpenAI, we check if a similar problem exists in Redis cache.
2. **Semantic Similarity:** For slightly different wordings of the same problem, I implemented a similarity threshold using problem structure analysis.
3. **TTL Strategy:** Cache entries expire after 7 days, balancing freshness with cost savings.

**Implementation:**

```
@Cacheable(value = "aiResponses", key = "#problemHash", unless = "#result == null")
public AIResponse getAISolution(String problem) {
    String problemHash = generateNormalizedHash(problem);
    // Check cache first, then call OpenAI if miss
}
```

**Result:** 40% of requests hit cache, saving approximately $500/month in API costs while improving response time significantly."

---

# Q3: Migration Experience (Spring Boot 2 → 3.2, Angular 4 → 16)

**Best Answer:**

"I led two major migrations that were critical for keeping our applications modern and secure.

**Spring Boot 2 to 3.2 Migration:**

**Challenges Faced:**

1. **Jakarta EE Namespace Change:** The biggest breaking change—all `javax.*` packages moved to `jakarta.*`. This affected every controller, service, and configuration class.

2. **Spring Security Rewrite:** `WebSecurityConfigurerAdapter` was removed. I rewrote all security configurations using the new `SecurityFilterChain` bean approach.

3. **Property Changes:** Several property names changed (e.g., `spring.redis.*` to `spring.data.redis.*`).

**How I Resolved Them:**

1. **Phased Approach:** I created a migration branch and tackled changes module by module rather than all at once.

2. **Automated Refactoring:** Used IntelliJ's structural search to batch-replace `javax` imports with `jakarta`.

3. **Comprehensive Testing:** Wrote integration tests before migration to ensure behavior remained consistent. Ran full regression after each phase.

4. **Dependency Matrix:** Created a spreadsheet tracking every dependency version change and verified compatibility.

**Angular 4 to 16 Migration:**

**Challenges:**

1. **RxJS Breaking Changes:** Migrated from RxJS 5 to 7—changed `map`, `filter` to pipeable operators.

2. **Standalone Components:** Gradually introduced standalone components in Angular 14+.

3. **Module Structure:** Refactored to use lazy loading for better performance.

**Resolution:**

1. **Incremental Updates:** Upgraded version by version (4→8→12→16) rather than jumping directly.

2. **ng update:** Used Angular CLI's update schematics which handled most breaking changes automatically.

3. **ESLint Migration:** Moved from TSLint to ESLint with Angular-specific rules.

**Result:** Both migrations completed with zero production downtime. The Spring Boot 3.2 upgrade improved startup time by 30%, and Angular 16 reduced bundle size by 25%."

---

# Q4: AWS S3 Integration

**Best Answer:**

"I integrated AWS S3 in RoboGebra.ai for handling student-uploaded problem images and generated solution PDFs.

**Problems It Solved:**

1. **Scalability:** Local storage couldn't handle growing file volumes. S3 provides unlimited storage with 99.999999999% durability.

2. **Cost:** Pay only for what you use, with lifecycle policies to archive old files to Glacier.

3. **Performance:** CloudFront CDN integration reduced image load times by 60% for global users.

**Implementation:**

```
@Service
public class S3StorageService {

    private final S3Client s3Client;

    public String uploadFile(MultipartFile file, String folder) {
        String key = folder + "/" + UUID.randomUUID() + getExtension(file);

        PutObjectRequest request = PutObjectRequest.builder()
            .bucket(bucketName)
            .key(key)
            .contentType(file.getContentType())
            .build();

        s3Client.putObject(request, RequestBody.fromInputStream(
            file.getInputStream(), file.getSize()));

        return generatePresignedUrl(key);
    }
}
```

**Security Measures:**

1. **Pre-signed URLs:** Files are private by default. I generate time-limited pre-signed URLs (15 minutes) for authorized access.
2. **IAM Roles:** Application uses IAM role with minimal permissions—only `PutObject` and `GetObject` on specific bucket prefix.
3. **Bucket Policies:** Block public access, enforce HTTPS-only, enable server-side encryption (SSE-S3).
4. **Content Validation:** Server-side validation of file types and size limits before upload.

**Why S3 Over Local Storage:**

| Aspect | Local Storage | AWS S3 |
|---|---|---|
| Scalability | Limited by disk | Unlimited |
| Availability | Single point of failure | 99.99% SLA |
| Cost | Fixed infrastructure | Pay-per-use |
| CDN | Manual setup | CloudFront integration |
| Backup | Manual | Built-in versioning |

**Result:** Handled 50,000+ file uploads with zero data loss, reduced storage costs by 40% using lifecycle policies."

---

# Q5: Agile & Teamwork

**Best Answer:**

"I've worked in Agile Scrum for the past 4 years and genuinely believe it improves both code quality and team collaboration.

**My Agile Practices:**

**Sprint Planning:**

- I actively participate in estimation using story points
- I break down complex features into smaller, deliverable user stories
- I identify technical dependencies early and communicate blockers

**Daily Standups:**

- I keep updates concise: what I did, what I'll do, any blockers
- I proactively offer help when teammates mention challenges in my area of expertise
- If I'm blocked, I don't wait—I escalate immediately

**Code Reviews:**

- I review PRs daily, providing constructive feedback on design, performance, and maintainability
- I use reviews as mentoring opportunities for junior developers

**Sprint Retrospectives:**

- I suggest process improvements—for example, I proposed adding automated integration tests to our CI pipeline, which caught 30% more bugs before deployment

**Ownership Example:** In one sprint, a critical feature was at risk because the assigned developer was sick. I took ownership, worked extra hours, and delivered it on time. I documented everything so the original developer could maintain it easily."

---

# Q6: Production Issue Handling

**Best Answer:**

"Let me share a critical production issue I handled recently.

**The Situation:** On a Friday evening, our monitoring alerted that API response times had spiked from 200ms to 5+ seconds. Users were complaining about timeouts.

**My Immediate Actions:**

1. **Triage (First 5 minutes):**
   - Checked Grafana dashboards—identified MongoDB queries as the bottleneck
   - Verified no recent deployments that could have caused it
   - CPU and memory were normal, ruling out infrastructure issues
2. **Root Cause Analysis (Next 15 minutes):**
   - Enabled slow query logging in MongoDB
   - Found a collection scan happening on a new feature's query that was missing an index
   - A developer had added a new filter without creating the supporting index
3. **Immediate Fix:**
   - Created the missing compound index:

```
db.problems.createIndex({ "category": 1, "difficulty": 1, "createdAt": -1 })
```

   - Response times normalized within minutes
4. **Prevention:**
   - Added query explain analysis to our code review checklist
   - Implemented automated index recommendation alerts in our monitoring

**Communication:**

- Immediately notified the team lead and product manager about the issue and ETA for fix
- Posted incident summary in Slack with root cause and prevention measures
- Created a postmortem document for the team

**Result:** Issue resolved in 25 minutes. Implemented automated monitoring to prevent similar issues. No data loss or customer impact beyond temporary slowness."

---

# SECTION 2: TECHNICAL QUESTIONS

---

# Core Java (Java 17 Focus)

## Q: What are the new features in Java 17?

**Answer:**

"Java 17 is an LTS release with several important features I've used in production:

1. **Sealed Classes:** Control which classes can extend a class

```
public sealed class Shape permits Circle, Rectangle, Triangle {}
```

2. **Pattern Matching for instanceof:**

```
if (obj instanceof String s) {
    System.out.println(s.length()); // No casting needed
}
```

3. **Records:** Immutable data carriers—I use these extensively for DTOs

```
public record UserDTO(Long id, String name, String email) {}
```

4. **Text Blocks:** Multi-line strings for SQL, JSON templates

```
String json = """
    {
        "name": "John",
        "age": 30
    }
    """;
```

5. **Switch Expressions:** Cleaner switch with arrow syntax and yield

```
String result = switch (day) {
    case MONDAY, FRIDAY -> "Working";
    case SATURDAY, SUNDAY -> "Weekend";
    default -> "Midweek";
};
```

I migrated our codebase to use records for all DTOs, reducing boilerplate by 40%."

---

# Q: Difference between HashMap and ConcurrentHashMap

**Answer:**

| Aspect | HashMap | ConcurrentHashMap |
|---|---|---|
| Thread Safety | Not thread-safe | Thread-safe |
| Null Keys/Values | Allows one null key, multiple null values | No nulls allowed |
| Locking | N/A | Segment-level (bucket) locking |
| Performance | Faster in single-threaded | Better in multi-threaded |
| Iterator | Fail-fast | Weakly consistent |

**When to use:**

- **HashMap:** Single-threaded scenarios, local method variables
- **ConcurrentHashMap:** Shared caches, concurrent access scenarios

**Real Example:** In our caching layer, I used ConcurrentHashMap for an in-memory rate limiter:

```
private final ConcurrentHashMap<String, AtomicInteger> requestCounts = new ConcurrentHashMap<>();

public boolean allowRequest(String userId) {
    return requestCounts
        .computeIfAbsent(userId, k -> new AtomicInteger(0))
        .incrementAndGet() <= MAX_REQUESTS;
}
```

---

# Q: How does Garbage Collection work?

**Answer:**

"JVM automatically manages memory through garbage collection:

**Generational Model:**

1. **Young Generation (Eden + Survivor):** New objects created here. Minor GC collects frequently.
2. **Old Generation (Tenured):** Long-lived objects promoted here. Major GC less frequent but slower.
3. **Metaspace:** Class metadata (replaced PermGen in Java 8)

**GC Process:**

1. **Mark:** Identify reachable objects from GC roots
2. **Sweep:** Remove unreachable objects
3. **Compact:** Defragment memory (some collectors)

**Common Collectors:**

- **G1GC:** Default in Java 9+, good balance of latency and throughput
- **ZGC:** Low latency (<10ms pauses), good for large heaps
- **Parallel GC:** High throughput, longer pauses

**In Production:** I've tuned GC for our Spring Boot services:

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:+ParallelRefProcEnabled
```

Monitoring via Actuator metrics helped identify memory leaks in a long-running service."

---

# Q: Why is String immutable?

**Answer:**

"String is immutable in Java for several important reasons:

1. **Security:** Strings are used for sensitive data (passwords, connection strings). Immutability prevents malicious modification.

2. **Thread Safety:** Immutable objects are inherently thread-safe—no synchronization needed.

3. **String Pool (Interning):** JVM caches string literals. Immutability makes this caching safe and memory-efficient.

4. **Hashcode Caching:** String caches its hashcode. Used extensively as HashMap keys—immutability ensures hash consistency.

5. **Class Loading:** Class names are strings. Immutability prevents security vulnerabilities during class loading.

**Practical Implication:** String concatenation in loops creates many objects. I use StringBuilder for performance:

```
// Bad - creates many String objects
String result = "";
for (String s : list) {
    result += s;
}

// Good - single StringBuilder
StringBuilder sb = new StringBuilder();
for (String s : list) {
    sb.append(s);
}
```"

---

## Spring Boot & Microservices

### Q: What is global exception handling?

**Answer:**

"Global exception handling centralizes error handling across all controllers using `@ControllerAdvice`, ensuring consistent error re

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            "NOT_FOUND",
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage
            ));
        return ResponseEntity.badRequest()
            .body(new ErrorResponse("VALIDATION_ERROR", errors));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneral(Exception ex) {
        log.error("Unexpected error", ex);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred"));
    }
}
```

**Benefits:**

- Consistent error format across APIs
- Centralized logging
- No try-catch clutter in controllers
- Security—never expose stack traces to clients"

---

# Q: How do you secure REST APIs?

**Answer:**

"I implement multi-layered security:

**1. Authentication (JWT):**

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())  // Stateless API
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

**2. Authorization (Method-level):**

```
@PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
public User getUser(Long userId) { }
```

**3. Input Validation:**

```
public ResponseEntity<?> createUser(@RequestBody @Valid UserRequest request)
```

**4. Rate Limiting:** Prevent abuse with request throttling per user/IP.

**5. HTTPS Only:** All production traffic encrypted.

**6. Security Headers:**

```
.headers(headers -> headers
    .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'"))
    .frameOptions(frame -> frame.deny()))
```

**7. Audit Logging:** Log all authentication attempts and sensitive operations."

---

# Q: What happens when a microservice fails?

**Answer:**

"I implement resilience patterns to handle microservice failures gracefully:

**1. Circuit Breaker (Resilience4j):**

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
public PaymentResponse processPayment(PaymentRequest request) {
    return paymentClient.process(request);
}

public PaymentResponse paymentFallback(PaymentRequest request, Exception e) {
    // Queue for retry or return cached response
    return PaymentResponse.pending("Payment queued for processing");
}
```

**2. Retry with Backoff:**

```
@Retry(name = "inventoryService", maxAttempts = 3)
public InventoryResponse checkInventory(String productId) { }
```

**3. Timeout:**

```
@TimeLimiter(name = "shippingService", fallbackMethod = "shippingFallback")
public CompletableFuture<ShippingRate> getShippingRate() { }
```

**4. Bulkhead:** Isolate failures—limit concurrent calls to prevent cascade.

**5. Fallback Strategies:**

- Return cached data
- Default response
- Queue for async retry
- Graceful degradation (partial functionality)

**In Production:** We had a payment service outage. Circuit breaker opened within 5 failed requests, fallback queued transactions, and users saw 'Payment processing' instead of errors. When service recovered, queued payments processed automatically."

---

# MongoDB & MySQL

## Q: When do you choose MongoDB over MySQL?

**Answer:**

| Factor | Choose MongoDB | Choose MySQL |
|---|---|---|
| Schema | Flexible, evolving | Fixed, well-defined |
| Relationships | Few, embedded docs | Complex joins |
| Scale | Horizontal (sharding) | Vertical primarily |
| Transactions | Document-level (multi-doc in 4.0+) | ACID transactions |
| Data Type | JSON-like documents | Structured rows |
| Use Case | Content, catalogs, logs | Financial, inventory |

**In RoboGebra.ai:**

- **MongoDB:** Problem bank with varying structures, user activity logs, AI responses (flexible schema)
- **MySQL:** User accounts, subscriptions, payments (relational integrity, transactions)

**Example Decision:** Math problems have different attributes—algebra has equations, geometry has shapes. MongoDB's flexible schema handles this naturally:

```
// Algebra problem
{ type: "algebra", equation: "2x + 5 = 15", variables: ["x"] }

// Geometry problem
{ type: "geometry", shape: "triangle", sides: [3, 4, 5], angles: [90, 53, 37] }
```

In MySQL, this would require complex inheritance tables or JSON columns."

---

## Q: How does indexing improve performance?

**Answer:**

"Indexes are data structures that allow the database to find data without scanning every row.

**Without Index:** Full table scan—O(n) time **With Index:** B-tree lookup—O(log n) time

**Example:**

```
-- Without index: scans 1 million rows
SELECT * FROM orders WHERE customer_id = 123;  -- 2000ms

-- After adding index
CREATE INDEX idx_customer_id ON orders(customer_id);
-- Same query: 5ms
```

**MongoDB Compound Index:**

```
db.problems.createIndex({ category: 1, difficulty: 1, createdAt: -1 })
```

This optimizes queries filtering by category AND difficulty, sorted by date.

**Best Practices I Follow:**

1. Index columns used in WHERE, JOIN, ORDER BY
2. Use compound indexes for multi-column queries
3. Avoid over-indexing (slows writes)
4. Monitor slow queries and add indexes accordingly
5. Use EXPLAIN to verify index usage

**Real Impact:** Added a compound index in our problem collection—query time dropped from 800ms to 15ms for filtered searches."

---

# Angular 16

## Q: What are standalone components?

**Answer:**

"Standalone components in Angular 14+ don't require NgModule declaration—they're self-contained.

**Traditional:**

```
@NgModule({
    declarations: [UserComponent],
    imports: [CommonModule, FormsModule],
    exports: [UserComponent]
})
export class UserModule {}
```

**Standalone:**

```
@Component({
    selector: 'app-user',
    standalone: true,
    imports: [CommonModule, FormsModule],  // Import directly
    template: `...`
})
export class UserComponent {}
```

**Benefits:**

1. **Reduced Boilerplate:** No NgModule needed
2. **Better Tree Shaking:** Unused code eliminated more effectively
3. **Simpler Lazy Loading:**

```
// Route-level lazy loading
{ path: 'user', loadComponent: () => import('./user.component').then(m => m.UserComponent) }
```

4. **Easier Testing:** Component is self-contained

**Migration Strategy:** In our Angular 16 migration, I gradually converted components to standalone, starting with leaf components and working up to features."

---

## Q: Difference between Observables and Promises

**Answer:**

| Aspect | Promise | Observable |
|---|---|---|
| Values | Single value | Multiple values over time |
| Execution | Eager (runs immediately) | Lazy (runs on subscribe) |
| Cancellation | Not cancellable | Cancellable (unsubscribe) |
| Operators | Limited (.then, .catch) | Rich (map, filter, debounce, etc.) |

**When to Use:**

- **Promise:** Single async result (HTTP request, file read)
- **Observable:** Streams (user input, WebSocket, multiple events)

**Example - Search with Debounce:**

```
// Observable power - impossible with Promises
this.searchInput.valueChanges.pipe(
    debounceTime(300),           // Wait 300ms after typing stops
    distinctUntilChanged(),       // Only if value changed
    switchMap(term => this.api.search(term))  // Cancel previous request
).subscribe(results => this.results = results);
```

**HTTP in Angular:**

```
// Observable - can be cancelled, transformed
this.http.get<User[]>('/api/users').pipe(
    map(users => users.filter(u => u.active)),
    catchError(err => of([]))  // Return empty on error
).subscribe();
```

I always unsubscribe in ngOnDestroy to prevent memory leaks."

---

# SECTION 3: CODING QUESTIONS

---

## Q: Reverse a string without built-in methods

```java
public String reverseString(String input) {
    if (input == null || input.length() <= 1) {
        return input;
    }

    char[] chars = input.toCharArray();
    int left = 0;
    int right = chars.length - 1;

    while (left < right) {
        // Swap characters
        char temp = chars[left];
        chars[left] = chars[right];
        chars[right] = temp;

        left++;
        right--;
    }

    return new String(chars);
}

// Test
// Input: "hello" -> Output: "olleh"
// Input: "a" -> Output: "a"
// Input: null -> Output: null
```

---

## Q: Find duplicate elements in a list

```java
public List<Integer> findDuplicates(List<Integer> numbers) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = new HashSet<>();

    for (Integer num : numbers) {
        if (!seen.add(num)) {  // add() returns false if already exists
            duplicates.add(num);
        }
    }

    return new ArrayList<>(duplicates);
}

// Alternative using Stream API
public List<Integer> findDuplicatesStream(List<Integer> numbers) {
    return numbers.stream()
        .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .toList();
}

// Test
// Input: [1, 2, 3, 2, 4, 3, 5] -> Output: [2, 3]
```

---

## Q: Count word frequency

```
public Map<String, Long> countWordFrequency(String text) {
    if (text == null || text.isBlank()) {
        return Collections.emptyMap();
    }

    return Arrays.stream(text.toLowerCase().split("\\s+"))
        .collect(Collectors.groupingBy(
            Function.identity(),
            Collectors.counting()
        ));
}

// Test
// Input: "the quick brown fox jumps over the lazy dog the"
// Output: {the=3, quick=1, brown=1, fox=1, jumps=1, over=1, lazy=1, dog=1}
```

---

## Q: Validate parentheses

```
public boolean isValidParentheses(String s) {
    if (s == null || s.isEmpty()) {
        return true;
    }

    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs = Map.of(')', '(', '}', '{', ']', '[');

    for (char c : s.toCharArray()) {
        if (pairs.containsValue(c)) {
            // Opening bracket - push to stack
            stack.push(c);
        } else if (pairs.containsKey(c)) {
            // Closing bracket - check match
            if (stack.isEmpty() || stack.pop() != pairs.get(c)) {
                return false;
            }
        }
    }

    return stack.isEmpty();
}

// Test
// Input: "()[]{}" -> true
// Input: "([)]" -> false
// Input: "{[]}" -> true
```

---

# SECTION 4: BEHAVIORAL QUESTIONS

---

## Q: What would you do if your teammate is not delivering on time?

**Best Answer:**

"I'd approach this constructively in three steps:

**1. Understand First:** I'd have a private, non-judgmental conversation. Maybe they're blocked, unclear on requirements, or dealing with personal issues. I'd ask: 'I noticed the feature is taking longer than planned. Is there anything blocking you or any way I can help?'

**2. Offer Support:** If it's a technical blocker, I'd offer to pair program or review their approach. If it's unclear requirements, I'd help clarify with the product owner. If they're overloaded, I'd suggest we discuss with the team lead about redistributing work.

**3. Escalate if Needed:** If the pattern continues and affects the sprint, I'd raise it in the retrospective as a team issue—not calling anyone out, but discussing how we can better estimate and identify blockers early.

**Real Example:** A junior developer was struggling with a complex integration. Instead of waiting for them to miss the deadline, I proactively offered to review their code. Found they were overcomplicating it. A 30-minute pairing session unblocked them, and they delivered on time."

---

## Q: How do you handle tight deadlines?

**Best Answer:**

"I handle tight deadlines through prioritization, communication, and focused execution:

**1. Prioritize Ruthlessly:** I identify the MVP—what's the minimum that delivers value? I negotiate with stakeholders if needed. Not everything is equally important.

**2. Communicate Early:** If I foresee a risk, I raise it immediately—not on the deadline day. 'I can deliver features A and B by Friday. Feature C needs until Monday. Which is more critical?'

**3. Eliminate Distractions:** I block focused time, decline non-critical meetings, and avoid context switching. Deep work is more productive than long hours of interrupted work.

**4. Technical Shortcuts (Carefully):** I might take calculated shortcuts—simpler implementation, skip nice-to-haves, defer optimization. But I NEVER skip testing or create security holes.

**Real Example:** Before a major demo, we had 2 days to complete 5 days of work. I broke features into must-have vs nice-to-have. We delivered 3 critical features polished, rather than 5 features half-done. The demo was successful, and we completed the rest in the next sprint."

---

## Q: How do you explain a technical issue to a non-technical stakeholder?

**Best Answer:**

"I use analogies, focus on impact, and avoid jargon.

**Example Scenario:** A stakeholder asked why the website was slow. Instead of saying 'The database query is doing a full table scan because the index is missing'—

**My Explanation:** 'Imagine a library with 1 million books but no catalog system. To find one book, you'd have to check every shelf. That's what our database is doing. We need to add a catalog—which in tech terms is an index. It'll take 2 hours to implement, and the page will load 10x faster.'

**My Approach:**

1. **Use analogies** they can relate to (library, traffic, etc.)
2. **Focus on impact** ('users will see 3-second loads instead of 30 seconds')
3. **Give clear timelines** ('fixed by end of day')
4. **Avoid blame** ('we identified an optimization opportunity' not 'someone forgot to add an index')
5. **Offer options** if applicable ('we can do a quick fix today or a complete solution next week')

This builds trust—stakeholders feel informed without being overwhelmed."

---

# QUICK TIPS FOR BERRIBOT

**DO:**

- Speak clearly and confidently
- Use "I" not "we" for your contributions
- Follow Context → Action → Result format
- Give specific numbers (40% improvement, 3 services, etc.)
- Pause briefly before answering—shows thoughtfulness

**DON'T:**

- Read from notes (Berribot detects this)
- List technologies without context
- Give vague answers ("I worked on many things")
- Speak in monotone
- Ramble—keep answers 1-2 minutes max

**Power Phrases:**

- "I identified the problem and implemented..."
- "The impact was..."
- "I took ownership of..."
- "My specific contribution was..."
- "This reduced/improved/saved..."

---

Good luck with your Berribot assessment! □